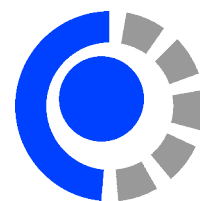




UNIVERSIDAD NACIONAL DEL COMAHUE
FACULTAD DE INFORMÁTICA



TESIS DE LICENCIATURA EN CIENCIAS DE LA COMPUTACIÓN

**Pruebas y análisis de algoritmos cuánticos para
optimización combinatoria**

Adriano Mauricio Lusso

Christian Nelson Gimenez

Alejandro Mata Ali

NEUQUÉN

ARGENTINA

2025

PREFACIO

Esta tesis es presentada como parte de los requisitos finales para optar al grado académico de *Licenciado en Ciencias de la Computación*, otorgado por la Universidad Nacional del Comahue, y no ha sido presentada previamente para la obtención de otro título en esta Universidad u otras. La misma es el resultado de la investigación llevada a cabo en el Departamento de Teoría de la Computación, de la Facultad de Informática, en el período comprendido entre noviembre de 2024 y agosto de 2025, bajo la dirección del Lic. Christian Nelson Gimenez y la codirección del Mg. Alejandro Mata Ali.

Adriano Mauricio Lusso
FACULTAD DE INFORMÁTICA
UNIVERSIDAD NACIONAL DEL COMAHUE
Neuquén, agosto de 2025.



UNIVERSIDAD NACIONAL DEL COMAHUE

Facultad de Informática

La presente tesis ha sido aprobada el día, mereciendo la calificación de

DEDICATORIAS

A Sawyer, el perruno otoñesco.

AGRADECIMIENTOS

A mi mamá y papá, por ser los principales responsables de mis logros. A mi hermano y mejor amigo, Rami, con quien comparto las mejores risas. A Valentina, por su gran compañía y apoyo en mis últimos años de la licenciatura. A mis amigos y amigas, que llenan mis fines de semana de alegría. A mis mentores, entre estos, Laura Cecchi, Christian Gimenez, Alejandro Mata Ali, Víctor Onofre, Alberto Maldonado y, por su guía y enseñanza. Y a todas las personas que, de una u otra forma, me acompañaron en este camino.

RESUMEN

Desde sus orígenes, la computación ha evolucionado tanto en sus fundamentos teóricos como en sus capacidades tecnológicas, permitiendo abordar problemas cada vez más complejos. No obstante, ciertos desafíos, como los que plantea la optimización combinatoria, continúan siendo ineficientes de resolver dentro del paradigma clásico. En este contexto, la computación cuántica, y en particular algoritmos como el Algoritmo Cuántico de Optimización Aproximada (*Quantum Approximate Optimisation Algorithm*) (QAOA), surgen como alternativas prometedoras. Esta tesis estudia la aplicabilidad de QAOA al Problema de Reasignación de Puestos de Trabajo (*Job Reassignment Problem*) (JRP), un problema con relevancia industrial directa que ha sido escasamente abordado en el ámbito de la investigación desde la perspectiva de la computación cuántica.

La propuesta consiste en implementar el JRP mediante QAOA y realizar simulaciones en entornos cuánticos sin ruido. Se define un conjunto de hiperparámetros para el algoritmo y se genera una muestra representativa de 105 instancias del JRP. Luego, se establecen métricas de rendimiento para evaluar la calidad de las soluciones obtenidas y la robustez del algoritmo ante distintos escenarios. Los datos recopilados son posteriormente analizados y comparados.

La motivación para explorar QAOA en este contexto es doble. Por un lado, aplicar este algoritmo a problemas combinatorios clásicos —como los de asignación— permite estudiar su desempeño y escalabilidad bajo restricciones realistas, propias de los dispositivos Cuánticos, Ruidosos y de Escala Intermedia (*Noisy Intermediate-Scale Quantum*) (NISQ): cantidad limitada de bit cuánticos, circuitos poco profundos y restricciones en la cantidad de iteraciones de optimización. Por otro lado, si bien las formulaciones clásicas de JRP pueden resolverse de forma eficiente, en la práctica industrial suelen abordarse con restricciones blandas, lo que complejiza el problema y reduce drásticamente la efectividad de los solucionadores clásicos. En contraste, en QAOA este tipo de restricciones pueden integrarse de manera natural gracias a su estructura variacional, evitando así los costos computacionales significativos que enfrentan los métodos clásicos.

Los resultados obtenidos muestran que QAOA es capaz de generar soluciones aproximadas de alta calidad para el JRP, alcanzando razones de aproximación promedio cercanas a 0,9 y mejoras de ganancia de hasta un 12 %. Se observó una relación consistente entre el incremento del parámetro de profundidad p y el aumento del rendimiento, lo cual también implica una mayor cantidad de iteraciones del optimizador clásico embebido dentro de QAOA. Además, los experimentos destacan la influencia del desbalance entre trabajadores y puestos vacantes, la efectividad de la inicialización *Trotterised Quantum Annealing* (TQA), y el potencial del Aprendizaje por Transferencia para reducir los tiempos de optimización sin comprometer la calidad de las soluciones.

ABSTRACT

Since its inception, computing has evolved in both theoretical and technological dimensions, enabling the resolution of increasingly complex problems. However, certain challenges — such as those posed by combinatorial optimisation — remain inefficient to solve within the classical paradigm. In this context, quantum computing, and in particular algorithms such as the Quantum Approximate Optimisation Algorithm (QAOA), emerge as promising alternatives. This thesis investigates the applicability of QAOA to the Job Reassignment Problem (JRP), a problem with direct industrial relevance that has received scant attention in the research literature of a quantum computing perspective.

The proposed approach consists of implementing the JRP using QAOA and running simulations in noiseless quantum environments. A set of hyperparameters is defined for the algorithm, and a representative sample of 105 JRP instances is generated. Performance metrics are established to evaluate the quality of the solutions and the algorithm’s robustness across different scenarios. The collected data is then analysed and compared.

The motivation for exploring QAOA in this context is twofold. Firstly, applying the algorithm to classical combinatorial problems — such as assignment problems — provides insight into its performance and scalability under realistic constraints typical of Noisy Intermediate-Scale Quantum (NISQ) devices: limited qubit counts, shallow circuits, and restrictions on the number of optimisation iterations. Secondly, although classical formulations of the JRP can be solved efficiently, in industrial practice they are often tackled with soft constraints, which considerably complicates the problem and drastically reduces the effectiveness of classical solvers. By contrast, QAOA can naturally incorporate such constraints thanks to its variational structure, thus avoiding the significant computational overhead faced by classical methods.

The results show that QAOA can produce high-quality approximate solutions for the JRP, achieving average approximation ratios close to 0.9 and up to 12% improvement in gain. A consistent relationship was found between increasing the QAOA depth parameter p and improved performance, although this also results in a higher number of iterations of the classical optimizer embedded within the algorithm. Additionally, the experiments highlight the importance of the imbalance between workers and vacant positions, the effectiveness of Trotterised Quantum Annealing (TQA) initialisation, and the potential of transfer learning to reduce optimisation time without compromising solution quality.

Índice general

1. Introducción	1
1.1. Objetivos	2
1.1.1. Objetivo General	2
1.1.2. Objetivos Específicos	2
1.2. Estructura de la Tesis	3
2. Marco Teórico	5
2.1. Introducción a la Computación e Información Cuántica	5
2.1.1. Notación de Dirac	5
2.1.2. El Bit Cuántico	6
2.1.3. La Esfera de Bloch	6
2.1.4. Compuertas de un Qubit	7
2.1.5. Medición de un Qubit	9
2.1.6. Fase Local y Global	9
2.1.7. Sistemas de Múltiples Qubits	10
2.1.8. Circuitos Cuánticos	11
2.1.9. El Operador Hamiltoniano	12
2.1.10. El Valor Esperado	13
2.1.11. Estado Actual del Hardware Cuántico	14
2.2. Modelo QUBO	15
2.2.1. Penalizaciones Cuadráticas	15
2.2.2. Equivalencia al Modelo Ising	16
2.3. Algoritmo de Optimización Aproximada Cuántica	17
2.3.1. El Ansatz Cuántico	17
2.3.2. Optimización de los Parámetros Variacionales	18
2.3.3. Inicialización TQA	19
2.3.4. Aprendizaje por Transferencia	19
2.4. Problema de Reasignación de Puestos de Trabajo	19
2.4.1. Definición de los Puntajes S_{ij}	20
2.4.2. Formulación QUBO	20
2.4.3. Comparación y Limitaciones de Enfoques de Resolución Clásica	21
3. Diseño de los Procedimientos	23
3.1. Flujo de Trabajo Principal	23
3.1.1. Configuraciones de Entrada	26
3.1.2. Métricas de Rendimiento Algorítmico	27
3.2. Procedimientos de Postprocesamiento	27
3.2.1. Generación de Curvas MR y PM	28
3.2.2. Aplicación de Aprendizaje por Transferencia	29

4. Implementación de los Procedimientos	31
4.1. Herramientas Utilizadas	31
4.1.1. Comparación entre Herramientas para QAOA	31
4.1.2. Introducción a la Librería OpenQAOA	32
4.2. Entorno de Ejecución	33
4.3. Estructuras de Datos Principales	33
4.4. Secciones de Código Principales	33
5. Resultados y Discusiones	35
5.1. Razón de Aproximación	35
5.2. Diferencia Ising	37
5.3. Cantidad de Iteraciones del Optimizador	39
5.4. Mejora de Ganancia	41
5.5. Curvas MR y PM	42
5.6. Razón de Aproximación de Aprendizaje por Transferencia	44
5.7. Resumen de las Discusiones	45
6. Conclusiones	47
6.1. Trabajos Futuros	48
6.2. Publicaciones	48
Referencias Bibliográficas	50
A. Estructuras de datos de la implementación	57
B. Métodos y funciones de la implementación	61

Índice de figuras

2.1. Representación de la Esfera de Bloch.	7
2.2. Compuertas cuánticas de un qubit más utilizadas.	8
2.3. Esfera de Bloch con 6 estados cuánticos graficados.	10
2.4. Circuito cuántico genérico con algunas compuertas simples.	12
2.5. Diseño de la arquitectura algorítmica de QAOA.	17
3.1. Diseño de cuatro capas del flujo de trabajo principal, conectado con el postproce- samiento y el análisis de resultados.	24
3.2. Diseño de la primer capa: Muestreo de configuraciones.	24
3.3. Diseño de la segunda capa: Muestreo de instancias.	24
3.4. Diseño de la tercer capa: Analizador de rendimiento algorítmico.	25
3.5. Diseño de uno de los componentes de la cuarta capa: Solucionador QAOA	25
3.6. Aplicación de Aprendizaje por Transferencia.	29
5.1. Razón de aproximación de las 105 instancias de JRP.	36
5.2. Estadísticas de razón de aproximación.	36
5.3. Diferencia Ising de las 105 instancias de JRP.	38
5.4. Estadísticas de diferencia Ising.	38
5.5. Cantidad de iteraciones del optimizador para las 105 instancias de JRP.	40
5.6. Estadísticas de la cantidad de iteraciones del optimizador.	40
5.7. Mejora de ganancia de las 105 instancias de JRP.	41
5.8. Estadísticas de mejora de ganancia.	42
5.9. Curvas MR.	42
5.10. Curvas PM.	43
5.11. Razón de aproximación del Aprendizaje por Transferencia.	44

Índice de Código

A.1. Ejemplo de diccionario de resultados generado luego de utilizar OpenQAOA. . . .	58
A.2. Ejemplo de archivo JSON que almacena una submuestra de cinco ejecuciones QAOA implementado JRP.	59
A.3. Ejemplo de archivo JSON que almacena una ejecución de aplicación de Aprendizaje por Transferencia.	60
B.1. Implementación del Solucionador QAOA.	62
B.2. Configuraciones para inicializar instancia JRP implementadas en Python.	63
B.3. Hiperparámetros para QAOA implementados en Python.	64
B.4. Estructura iterativa para ejecutar el flujo de trabajo principal con cada configuración de entrada.	65
B.5. Implementación del método <code>sample_workflows_with_fixedInstances</code>	66
B.6. Implementación del método <code>__run_workflow</code>	67
B.7. Instanciación de <code>EvaluationQAOASolver</code> para el postprocesamiento.	67
B.8. Implementación de la clase <code>EvaluationQAOASolver</code> , encargada de realizar la aplicación de Aprendizaje por Transferencia.	68

Capítulo 1

Introducción

Desde la concepción de la Máquina de Turing como modelo abstracto de cómputo universal, las ciencias de la computación han visto un avance significativo en el desarrollo de hardware y software [1]. Este progreso ha impactado de forma directa en la sociedad permitiendo, entre otras cosas, la creación de redes de intercomunicación, el almacenamiento de grandes volúmenes de datos y la realización de cálculos matemáticos complejos. Aun así, existen limitaciones respecto a qué problemas se pueden modelar eficientemente con la Máquina de Turing. Entre éstos, se encuentran algunos de los problemas de optimización combinatoria, en los que se busca una solución óptima en un espacio de soluciones candidato que crece rápidamente en función del tamaño del problema a resolver [2]. Motivado por las limitaciones propias del paradigma clásico, se definió el paradigma de la computación cuántica, que busca extender los límites de los problemas eficientemente tratables [3]. Este paradigma tiene su base teórica en la mecánica cuántica y presenta principios, tales como el de la superposición y el entrelazamiento, que no forman parte de modelos tradicionales de cómputo [3].

En la última década, se han llevado a cabo grandes avances en cuanto al desarrollo e investigación de la computación cuántica. Uno de los grandes objetivos es diseñar algoritmos que sean de utilidad con solo unos cientos de qubits (bits cuánticos) y que puedan lograr un desempeño decente en el hardware actual, a pesar de los errores que se producen en la interacción del computador con el ambiente que lo rodea (fenómeno denominado “ruido”). También, se trabaja en el diseño de algoritmos *quantum-inspired* [4], que son algoritmos de cómputo clásico que se inspiran en el formalismo matemático de la mecánica cuántica.

Debido al problema del ruido en el hardware, se considera que el área se encuentra actualmente en la Era de la Computación Cuántica Ruidosa y de Escala Intermedia (*Noisy Intermediate-Scale Quantum*) (NISQ) [5]. Para solventar esto se busca mejorar la fabricación y el control de hardware para que éste sea capaz de ejecutar algoritmos cuánticos a gran escala y sin errores en el cómputo intermedio. Esta ansiada era es conocida como Computación Cuántica Tolerante a Fallo (*Fault-Tolerant Quantum Computing*) (FTQC) [6]. Aun así, la estimación de su llegada por parte de la academia es imprecisa.

La limitada cantidad de computadoras cuánticas en la era NISQ, así como los altos costos de acceso a las mismas, pueden significar una barrera a la hora de introducir a nuevas instituciones e investigadores. Como solución temporal, se utilizan simuladores de cómputo cuántico [7], que son módulos de software ejecutados en hardware clásico, y que logran simular las operaciones realizadas sobre un circuito cuántico. Estas simulaciones pueden realizarse simulando el ruido propio del hardware real, o bien ejecutando las operaciones de forma ideal y sin errores de cómputo. Aún así, la finalidad de los simuladores es principalmente la investigación accesible, ya que al estar soportados sobre un hardware clásico, se pierde toda ventaja computacional que existiría en un verdadero dispositivo cuántico.

Dentro de las áreas de posible aplicación de la computación cuántica, la resolución de problemas de optimización combinatoria ha presentando un gran potencial [8, 9], donde el Algo-

ritmo Cuántico de Optimización Aproximada (*Quantum Approximate Optimisation Algorithm*) (QAOA) [10], el Templado Cuántico [11] y el *Quantum Variational Eigensolver* (QVE) [12] dominan el eje de la investigación. También es importante destacar la presencia de algoritmos *quantum-inspired* como las Redes Tensoriales [13, 14]. Tanto problemas clásicos (como el Problema del Máximo Corte [15] y el Problema de la Mochila [16]) como problemas de aplicación industrial han sido implementados y analizados con estos nuevos procedimientos [17, 18, 19].

Para el desarrollo de esta tesis se presenta el Problema de Reasignación de Puestos de Trabajo (*Job Reassignment Problem*) (JRP) [20], dado que el mismo no ha sido investigado en profundidad con algoritmos cuánticos, a pesar de poder ofrecer una aplicabilidad industrial directa. Dicho problema introduce una organización laboral donde n trabajadores están asignados a sus respectivos n puestos de trabajo. Dadas ciertas circunstancias organizativas, aparecen disponibles m puestos de trabajo vacantes de alta prioridad. En base a la prioridad de todos los trabajos y las afinidades de los trabajadores con los mismos, se debe analizar cuáles reasignaciones a los puestos vacantes son convenientes, de forma que se maximice la realización de trabajos de alta prioridad y la satisfacción de los trabajadores respecto al trabajo que realizan.

En esta tesis se propone implementar JRP haciendo uso del ya mencionado algoritmo QAOA y ejecutar simulaciones en entornos sin ruido cuántico. Para ello, se determinarán un conjunto de hiperparámetros para el algoritmo seleccionado, y se realizará un muestreo de instancias de JRP. También, se determinará un conjunto de métricas de interés para analizar su rendimiento respecto a la calidad de las soluciones obtenidas y su robustez ante diferentes instancias. En base a las muestras tomadas, se compararán los resultados y se discutirá sobre los mismos.

La motivación para explorar QAOA en este contexto es doble. Por un lado, aplicar este algoritmo a problemas combinatorios clásicos, como los de asignación, permite analizar su rendimiento y escalabilidad en escenarios realistas, caracterizados por las limitaciones típicas de los dispositivos NISQ: un número reducido de qubits, circuitos poco profundos y un límite en las iteraciones de optimización. Por otro lado, si bien las formulaciones clásicas de JRP pueden resolverse de forma eficiente, en la práctica industrial suelen abordarse con restricciones blandas [21, 22], lo que complejiza el problema y reduce drásticamente la efectividad de los solucionadores clásicos [23]. En contraste, en QAOA este tipo de restricciones pueden integrarse de manera natural gracias a su estructura variacional, evitando así los costos computacionales significativos que enfrentan los métodos clásicos [24].

1.1. Objetivos

1.1.1. Objetivo General

El objetivo de esta tesis es realizar pruebas del Algoritmo Cuántico de Optimización Aproximada (QAOA) para el Problema de Reasignación de Puestos de Trabajo (JRP). En base a estos resultados, se analizará su rendimiento respecto a la calidad de las soluciones obtenidas y su robustez ante diferentes instancias.

1.1.2. Objetivos Específicos

- O1. Implementar JRP en QAOA.
- O2. Determinar un conjunto de hiperparámetros para el algoritmo y problema seleccionados.
- O3. Realizar un muestreo de instancias de JRP en QAOA.
- O4. Comparar los resultados respecto a las métricas que describen el rendimiento algorítmico, para discutir sobre los mismos.

1.2. Estructura de la Tesis

Además del presente capítulo introductorio, esta tesis se compone de 5 más. A continuación se describe brevemente el contenido de cada uno de ellos:

Capítulo 2: Marco Teórico

Se presentan los conceptos fundamentales de la computación cuántica, incluyendo la notación de Dirac, qubits, circuitos y compuertas cuánticas. Luego, se describe el modelo QUBO como un formalismo para representar problemas de optimización combinatoria susceptibles de codificación en circuitos cuánticos. Seguidamente, se detalla QAOA, que utiliza dicha formulación para resolver problemas de optimización combinatoria. Finalmente, se expone el problema JRP, describiendo sus componentes y su codificación mediante el modelo QUBO.

Capítulo 3: Diseño de los Procedimientos

En este capítulo se define el flujo de trabajo principal, el cual genera los principales resultados de esta tesis. Se incluyen sus datos de entrada, funcionamiento interno y métricas de análisis resultantes. También se detalla un conjunto de procedimientos de postprocesamiento, que permiten obtener métricas de análisis adicionales.

Capítulo 4: Implementación de los Procedimientos

En este capítulo se presentan las herramientas seleccionadas y el entorno de ejecución configurado para implementar los procedimientos diseñados. Se incluye una discusión sobre diferentes librerías que implementan QAOA, junto a las ventajas y desventajas que fundamentan cual fue la elegida. Finalmente, se muestran ejemplos de código sobre las estructuras de datos principales, así como de las funciones y métodos programados.

Capítulo 5: Resultados y Discusiones

Se presentan los resultados del flujo de trabajo principal y de los procedimientos de postprocesamiento, junto a una discusión de estos.

Capítulo 6: Conclusiones

Por último, se exhiben las conclusiones de esta tesis y los resultados obtenidos. Además, se proponen líneas de desarrollo para trabajos a futuro.

Capítulo 2

Marco Teórico

En este capítulo se desarrolla el marco teórico necesario para la comprensión de esta de tesis. El mismo se compone de cuatro secciones principales. La primera, brinda una introducción a la computación e información cuántica. Esta sección detalla los elementos básicos, explicando cómo se almacena la información, como utilizarla e interpretarla para un correcto uso de la misma. Este primer apartado toma como referencia principal al libro *Quantum Computation and Quantum Information* de Michael A. Nielsen e Isaac L. Chuang [3].

Luego, se introduce al modelo QUBO como marco de formulación de problemas de optimización combinatoria. Posteriormente se detallan la tercer y cuarta secciones, que presentan el funcionamiento del algoritmo QAOA y el problema JRP respectivamente.

2.1. Introducción a la Computación e Información Cuántica

La computación e información cuántica son dos campos de estudio sobre las tareas de procesamiento de información que pueden ser logradas usando sistemas cuánticos. Son campos multidisciplinarios que se han nutrido de otras áreas del conocimiento, entre las que se encuentran la mecánica cuántica, las ciencias de la computación y la teoría de la información.

2.1.1. Notación de Dirac

Durante el desarrollo de esta tesis, se utilizará la notación de Dirac, creada por Paul Dirac en 1939 [25]. Esta notación busca facilitar la representación de los cálculos de álgebra lineal frecuentes en la mecánica cuántica. Desde su publicación, tomó gran popularidad como estándar y fue adoptada por otras áreas como la computación cuántica. Se describen en la Tabla 2.1 algunos componentes de la notación de Dirac.

Notación	Descripción
z^*	Conjugado complejo del número complejo z .
$ \phi\rangle$	Vector. Se lo denomina <i>ket</i> .
$\langle\phi $	Vector dual a $ \phi\rangle$. Se lo denomina <i>bra</i> .
$\langle\phi \varphi\rangle$	Producto interno entre los vectores $ \phi\rangle$ y $ \varphi\rangle$.
$ \phi\rangle \otimes \varphi\rangle$	Producto tensorial entre los vectores $ \phi\rangle$ y $ \varphi\rangle$.
$ \phi\rangle \varphi\rangle$	Notación reducida al producto tensorial entre los vectores $ \phi\rangle$ y $ \varphi\rangle$.
A^*	Conjugado complejo de la matriz A .
A^T	Transpuesta de la matriz A .
$A^\dagger$	Transpuesta conjugada de la matriz A .
$\langle\phi A \varphi\rangle$	Producto interno entre $ \phi\rangle$ y $A \varphi\rangle$.

Tabla 2.1: Componentes de la notación de Dirac utilizados en esta tesis.

2.1.2. El Bit Cuántico

La computación e información cuántica, si bien comparten ciertos conceptos con las ciencias de la computación, también presentan características propias que las distinguen. Se define así como **bit cuántico**, o **qubit**, a la unidad mínima de información cuántica [3]. Al igual que al bit, se lo denota como un objeto matemático abstracto que sirve de representación formal para un objeto físico. En el caso del qubit, su teoría formal física es la mecánica cuántica.

Un qubit es capaz de almacenar un estado cuántico, siendo los más sencillos de comprender $|0\rangle$ y $|1\rangle$. Estos corresponden respectivamente los estados 0 y 1 de un bit clásico. En la Ecuación (2.1), se realiza una primera formalización del estado arbitrario de un qubit $|\psi\rangle$ como una combinación lineal de $|0\rangle$ y $|1\rangle$, también nombrada **superposición**.

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle \quad \alpha, \beta \in \mathbb{C} \quad (2.1)$$

Otra representación usual utilizada para un qubit es la de un vector en un espacio vectorial complejo bidimensional, presentada en la Ecuación (2.2). Si se utilizara esta notación en la computación clásica, los estados 0 y 1 podrían representarse como lo indica la Ecuación (2.3). En cambio, un qubit, al incorporar la superposición, también permite representar todo el resto de vectores en el dominio de los complejos.

$$\begin{pmatrix} \alpha \\ \beta \end{pmatrix} \quad \alpha, \beta \in \mathbb{C} \quad (2.2)$$

$$|0\rangle = \text{bit } 0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, |1\rangle = \text{bit } 1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (2.3)$$

Los estados especiales $|0\rangle$ y $|1\rangle$ se conocen como la **base computacional** y forman una base ortonormal con respecto al espacio vectorial. Se puede generalizar aún más la definición de un qubit al definirlo como una combinación lineal de sus estados base ($|0\rangle$ y $|1\rangle$ en el caso de un qubit representado en la base computacional). Existen otras bases relevantes en el campo de la computación cuántica, pero en el marco de los temas tratados en esta tesis, se limitará a hablar principalmente de la base computacional.

2.1.3. La Esfera de Bloch

Se ha explicado ya la representación de un qubit como superposición lineal y como vector. Esta última puede ser visualizada en un espacio vectorial. Se presenta entonces a la Esfera de Bloch, que facilita la visualización del estado de un qubit representado como un vector [26]. Para su explicación detallada, se reescribirá la Ecuación (2.1) como lo indica la Ecuación (2.4), siendo ésta la definición más completa del estado cuántico de un qubit arbitrario $|\psi\rangle$ que se usará en esta tesis.

$$|\psi\rangle = e^{i\gamma} \left(\cos \frac{\theta}{2} |0\rangle + e^{i\phi} \sin \frac{\theta}{2} |1\rangle \right) \quad \gamma, \theta, \phi \in \mathbb{R} \quad (2.4)$$

Los factores $\cos \frac{\theta}{2}$ y $\sin \frac{\theta}{2}$ son las **amplitudes** asociadas a cada estado base $|0\rangle$ y $|1\rangle$ respectivamente. Los factores $e^{i\gamma}$ y $e^{i\phi}$ son la **fase global** y **fase local** del estado cuántico. En la Sección 2.1.6, se explicará por qué el factor $e^{i\gamma}$ puede ser ignorado y la importancia que tiene el factor $e^{i\phi}$. Así, la fórmula puede simplificarse como lo indica la Ecuación (2.5).

$$|\psi\rangle = \cos \frac{\theta}{2} |0\rangle + e^{i\phi} \sin \frac{\theta}{2} |1\rangle, \quad \theta, \phi \in \mathbb{R} \quad (2.5)$$

En la Figura 2.1, los reales θ y ϕ permiten representar al qubit $|\psi\rangle$ como un vector $\vec{a}$ en la Esfera de Bloch. Se observa que θ indica un ángulo de rotación alrededor del eje **Y**. A su vez, ϕ indica un ángulo de rotación alrededor del eje **Z**. Todo vector representado en la Esfera de Bloch debe tener módulo 1. Si se quisiera representar un bit clásico en la Esfera de Bloch, el mismo

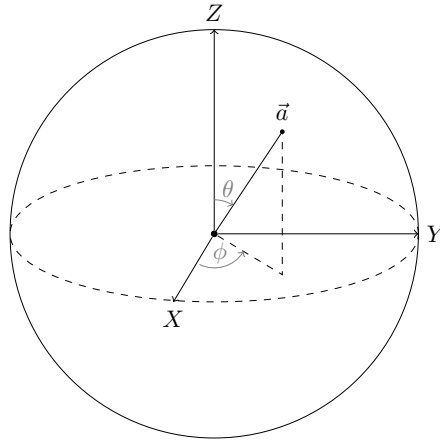


Figura 2.1: Representación de la Esfera de Bloch, donde se ejemplifica un qubit como un vector $\vec{a}$, en función de los ángulos de rotación θ y ϕ .

solo podría posicionarse en los polos del eje $\mathbf{Z}$, correspondientes a los estados $|0\rangle$ y $|1\rangle$. Un qubit, por el contrario, puede posicionarse alrededor de toda la esfera.

Si bien esta representación puede facilitar en gran medida el entendimiento de un qubit, se encuentra limitada por el hecho de que no existe una generalización sencilla de la Esfera de Bloch para representar sistemas de múltiples qubits.

2.1.4. Compuertas de un Qubit

Para operar sobre un qubit, se define el concepto de compuerta. Su funcionamiento es similar a las compuertas de la computación clásica. Matemáticamente, se representan como matrices cuadradas de segundo orden. En la Esfera de Bloch, se representan como rotaciones del vector estado alrededor de la misma. Dado un qubit representado como *ket*, $|\phi\rangle$, y una compuerta U , se puede aplicar la compuerta sobre el qubit usando una multiplicación matriz-vector $U|\phi\rangle$. En cambio, si el qubit se representa como un *bra*, se usa una multiplicación vector-matriz $\langle\phi|U^\dagger$ [3].

Al igual que los qubits, las compuertas deben cumplir con sus propias características. Un qubit debe estar normalizado, es decir, que $|\alpha|^2 + |\beta|^2 = 1$. Por lo tanto, el mismo qubit debe mantener esta propiedad luego de ser afectado por una compuerta. Esto puede ser garantizado con la propiedad de **unitariedad**, que describe que dada una compuerta U , se debe cumplir que $U^\dagger U = I$. Esto es, que el producto de la transpuesta conjugada de U y de U sea igual a la matriz identidad.

En la Figura 2.2 se presentan las compuertas de un qubit más utilizadas. Cada fila corresponde a la información de una compuerta, indicando su representación como matriz, el efecto de la misma en los estados cuánticos $|0\rangle$ y $|1\rangle$ y su representación en un circuito cuántico.

La compuerta identidad no realiza efecto alguno sobre el estado cuántico. La compuerta X realiza una rotación de 180° alrededor del eje $\mathbf{X}$ de la Esfera de Bloch, actuando como una compuerta *NOT* clásica en los estados $|0\rangle$ y $|1\rangle$. La compuerta Z realiza una rotación de 180° alrededor del eje $\mathbf{Z}$, mientras que las compuertas S y T realizan una rotación de 90° y 45° respectivamente sobre el mismo eje. La compuerta Y realiza una rotación de 180° alrededor del eje $\mathbf{Y}$ y es equivalente a aplicar iXZ en el qubit.

Por otro lado, la compuerta Hadamard, notada como H , realiza una rotación de 90° sobre el eje $\mathbf{Y}$ seguido de una rotación de 180° alrededor del eje $\mathbf{X}$. Esta última es una de las operaciones más poderosas del cómputo cuántico ya que se utiliza en varios algoritmos para, entre otras cosas, lograr una **superposición igualitaria de estados base**, es decir, un estado cuántico donde todas las amplitudes de sus estados base coinciden. Su importancia radica en la comparación con el paradigma clásico, ya que mientras un bit puede tener únicamente el valor 0 o 1 codificados en

$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$ 0\rangle \mapsto 0\rangle \quad +\rangle \mapsto +\rangle$ $ 1\rangle \mapsto 1\rangle \quad -\rangle \mapsto -\rangle$	$\text{---}[I]\text{---}$
$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	$ 0\rangle \mapsto 1\rangle \quad +\rangle \mapsto +\rangle$ $ 1\rangle \mapsto 0\rangle \quad -\rangle \mapsto - -\rangle$	$\text{---}[X]\text{---}$
$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$	$ 0\rangle \mapsto i 1\rangle \quad +\rangle \mapsto -i -\rangle$ $ 1\rangle \mapsto -i 0\rangle \quad -\rangle \mapsto i +\rangle$	$\text{---}[Y]\text{---}$
$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	$ 0\rangle \mapsto 0\rangle \quad +\rangle \mapsto -\rangle$ $ 1\rangle \mapsto - 1\rangle \quad -\rangle \mapsto +\rangle$	$\text{---}[Z]\text{---}$
$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$	$ 0\rangle \mapsto +\rangle \quad +\rangle \mapsto 0\rangle$ $ 1\rangle \mapsto -\rangle \quad -\rangle \mapsto 1\rangle$	$\text{---}[H]\text{---}$
$S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$	$ 0\rangle \mapsto 0\rangle$ $ 1\rangle \mapsto i 1\rangle$	$\text{---}[S]\text{---}$
$T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$	$ 0\rangle \mapsto 0\rangle$ $ 1\rangle \mapsto e^{i\pi/4} 1\rangle$	$\text{---}[T]\text{---}$
$RZ(\theta) = \begin{bmatrix} e^{-i\frac{\theta}{2}} & 0 \\ 0 & e^{i\frac{\theta}{2}} \end{bmatrix}$	$ 0\rangle \mapsto e^{-i\frac{\theta}{2}} 0\rangle$ $ 1\rangle \mapsto e^{i\frac{\theta}{2}} 1\rangle$	$\text{---}[RZ(\theta)]\text{---}$
$RX(\theta) = \begin{bmatrix} \cos(\frac{\theta}{2}) & -i \cdot \sin(\frac{\theta}{2}) \\ -i \cdot \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{bmatrix}$	$ 0\rangle \mapsto \cos(\frac{\theta}{2}) 0\rangle - i \cdot \sin(\frac{\theta}{2}) 1\rangle$ $ 1\rangle \mapsto -i \cdot \sin(\frac{\theta}{2}) 0\rangle + \cos(\frac{\theta}{2}) 1\rangle$	$\text{---}[RX(\theta)]\text{---}$

Figura 2.2: Compuertas cuánticas de un qubit más utilizadas.

un momento dado, un qubit en una superposición igualitaria de estados base (asumiendo que se trabaja sobre la base computacional $\{|0\rangle, |1\rangle\}$) tiene ambos valores codificados al mismo tiempo. Dos estados cuánticos conocidos, presentados en las Ecuaciones (2.6) y (2.7), pueden ser logrados haciendo uso de la compuerta H sobre los qubits $|0\rangle$ y $|1\rangle$ respectivamente. Tal es su importancia que $\{|+\rangle, |-\rangle\}$ es el conjunto de estados base de la **base de Hadamard** [27]. En las definiciones de $|+\rangle$ y $|-\rangle$, los términos $1 \cdot \frac{1}{\sqrt{2}}$ y $(-1) \cdot \frac{1}{\sqrt{2}}$ permiten observar fácilmente las amplitudes y la fase local de los estados cuánticos.

$$H|0\rangle = |+\rangle = \frac{1}{\sqrt{2}}|0\rangle + 1 \cdot \frac{1}{\sqrt{2}}|1\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle \quad (2.6)$$

$$H|1\rangle = |-\rangle = \frac{1}{\sqrt{2}}|0\rangle + (-1) \cdot \frac{1}{\sqrt{2}}|1\rangle = \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle \quad (2.7)$$

Finalmente, se presentan las compuertas con rotación parametrizada, que permiten indicar un ángulo θ sobre el que rotar respecto a alguno de los ejes de la Esfera de Bloch. De esta forma, $RX(\theta)$ y $RZ(\theta)$ realizan una rotación de ángulo θ sobre el eje X y Z respectivamente. Éstas son dos de las compuertas utilizadas en la implementación del Algoritmo Cuántico de Optimización Aproximada.

2.1.5. Medición de un Qubit

Además de la forma de operar con un qubit, es importante explicar el mecanismo por el cual se puede acceder a la información almacenada. En un bit clásico, se puede leer directamente el valor guardado, siendo éste 0 o 1. Una computadora clásica realiza esto constantemente al acceder a sus múltiples unidades de memoria. En cambio, para un qubit, no se puede obtener de manera directa su estado cuántico, es decir, sus amplitudes y fase local. La mecánica cuántica plantea que solo puede obtenerse una cantidad restringida de información del estado haciendo uso de **mediciones cuánticas**. Éstas se describen como una colección de **operadores de medición** $\{M_m\}$ que actúan sobre el sistema cuántico $|\psi\rangle$ con el subíndice m refiriendo a una de las posibles salidas de la medición [3]. La probabilidad de medir m para $|\psi\rangle$ está dada por la Ecuación (2.8). Tras realizar una medición, el estado del sistema evolucionará a $|\phi_m\rangle$, definido por la Ecuación (2.9). Esto quiere decir que, a diferencia de la computación clásica, en la computación cuántica la operación destinada a la lectura de información genera un cambio en el estado almacenado.

$$p(m) = \langle\psi| M_m^\dagger M_m |\psi\rangle \quad (2.8)$$

$$|\phi_m\rangle = \frac{M_m |\psi\rangle}{\sqrt{\langle\psi| M_m^\dagger M_m |\psi\rangle}} \quad (2.9)$$

En base a esta definición general, se puede especificar la **medición de un qubit con respecto a la base computacional**. Esta medición tiene dos posibles salidas 0 y 1 definidas por los operadores de medición $M_0 = |0\rangle\langle 0|$ y $M_1 = |1\rangle\langle 1|$. Suponiendo un estado $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, la probabilidad de obtener la salida 0 se define en la Ecuación (2.10).

$$p(0) = \langle\psi| M_0^\dagger M_0 |\psi\rangle = |\alpha|^2 \quad (2.10)$$

Similarmente, la probabilidad de obtener la salida 1 es $p(1) = |\beta|^2$. A este efecto se lo conoce como **colapso**, ya que al realizar una medición respecto a la base computacional, la superposición de estados base de $|\psi\rangle$ colapsa a uno de sus estado base. Es decir, el estado cuántico evoluciona a $|0\rangle$ o a $|1\rangle$ con las probabilidades previamente mencionadas.

El efecto del colapso tras una medición respecto a la base computacional es una limitación a tener en cuenta en ciertos contextos ya que, como se mencionó previamente, medir un estado cuántico implica que éste cambie. Si se quisiera realizar un muestreo de n mediciones sobre $|\psi\rangle$, sería necesario volver a preparar el mismo estado n veces, ya que $|\psi\rangle$ habría mutado tras cada una de las mediciones.

Es interesante plantear, de manera informal, la medición de un qubit como la tirada de una moneda imperfecta. Mientras el qubit no es medido, se lo puede pensar como una moneda imperfecta girando en el aire. En el momento que se realiza la medición, el qubit colapsará a uno de sus estados base ($|0\rangle$ o $|1\rangle$), al igual que la moneda imperfecta (cara o cruz). La propiedad de imperfección está relacionada a que las probabilidades de cara o cruz no son uniformes, sino que uno de los lados puede tener más probabilidad de caer hacia arriba. Si se tratara del caso particular de una moneda con la misma probabilidad de ocurrencia en ambas caras, su representación con un qubit sería $|+\rangle$ o $|-\rangle$.

2.1.6. Fase Local y Global

En la Ecuación (2.4), se plantea una de las definiciones más completas para el estado cuántico de un qubit arbitrario $|\phi\rangle$. En la misma, se pueden distinguir los estados base $|0\rangle$ y $|1\rangle$, las amplitudes $\cos\frac{\theta}{2}$ y $\sin\frac{\theta}{2}$ y los factores $e^{i\gamma}$ y $e^{i\phi}$, conocidos como **fase global** y **fase local** respectivamente. El término **fase** es comúnmente usado en la mecánica cuántica y, junto a las amplitudes, se encarga de caracterizar a un sistema cuántico.

Como formalismo matemático, un estado cuántico $|\psi\rangle$ es diferente a otro estado $e^{i\gamma}|\psi\rangle$, donde $e^{i\gamma}$ es la fase global. Pero, en términos físicos, se considera que la fase global no tiene efecto alguno

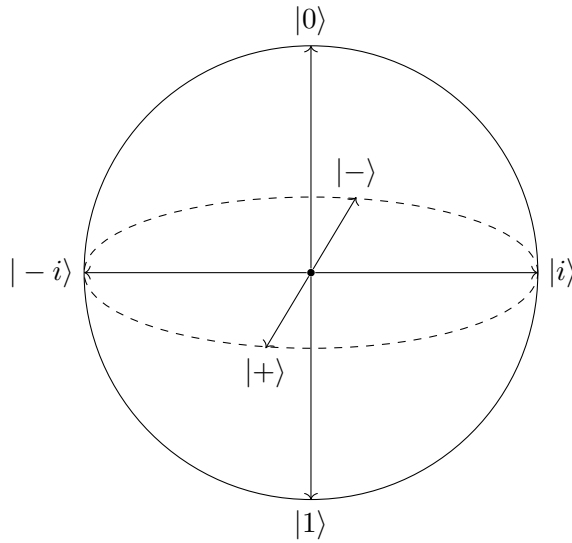


Figura 2.3: Esfera de Bloch donde se grafican 6 estados cuánticos ampliamente conocidos y notados por sí mismos. Estos son $|0\rangle$, $|1\rangle$, $|+\rangle$, $|-\rangle$, $|i\rangle$ y $|-i\rangle$.

en el estado cuántico. Esto se debe a que las estadísticas obtenidas de la medición de un estado y del otro resultan ser las mismas. En la Ecuación (2.11), suponiendo un operador de medición M_m , se observan las probabilidades de medición de la salida m para $|\psi\rangle$. En la Ecuación (2.12), la misma medición es realizada sobre $e^{i\gamma}|\psi\rangle$. Se observa que, indiferentemente de cual sea la fase global $e^{i\gamma}$, las mediciones de $|\psi\rangle$ y $e^{i\gamma}|\psi\rangle$ son equivalentes. Debido a esto, se plantea la equivalencia $|\psi\rangle \sim e^{i\gamma}|\psi\rangle$ entre los estados, permitiendo así ignorar $e^{i\gamma}$.

$$P(m) = \langle \psi | M_m^\dagger M_m | \psi \rangle \quad (2.11)$$

$$P(m) = \langle \psi | e^{-i\gamma} M_m^\dagger M_m e^{i\gamma} | \psi \rangle = \langle \psi | M_m^\dagger e^{-i\gamma} e^{i\gamma} M_m | \psi \rangle = \langle \psi | M_m^\dagger M_m | \psi \rangle \quad (2.12)$$

El factor de la fase local, como muestra la Ecuación (2.5), se encuentra multiplicando a la amplitud $\sin \frac{\theta}{2}$. A diferencia de la fase global, la fase local multiplica solo a una de las amplitudes [28]. Estados cuánticos con diferente fase local no son equivalentes entre sí. Previamente ya se han introducido a los estados $|+\rangle$ y $|-\rangle$. Estos difieren entre sí en su fase local. Otros estados cuánticos que difieren entre sí por su fase local son $|i\rangle$ y $|-i\rangle$, presentados en las Ecuaciones (2.13) y (2.14). En la Figura 2.3, se muestra una Esfera de Bloch junto a seis estados cuánticos ampliamente conocidos y notados por sí mismos: $|0\rangle$, $|1\rangle$, $|+\rangle$, $|-\rangle$, $|i\rangle$ y $|-i\rangle$. Se observa que todos coinciden con los ejes cartesianos de la esfera. Como ya se ha dicho, cualquier vector normalizado representado en la Esfera de Bloch es considerado un estado cuántico, y estos seis son recurrentemente mencionados en la literatura.

$$|i\rangle = \frac{1}{\sqrt{2}} |0\rangle + i \cdot \frac{1}{\sqrt{2}} |1\rangle \quad (2.13)$$

$$|-i\rangle = \frac{1}{\sqrt{2}} |0\rangle + (-i) \cdot \frac{1}{\sqrt{2}} |1\rangle \quad (2.14)$$

2.1.7. Sistemas de Múltiples Qubits

Se puede generalizar la base computacional de un sistema de n qubits como $\{|0\rangle, |1\rangle\}^{\otimes n}$ [3]. Suponiendo un sistema de dos qubits, la base computacional es $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$. En la Ecuación (2.15) se define a un sistema de dos qubits como una superposición de sus estados base.

$$|\psi\rangle = \alpha_{00} |00\rangle + \alpha_{01} |01\rangle + \alpha_{10} |10\rangle + \alpha_{11} |11\rangle \quad (2.15)$$

Al igual que con sistemas de un qubit, $|\psi\rangle$ debe respetar la propiedad de normalización, es decir, $\sum_{x \in \{0,1\}^n} |\alpha_x|^2 = 1$. Las mediciones en sistemas de múltiples qubits pueden realizarse sobre todo el sistema o sobre un subconjunto de sus qubits. Si se mide un subconjunto de qubits, el estado posterior a la medición debe ser renormalizado. Siguiendo el ejemplo del sistema de dos qubits, si únicamente medimos con respecto a la base computacional al qubit más a la izquierda, la probabilidad de que colapse a $|0\rangle$ es de $|\alpha_{00}|^2 + |\alpha_{01}|^2$. El estado posterior a la medición resultará en lo mostrado en la Ecuación (2.16).

$$|\psi'\rangle = \frac{\alpha_{00}|00\rangle + \alpha_{01}|01\rangle}{\sqrt{|\alpha_{00}|^2 + |\alpha_{01}|^2}} \quad (2.16)$$

Para sistemas de múltiples qubits, se definen las **compuertas controladas**. Las mismas funcionan como una extensión de las compuertas de un qubit. De esta manera, para una compuerta controlada genérica, existen n qubits control y m qubits objetivo, donde los qubits objetivo se verán afectados por la compuerta en función del estado de los qubits control. El ejemplo típico es la compuerta *control-X* (también llamada *CX* o *CNOT*). Si el qubit control está en estado $|0\rangle$, el qubit objetivo no sufrirá cambio alguno. En cambio, si el qubit control está en estado $|1\rangle$, el qubit objetivo se verá afectado por una compuerta *X*. De esta forma, dado un estado cuántico de dos qubits, la aplicación de *CX* para las diferentes combinaciones de valores se vería de la siguiente forma: $|00\rangle \mapsto |00\rangle$; $|01\rangle \mapsto |01\rangle$; $|10\rangle \mapsto |11\rangle$; $|11\rangle \mapsto |10\rangle$.

Para el desarrollo de esta tesis, es de interés presentar la compuerta $RZZ(\theta)$, que será utilizada junto a las compuertas de un qubit $RX(\theta)$ y $RZ(\theta)$ para implementar el Algoritmo Cuántico de Optimización Aproximada. Dados un qubit control y uno objetivo, aplicar $RZZ(\theta)$ es equivalente a las siguientes operaciones:

1. Aplicar *CX* entre los qubits control y objetivo.
2. Aplicar $RZ(\theta)$ sobre el qubit objetivo.
3. Finalizar repitiendo *CX* los qubits entre control y objetivo.

2.1.8. Circuitos Cuánticos

Un circuito cuántico es un conjunto de n líneas de tiempo o cables, que representan el ciclo de vida de n qubits a medida que son afectados por compuertas [3]. Son leídos de izquierda a derecha y cada una de las líneas representa a un qubit en particular. Por convención, se asume que el estado inicial de los qubits es el $|0\rangle$, a menos que el circuito indique lo contrario. También se asumirá que los cables se enumeran de arriba hacia abajo comenzando por el índice 0.

Como se presentó en la Figura 2.2, cada compuerta cuenta con un diseño que las diferencia del resto. Por otro lado, en la Figura 2.4, se observa un ejemplo de un circuito con algunas compuertas simples y de doble-control. Matemáticamente, su estado cuántico evoluciona a través del circuito como lo indica la Ecuación (2.17), donde cada compuerta se nota como C_i tal que C indica el nombre de la compuerta e i indica sobre qué qubit o qubits se está aplicando.

$$\begin{aligned}
|\phi\rangle &= |000\rangle \\
H_0 H_1 H_2 |\phi\rangle &= \frac{1}{\sqrt{8}}(|000\rangle + |001\rangle + |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle) \\
CX_{0,1} H_{0,1,2} |\phi\rangle &= \frac{1}{\sqrt{8}}(|000\rangle + |011\rangle + |010\rangle + |001\rangle + |100\rangle + |111\rangle + |110\rangle + |101\rangle) \\
RZ(\pi)_2 CX_{0,1} H_{0,1,2} |\phi\rangle &= \frac{1}{\sqrt{8}}(|000\rangle + |011\rangle + |010\rangle + |001\rangle - |100\rangle - |111\rangle - |110\rangle - |101\rangle) \\
H_2 RZ(\pi)_2 CX_{0,1} H_{0,1,2} |\phi\rangle &= \frac{1}{\sqrt{4}}(-|100\rangle - |111\rangle - |110\rangle - |101\rangle) \\
H_2 RZ(\pi)_2 CX_{0,1} H_{0,1,2} |\phi\rangle &= -\frac{1}{\sqrt{4}}(|100\rangle + |111\rangle + |110\rangle + |101\rangle) \\
H_2 RZ(\pi)_2 CX_{0,1} H_{0,1,2} |\phi\rangle &= \frac{1}{\sqrt{4}}(|100\rangle + |111\rangle + |110\rangle + |101\rangle)
\end{aligned} \quad (2.17)$$

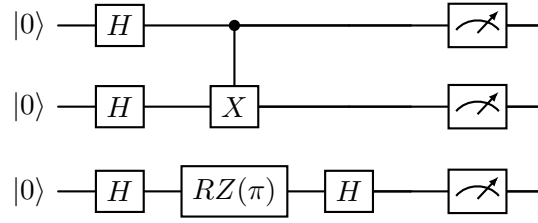


Figura 2.4: Circuito cuántico genérico en el que se observan algunas compuertas simples y de doble-qubit.

Inicialmente, el estado cuántico $|\phi\rangle$ comienza en $|000\rangle$. Tras aplicar la compuerta H sobre los 3 qubits, el estado evoluciona una superposición uniforme de todos los posibles estados base. Luego, se aplica la compuerta de doble-qubit CX , donde el qubit 0 es el control y el qubit 1 el objetivo. Esto genera cambios en algunos de los estados base de la superposición, dependiendo del valor del qubit control. Los estados base afectados por la compuerta CX son: $|001\rangle \mapsto |011\rangle$, $|011\rangle \mapsto |001\rangle$, $|101\rangle \mapsto |111\rangle$, $|111\rangle \mapsto |101\rangle$. Si se conmutan los términos correctamente, se puede observar que la aplicación de la compuerta CX no generó ningún cambio en $|\phi\rangle$. En tercer lugar, se aplica $RZ(\pi)$ en el qubit 2, es decir, una rotación de π a lo largo del eje Z . Esta rotación es equivalente a aplicar directamente la compuerta Z . Se observa que hubo un cambio en la fase local del qubit 2, causando que los estados base $|100\rangle$, $|101\rangle$, $|110\rangle$ y $|111\rangle$ ahora estén multiplicados por el factor -1 . Tras aplicar la siguiente y última compuerta H en el qubit 2, se observa que ahora existe una superposición uniforme únicamente entre los estados base $|100\rangle$, $|101\rangle$, $|110\rangle$ y $|111\rangle$. En este caso, se ejemplifica la fase global como un factor -1 que multiplica a los cuatro estados base del estado cuántico. Éste puede trasladarse como factor común multiplicador de la fracción $\frac{1}{\sqrt{4}}$ y, finalmente, eliminarlo dada la equivalencia presentada en la Sección 2.1.6.

2.1.9. El Operador Hamiltoniano

En el contexto de la computación cuántica aplicada a la optimización combinatoria, es necesario introducir el concepto del operador Hamiltoniano (o simplemente Hamiltoniano, como se encuentra habitualmente en la literatura). Para esto, se introduce el segundo postulado de la mecánica cuántica [3].

Postulado 2. *La evolución de un sistema cuántico **cerrado** es descrita por una **transformación unitaria**. Es decir, el estado $|\psi\rangle$ del sistema en el tiempo t_1 se relaciona al estado $|\psi'\rangle$ del mismo sistema en el tiempo t_2 a través de un operador unitario U , el cual solo depende de los tiempos t_1 y t_2 . Esto se representa como la igualdad definida en la Ecuación (2.18).*

$$|\psi'\rangle = U |\psi\rangle \quad (2.18)$$

El segundo postulado describe cómo dos estados en dos puntos temporales diferentes dentro de un mismo sistema cerrado están relacionados entre sí. Este postulado puede refinarse para describir la evolución de un sistema cuántico durante un tiempo continuo. En esta revisión, se se presenta el Hamiltoniano como un elemento fundamental.

Postulado 2'. *La evolución a través del tiempo t del estado de un sistema cuántico cerrado es descrita por la Ecuación de Schrödinger presentada en la Ecuación (2.19).*

$$i\hbar \frac{d|\psi\rangle}{dt} = H |\psi\rangle \quad (2.19)$$

Dicho de otra forma, la derivada respecto del tiempo t del estado de un sistema cuántico cerrado $|\psi\rangle$ (es decir, la rapidez y dirección en la que cambia $|\psi\rangle$ en un tiempo t) se obtiene a

partir del producto matriz-vector entre el Hamiltoniano H y $|\psi\rangle$. Es importante señalar que el Hamiltoniano introducido en el postulado utiliza la notación H , la cual coincide con la empleada para denotar la compuerta de Hadamard. Sin embargo, ambos conceptos son distintos y no deben confundirse.

En el postulado, $\hbar$ es una constante física conocida como la Constante de Planck. Su valor debe ser determinado experimentalmente y no tiene mayor importancia para este análisis. En la práctica, $\hbar$ suele ser absorbido por H tal que $H' = \frac{H}{\hbar}$, lo que simplifica la notación (se asumirá que el Hamiltoniano con $\hbar$ absorbido mantiene la notación original de H). Luego, se define al operador H como el Hamiltoniano del sistema cuántico a describir. Este puede depender, o no, del tiempo, lo que indica si H varía con el tiempo o si se mantiene constante.

La importancia del Hamiltoniano radica en que permite describir por completo las dinámicas a través del tiempo de un sistema cuántico cerrado, es decir, la dirección y rapidez de cambio del sistema, aunque determinar H es un problema difícil de resolver. A partir de lo anterior, la Ecuación (2.20) muestra cómo obtener el estado de un sistema cuántico tras haber evolucionado durante un tiempo t con un Hamiltoniano independiente del tiempo. De esta forma, si H describe como evoluciona $|\psi\rangle$ en el tiempo, $e^{-itH}|\psi(0)\rangle$ representa el estado resultante $|\psi(t)\rangle$ tras una evolución durante el tiempo t .

$$|\psi(t)\rangle = e^{-itH}|\psi(0)\rangle \quad (2.20)$$

Un Hamiltoniano H es un operador hermítico (*hermitian*). Es decir, que se cumple que $H = H^\dagger$. Por esto mismo, puede ser descompuesto en la sumatoria de la Ecuación (2.21), donde cada autovalor E corresponde a su respectivo autovector $|E\rangle$. Los estados $|E\rangle$ son usualmente llamados como los **autoestados de energía**, mientras que cada E es la energía correspondiente de un autoestado $|E\rangle$.

$$H = \sum_E E |E\rangle \langle E| \quad (2.21)$$

De esta forma, la menor energía posible de un sistema cuántico definido por H corresponde a un autoestado $|E\rangle$ conocido como el **ground state**, y dicha energía se denomina **energía del ground state**.

Incorporar el concepto de Hamiltoniano en la computación cuántica permite describir como un conjunto de compuertas operan sobre un circuito. De esta forma, toda una colección de compuertas puede ser agrupada en un Hamiltoniano definido. Por otro lado, su utilidad en la optimización combinatoria viene de la mano de la existencia de la energía del **ground state**. Un problema de optimización puede ser codificado en un circuito cuántico a través de un Hamiltoniano H , de forma que encontrar el **ground state** del sistema implique también obtener la solución óptima del problema [29]. En la Sección 2.2.2 se explicará con mayor detalle como se utilizarán Hamiltonianos para codificar una instancia del problema JRP y ejecutarlo en QAOA.

2.1.10. El Valor Esperado

En la Sección 2.1.5, se introduce como obtener información de un sistema cuántico a partir de una medición. En la misma se menciona que un sistema cuántico tiene asociada una probabilidad de medición para cada posible resultado del mismo. Por otra parte, en la Sección 2.1.9 se explica que un sistema cuántico $|\psi\rangle$ evoluciona a partir de un determinado Hamiltoniano H , implementado como un conjunto de compuertas en una computadora cuántica. En esta sección, se profundizará en cómo obtener el valor esperado de un estado cuántico a partir de un muestreo de mediciones.

En estadística, el concepto de valor esperado representa un promedio ponderado por la probabilidad de ocurrencia de los posibles valores x de una variable aleatoria X [30]. Cuando X es una variable discreta, el valor esperado se define como lo indica la Ecuación (2.22), donde cada

posible valor discreto x es ponderado por la probabilidad $f(x)$.

$$\mathbb{E}(X) = \sum_x x \cdot f(x) \quad (2.22)$$

Calcular el valor esperado de un estado cuántico implica realizar un muestreo de N mediciones para generar una distribución de probabilidad de los resultados obtenidos. Su utilidad radica en la capacidad de resumir toda una distribución de probabilidad en un único parámetro. En la Sección 2.3.2 se hará uso del valor esperado de un sistema cuántico como criterio de optimización para el algoritmo QAOA.

2.1.11. Estado Actual del Hardware Cuántico

Actualmente no se ha consolidado una arquitectura de hardware cuántico como solución predominante. En consecuencia, la investigación se orienta hacia diversas plataformas físicas, cada una con ventajas técnicas específicas y limitaciones inherentes. Entre las principales líneas de desarrollo se destacan los dispositivos basados en superconductores [31], los iones atrapados [32], los átomos neutros [33] y los sistemas de Templado Cuántico (*Quantum Annealing*) [34].

En el ámbito de los superconductores, IBM presentó en 2023 el procesador Condor, que integra 1.121 qubits, constituyéndose como su chip más grande hasta la fecha [35]. De manera complementaria, Google introdujo Willow, un procesador de 105 qubits también fundamentado en esta tecnología [36]. En el caso de los iones atrapados, Quantinuum dio a conocer en 2024 su sistema H2-1, conformado por 56 qubits con conectividad total [37], mientras que IonQ ofrece el procesador Forte, con 36 qubits y acceso disponible mediante servicios en la nube [38]. Por otra parte, QuEra ha desarrollado la plataforma Aquila, un procesador analógico reconfigurable de 256 qubits basado en átomos neutros y accesible a través de AWS Braket [39]. Finalmente, en el enfoque de Templado Cuántico, la empresa D-Wave comercializa la plataforma Advantage, que dispone de aproximadamente 5.000 qubits [40], orientada principalmente a problemas de optimización combinatoria.

Si bien los dispositivos cuánticos de última generación reportan recuentos físicos que alcanzan varios cientos o incluso miles de qubits, la cantidad de qubits efectivamente utilizables para la ejecución de algoritmos prácticos es considerablemente menor. La razón principal radica en la presencia de ruido cuántico, entendido como la acumulación de errores proveniente de las operaciones realizadas y la decoherencia a través del tiempo de los estados cuánticos preparados [41], de modo que sólo una fracción de los qubits puede emplearse de forma confiable en tareas computacionales.

En la práctica, la mayoría de las plataformas actuales permiten trabajar con un número relativamente reducido de qubits lógicos efectivos, con aproximadamente 15 qubits [42]. Esta limitación refleja la capacidad real de los dispositivos para ejecutar algoritmos no triviales sin que los errores acumulados invaliden el resultado. En consecuencia, aunque existen procesadores con recuentos nominales que superan ampliamente estas cifras, el umbral de utilidad sigue estando restringido a decenas de qubits. Este desajuste entre qubits físicos y qubits lógicos útiles es un desafío central para la computación cuántica contemporánea.

Ante estas limitaciones, los simuladores de cómputo cuántico se han consolidado como una herramienta fundamental para la investigación y el desarrollo [7]. Bibliotecas como Qiskit Aer, Cirq o QuTiP permiten emular el comportamiento de circuitos cuánticos sobre hardware clásico, alcanzando decenas de qubits sin verse afectados por el ruido inherente de los dispositivos físicos. Estas simulaciones pueden realizarse simulando el ruido propio del hardware cuántico real, o bien ejecutando las operaciones de forma ideal y sin errores de cómputo. Aún así, la finalidad de los simuladores es principalmente hacer accesible la investigación de algoritmos cuánticos, ya que se pierde toda ventaja computacional que existiría en un verdadero dispositivo cuántico.

2.2. Modelo QUBO

En el área de los problemas de optimización combinatoria, se vuelve fundamental la decisión sobre qué tipo de representación se usará para los mismos. Por ejemplo, el Problema de la Mochila [43], el Problema del Camino más Corto [44] y el Problema del Corte Máximo [45] pueden verse beneficiados en su resolución de acuerdo a como se los represente formalmente.

El modelo de **Optimización Cuadrático Binario sin Restricciones** (*Quadratic Unconstrained Binary Optimisation*) (QUBO) se adapta a una gran cantidad de problemas de optimización encontrados en la industria y la ciencia [46, 47]. Se lo define formalmente en la Ecuación (2.23), donde $x \in \{0, 1\}^n$ es un vector de n variables de decisión binarias y Q es una matriz n -cuadrada simétrica de constantes reales. El vector x define la estructura de las soluciones candidato. Los valores de Q definen los pesos asociados a cada variable de decisión de x , es decir, el impacto que tiene cada variable de decisión en el costo del modelo.

$$\text{QUBO :maximizar/minimizar } y = x^t Q x \quad (2.23)$$

Para comprender la intuición del formalismo, se puede operar el producto matriz-vector Qx en Ecuación (2.24) para obtener un resultado que será notado como x^Q , donde cada variable de decisión x_i se encuentra ponderada por las constantes de Q . Las variables de decisión posicionadas en el índice i en los vectores x^t y x^Q son las mismas, y únicamente difieren en el factor que las pondera.

$$\text{QUBO :maximizar/minimizar } y = x^t Q x = x^t x^Q \quad (2.24)$$

Al operar sobre el producto vectorial desarrollado, el modelo puede ser expresado (de manera más intuitiva) como una función de costo en relación a un grafo $G = (V, E)$ [48], indicada en la Ecuación (2.25). En el grafo G , los nodos son nombrados como las variables de decisión de x y etiquetados como sus respectivos factores a_i . El arco entre un nodo x_i y x_j es etiquetado con el factor b_{ij} .

$$\text{maximizar/minimizar QUBO}(x) = \sum_{i \in V} a_i x_i + \sum_{(ij) \in E} b_{ij} x_i x_j \quad x_i \in \{0, 1\}, a_i, b_{ij} \in \mathbb{R} \quad (2.25)$$

Consecuentemente, el modelo QUBO tiene un conjunto de características que lo distinguen de otras formulaciones. Su función de costo está formada por variables únicamente binarias y términos a lo sumo cuadráticos. Es importante notar que lo anterior se cumple ya que, al operar en la Ecuación (2.23), el producto vectorial $x^t x^Q$ genera términos cuadráticos ($x_i^t x_j^Q$) o términos lineales $x_i^t x_i^Q = x_i^t = x_i^Q$, donde i, j son los índices de las variables en los vectores. El modelo QUBO tampoco permite hacer uso de restricciones en forma de funciones auxiliares a la función de costo. Existen diversos procedimientos para reformular un problema de optimización con restricciones en su versión del modelo QUBO. En esta tesis, se hará uso de las penalizaciones cuadráticas.

2.2.1. Penalizaciones Cuadráticas

La generación de penalizaciones cuadráticas, un procedimiento similar al método de Coeficientes de Lagrange [46], busca penalizar los estados del espacio de búsqueda que no cumplen con un conjunto de restricciones. Éstas no son funciones auxiliares a la función objetivo del problema, sino que son términos embebidos dentro de la misma. De esta manera, las penalizaciones cuadráticas no restringen el espacio de búsqueda como lo hace una restricción clásica.

En la Tabla 2.2, se observan algunas de las equivalencias típicas desde una restricción clásica hacia una penalización cuadrática, donde x, y, x_1, x_2, x_3 son variables binarias, $P \in \mathbb{R}^+$ para problemas de minimización y $P \in \mathbb{R}^-$ para problemas de maximización. Notar que el término de penalización resulta en el valor 0 únicamente cuando la restricción clásica no es infringida. En casos contrarios, el término de penalización toma un valor real diferente a 0, lo que permite penalizar el costo de la función objetivo.

Restricción clásica	Penalización equivalente
$x + y \leq 1$	$P(xy)$
$x + y \geq 1$	$P(1 - x - y + xy)$
$x + y = 1$	$P(1 - x - y + 2xy)$
$x \leq y$	$P(x - xy)$
$x_1 + x_2 + x_3 \leq 1$	$P(x_1x_2 + x_1x_3 + x_2x_3)$

Tabla 2.2: Tabla de penalizaciones equivalentes a una restricción clásica en un problema de minimización.

2.2.2. Equivalencia al Modelo Ising

El modelo Ising es un modelo matemático del ferromagnetismo en la mecánica estadística. Fue presentado por el físico Wilhelm Lenz y profundizado por el físico Ernst Ising durante su tesis doctoral [49]. Además de su uso en la mecánica estadística, el modelo Ising ha sido probado de gran utilidad para formular problemas de optimización combinatoria [50], codificando un problema de optimización como un Hamiltoniano H . De esta forma, dado H descompuesto según la Ecuación (2.21), se codifica cada solución candidato del problema y su respectivo costo como un autoestado $|E\rangle$ y energía E , donde el *ground state* corresponde a la solución óptima.

En esta sección, se desarrollará el proceso de codificación de un problema en el modelo QUBO hacia su Hamiltoniano equivalente en el modelo Ising. Éste consta de dos mapeos notados informalmente como lo indica la Ecuación (2.26), donde f_{Ising} es una función de costo y H_C un Hamiltoniano de costo, ambos pertenecientes al modelo Ising.

$$\text{QUBO} \mapsto f_{\text{Ising}} \mapsto H_C \quad (2.26)$$

Para desarrollar el primer mapeo, se define f_{Ising} en las Ecuaciones (2.27) y (2.28), dependiendo de si se debe maximizar o minimizar la función. De estas definiciones, se destaca su parecido estructural al modelo QUBO, con la diferencia de que el dominio de las variables binarias es $\{-1, 1\}$. De esta forma, dado un QUBO con sus respectivas variables binarias $\{x_i\}_{i=0}^n$, se puede obtener su f_{Ising} equivalente realizando el mapeo $x_i \equiv \frac{s_i+1}{2} \quad \forall x_i$, donde $s_i \in \{-1, 1\}$ [50].

$$\text{maximizar } f_{\text{Ising}}(s) = - \sum_{i \in V} h_i s_i - \sum_{(ij) \in E} J_{ij} s_i s_j \quad s_i \in \{-1, 1\}, J_{ij}, h_i \in \mathbb{R} \quad (2.27)$$

$$\text{minimizar } f_{\text{Ising}}(s) = \sum_{i \in V} h_i s_i + \sum_{(ij) \in E} J_{ij} s_i s_j \quad s_i \in \{-1, 1\}, J_{ij}, h_i \in \mathbb{R} \quad (2.28)$$

El segundo mapeo hacia H_C se logra reemplazando cada variable s_i de f_{Ising} por σ_i^z , obteniendo las expresiones de las Ecuaciones (2.29) y (2.30). El operador σ_i^z es una compuerta cuántica Z aplicada sobre el qubit i , pero se transforman en $RZ(2h_i)$ y $RZZ(2J_{ij})$ al aplicar H^C dentro de un exponente de evolución, como se indicó previamente la Ecuación (2.20).

$$\text{maximizar } H_C = - \sum_{i \in V} h_i \sigma_i^z - \sum_{(ij) \in E} J_{ij} \sigma_i^z \sigma_j^z \quad (2.29)$$

$$\text{minimizar } H_C = \sum_{i \in V} h_i \sigma_i^z + \sum_{(ij) \in E} J_{ij} \sigma_i^z \sigma_j^z \quad (2.30)$$

Nuevamente, la importancia de modelar un problema de optimización en formulación Ising radica en su capacidad para codificar las soluciones candidato del problema y sus energías dentro de un Hamiltoniano, el cual puede ser implementado en un circuito cuántico a partir de compuertas.

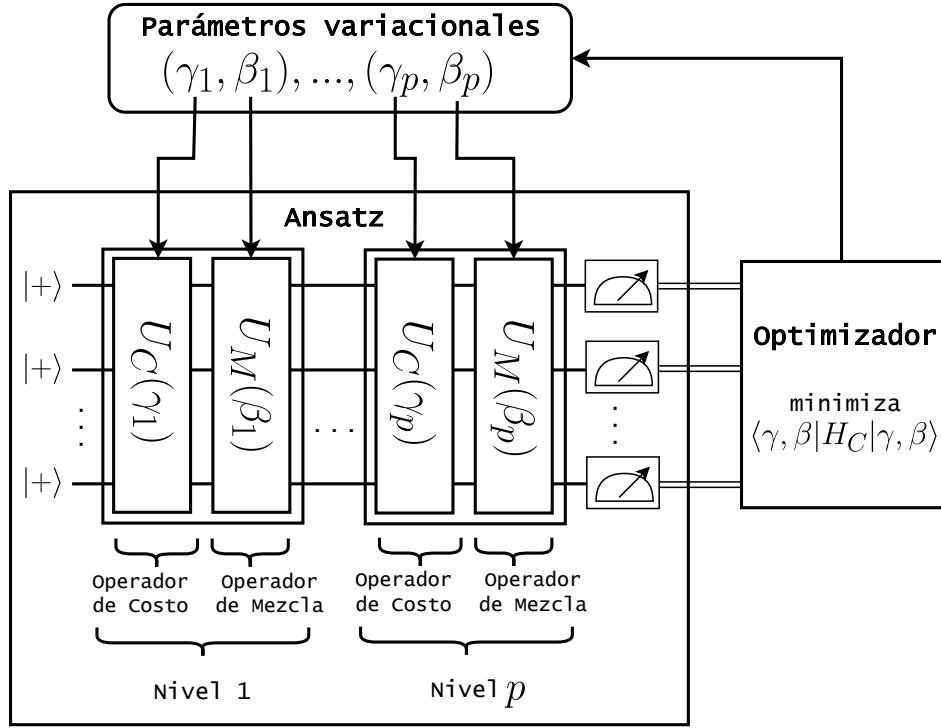


Figura 2.5: Diseño de la arquitectura algorítmica de QAOA.

2.3. Algoritmo de Optimización Aproximada Cuántica

El **Algoritmo de Optimización Aproximada Cuántica** (*Quantum Approximate Optimisation Algorithm*) (QAOA) [10, 51, 52] busca obtener soluciones aproximadas a problemas de optimización combinatoria. Gracias a su capacidad de absorber el ruido cuántico, se cree que puede ser de gran utilidad en computadoras NISQ.

En la Figura 2.5, se observa un diseño simplificado de la arquitectura algorítmica de QAOA, indicando como se interrelacionan sus componentes principales. El primer componente es un circuito cuántico denominado **ansatz**, que se caracteriza por sus $p \in \mathbb{N}$ niveles. Cada nivel del ansatz se compone de dos operadores. Dado un nivel $1 \leq j \leq p$, el primero de ellos es el **Operador de Costo** $U_C(\gamma_j)$, que utiliza el procedimiento presentado en la Sección 2.2.2 para codificar el Hamiltoniano del problema de optimización combinatoria. El segundo es el **Operador de Mezcla** $U_M(\beta_j)$, que tiene un rol fundamental al momento de aumentar la probabilidad de medición de soluciones aproximadamente óptimas. El siguiente componente, mostrado en la zona superior de la figura, es un conjunto de $2 \cdot p$ **parámetros variacionales** $(\vec{\gamma}, \vec{\beta}) = (\gamma_1, \dots, \gamma_p, \beta_1, \dots, \beta_p)$ que sirven de entrada para el ansatz. Éstos definen los ángulos de rotación de las compuertas que implementan los operadores. Como se introdujo en las Secciones 2.1.4 y 2.1.7, se utilizan las compuertas RX , RZ y RZZ como parte de la implementación de QAOA. Particularmente, RZ y RZZ se utilizan para $U_C(\gamma_j)$, mientras que RX se utiliza para $U_M(\beta_j)$. Finalmente, QAOA incorpora un **algoritmo de optimización clásico** que busca la configuración de parámetros variacionales que optimice el funcionamiento del ansatz.

2.3.1. El Ansatz Cuántico

El primer paso para construir el ansatz consiste en preparar el estado cuántico $|s\rangle$ como una superposición igualitaria de estados base en la base computacional [10, 51, 52]. El estado $|s\rangle$ se

define en la Ecuación (2.31).

$$|s\rangle = |+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} |z\rangle \quad (2.31)$$

Luego, se aplicarán los operadores $U_C(\gamma_j)$ y $U_M(\beta_j)$ en cada uno de los p niveles del ansatz. Éstos se definen en las Ecuaciones (2.32) y (2.33) respectivamente y representan la evolución del estado cuántico a través del tiempo haciendo uso de la Ecuación (2.20) [10, 51, 52]. H_C y H_M representan los hamiltonianos de la evolución del estado, mientras que los parámetros γ_j y β_j son un tiempo particular j de la evolución. Por un lado, el Hamiltoniano H_C se representa a partir de la formulación Ising presentada en la Ecuación (2.30). Como se menciona en la sección, se utilizan las compuertas $RZ(2h_i)$ y $RZZ(2J_{ij})$, pero al incorporar el parámetro variacional γ_j , las compuertas cambian a $RZ(2\gamma_j h_i)$ y $RZZ(2\gamma_j J_{ij})$. Por otro lado, el Hamiltoniano H_M se representa a partir de la Ecuación (2.34), donde el operador σ_i^x es una compuerta cuántica X aplicada sobre el qubit i que al hacer uso de la Ecuación (2.20), se transforma en una compuerta $RX(2\beta_j)$.

$$U_C(\gamma_j) = e^{-i\gamma_j H_C}, \quad (2.32)$$

$$U_M(\beta_j) = e^{-i\beta_j H_M}, \quad (2.33)$$

$$H_M = \sum_{j=1}^n \sigma_j^x \quad (2.34)$$

El estado resultante del ansatz tras inicializarlo como $|s\rangle$ y aplicar los p niveles de operadores se observa en la Ecuación (2.35).

$$|\gamma, \beta\rangle = U_M(\beta_p)U_C(\gamma_p) \dots U_M(\beta_1)U_C(\gamma_1) |s\rangle \quad (2.35)$$

2.3.2. Optimización de los Parámetros Variacionales

Para optimizar el funcionamiento del ansatz respecto a los parámetros variacionales configurados es necesario realizar una primera inicialización de los parámetros, seleccionar un algoritmo de optimización clásica y una función sobre la cual optimizar.

Existen múltiples protocolos de inicialización de parámetros variacionales. El más sencillo de éstos involucra una inicialización aleatoria de los mismos, aunque existen otros protocolos que buscan reducir el tiempo total de optimización. En la Sección 2.3.3, se introduce el protocolo de inicialización TQA, el cual será utilizado en este trabajo de tesis.

También existe una gran cantidad de métodos de optimización clásica, cada uno con diferentes características y ventajas según el problema a resolver. Entre algunos de los más conocidos se encuentran COBYLA [53], Nelder-Mead [54], ADAM [55] y BFGS [56]. En la presente tesis se utilizará el método Powell [57], cuya principal ventaja radica en poder optimizar funciones multi-variables sin realizar el cálculo de derivadas. Ésto lo vuelve un algoritmo que brinda buenos resultados a la hora de combinarlo con QAOA [58].

Sea cual sea el algoritmo de optimización seleccionado, la función a optimizar por el mismo será el valor esperado del sistema $H_C |\gamma, \beta\rangle$, notado como $\langle H_C \rangle$ y presentado en la Ecuación (2.36).

$$\langle H_C \rangle := \langle \gamma, \beta | H_C | \gamma, \beta \rangle \quad (2.36)$$

El valor $\langle H_C \rangle$ es utilizado como criterio de optimización de los parámetros variacionales. Tras realizar múltiples iteraciones de este proceso, se encuentran los parámetros $(\vec{\gamma}, \vec{\beta})$ que aproximadamente minimizan $\langle H_C \rangle$.

Para determinar el correcto funcionamiento del algoritmo, se debe ejecutar un último muestro de mediciones del ansatz con los parámetros variacionales optimizados. Asimismo, de dicha ejecución se obtienen un conjunto de soluciones candidato al problema de optimización combinatoria codificado.

2.3.3. Inicialización TQA

La inicialización de *Trotterised Quantum Annealing* (TQA) [59] es un protocolo de inicialización de parámetros variacionales para QAOA que permite al optimizador encontrar soluciones más cercanas al óptimo global del problema. Su funcionamiento se basa en discretizar la evolución temporal continua del Templado Cuántico en un conjunto de compuertas que aproximen de forma discreta tal evolución.

Su origen está relacionado al Templado Cuántico (*Quantum Annealing*) [11], uno de los primeros algoritmos de computación cuántica propuestos para problemas de optimización. En términos generales, el Templado Cuántico busca preparar el *ground state* de un Hamiltoniano objetivo H_C haciendo evolucionar durante un tiempo continuo T a un Hamiltoniano inicial H_B . De esta manera, se obtiene la solución óptima del problema de optimización a resolver, la cual está codificada en el *ground state* de H_C . Este algoritmo ha sido demostrado como universal, es decir, que si se lo ejecuta durante un tiempo lo suficientemente largo puede resolver cualquier problema de optimización para un tiempo continuo $T \rightarrow \infty$.

El Templado Cuántico no pertenece al paradigma de la computación cuántica basada en compuertas, por lo que para implementarlo, es necesario aplicar el algoritmo TQA, que discretiza la evolución de H_B en un tiempo continuo T a través de una secuencia finita de compuertas. Al aplicar TQA en un Templado Cuántico sobre tiempos de evolución discretos $t_i = i\Delta t$ tal que $i = 1, \dots, p$ y un intervalo temporal $\Delta_i = T/p$, se obtiene el circuito cuántico equivalente a un ansatz QAOA de profundidad p , cuyos parámetros variacionales son los presentados en la Ecuación (2.37). Estos parámetros resultantes son los que, finalmente, se utilizan en una inicialización TQA en base a un determinado intervalo temporal Δt e hiperparámetro p [59].

$$\gamma_i = \frac{i}{p}\Delta t, \quad \beta_i = (1 - \frac{i}{p})\Delta t \quad (2.37)$$

2.3.4. Aprendizaje por Transferencia

El proceso de optimización de parámetros variacionales puede resultar computacionalmente costoso en problemas de gran tamaño. La selección del optimizador, del protocolo de inicialización de parámetros y otro tipo de técnicas pueden cooperar en reducir el tiempo de optimización en QAOA. Entre estas técnicas se encuentra el **Aprendizaje por Transferencia**, que busca acelerar la optimización inicializando los parámetros variacionales con valores previamente optimizados en otras instancias de problemas. La reutilización de estos parámetros puede darse entre instancias de un mismo o de diferentes problemas. También puede utilizarse para extender hacia instancias con mayor cantidad de qubits. Incluso puede considerarse reutilizar los parámetros sin una segunda ronda de optimización. Esta técnica ha sido estudiada y experimentada por diferentes autores que han concluido en resultados optimistas respecto al Aprendizaje por Transferencia [60, 61, 62, 63, 64].

2.4. Problema de Reasignación de Puestos de Trabajo

Se plantea una organización con J trabajadores, cada uno con un trabajo asignado. Por circunstancias imprevistas, se instancian I trabajos nuevos de alta prioridad y que no tienen un trabajador asignado. En el Problema de Reasignación de Puestos de Trabajo (*Job Reassignment Problem*) (JRP), el objetivo es encontrar a los trabajadores que mejor se adapten a los I trabajos vacantes y reasignarlos. Para ésto, se tienen en cuenta $N = J \cdot I$ puntajes S_{ij} que indican qué tanto se adapta un trabajador i a un trabajo vacante j . La mejor configuración será aquella que maximiza la sumatoria de los puntajes S_{ij} [20].

2.4.1. Definición de los Puntajes S_{ij}

Cada S_{ij} de la función de costo se compone de la **Ganancia de Prioridad** Δ_{ij}^P y la **Ganancia de Afinidad** Δ_{ij}^A . La Ganancia de Prioridad se define como $\Delta_{ij}^P \equiv \mathcal{P}_i^V - \mathcal{P}_j^C$ teniendo la prioridad del trabajo vacante $\mathcal{P}_i^V \in (0, 1]$ y la prioridad del trabajo actualmente asignado al trabajador $\mathcal{P}_j^C \in (0, 1]$. Las prioridades del trabajo vacante y el asignado también pueden ser definidas como valores discretos, a saber: dadas D categorías de prioridad, se define $\mathcal{P}_i^V \in \{1, 2, \dots, D\}$ y $\mathcal{P}_j^C \in \{1, 2, \dots, D\}$ (en esta tesis, se optó por definir las prioridades como un valor categórico tal que $x \in \{1, 2, 3, 4, 5\}$). En el dominio del problema, el valor de prioridad de un trabajo arbitrario se define como la importancia que tiene el mismo para la productividad y eficiencia de la organización empresarial. De esta forma, un mismo trabajo puede tener diferentes valores de prioridad a lo largo del tiempo, dependiendo de la situación actual de la organización.

La Ganancia de Afinidad se define como $\Delta_{ij}^A \equiv \mathcal{A}_{ij}^V - \mathcal{A}_{jj}^C$, teniendo la afinidad personal del trabajador j con el trabajo vacante i como $\mathcal{A}_{ij}^V \in (0, 1]$, y la afinidad personal del trabajador con el trabajo que tiene asignado $\mathcal{A}_{jj}^C \in (0, 1]$. En el dominio del problema, el valor de afinidad de un trabajador con un trabajo caracteriza la motivación y satisfacción laboral que le genera al trabajador estar asignado en ese puesto. Diversos factores como la lejanía con su hogar, dificultad del trabajo y salario del mismo pueden afectar este valor. Finalmente, la Ecuación (2.38) define el puntaje total para un trabajador i y un trabajo vacante j . Los **Coefficientes de Ganancia** se representan como c^P y c^A y se definen como constantes positivas que cuantifican el peso relativo de cada uno de los términos.

$$S_{ij} = c^P \Delta_{ij}^P + c^A \Delta_{ij}^A \quad (2.38)$$

2.4.2. Formulación QUBO

La función QUBO del problema se define en dos partes: la función núcleo H^0 , que codifica la función objetivo del problema, y la función de restricción H^R [20]. Para facilitar la traducción de la formulación QUBO al modelo Ising que se usará en QAOA, se plantea JRP como un problema de minimización de costos en vez de maximización de puntaje. H^0 será negado, mientras que H^R penalizará aumentando el valor del costo.

En la Ecuación (2.39), se define la función núcleo. Se puede notar que para mayores ganancias de prioridad y afinidad, el valor de H^0 disminuye.

$$H^0 = - \sum_{ij} S_{ij} x_{ij} = -c^P \sum_{ij} (\Delta_{ij}^P x_{ij}) - c^A \sum_{ij} (\Delta_{ij}^A x_{ij}) \quad (2.39)$$

Por otro lado, la Ecuación (2.40) define la función de penalización total. Su primer término es presentado en la Ecuación (2.41). Este garantiza, para un $\lambda_1^R > 0$ lo suficientemente grande, que cada trabajo vacante i pueda ser tomado a lo sumo por un trabajador. Su segundo término es presentado en la Ecuación (2.42) y garantiza que, para un $\lambda_2^R > 0$ lo suficientemente grande, cada trabajador j sea reasignado a lo sumo a un trabajo vacante. Estas funciones de penalización hacen uso de la diferencia con la constante 0.5, permitiendo que la sumatorias de variables binarias de cada función deban resultar en 0 o 1 para minimizar la penalización recibida. Esto permite penalizar correctamente sin hacer uso de otras variables binarias auxiliares.

$$H^R = H_1^R + H_2^R \quad (2.40)$$

$$H_1^R = \lambda_1^R \sum_i \left(\sum_j x_{ij} - 0.5 \right)^2 \quad (2.41)$$

$$H_2^R = \lambda_2^R \sum_j \left(\sum_i x_{ij} - 0.5 \right)^2 \quad (2.42)$$

Las constantes λ_1^R y λ_2^R son los **coeficientes de penalización** e indican con qué grado se penalizará en caso de no cumplir con las restricciones equivalentes. Su valor es una heurística que debe ser configurado por el usuario, considerando que un valor pequeño puede no penalizar lo suficiente, y uno excesivamente grande puede sesgar la función objetivo.

Haciendo uso de las ecuaciones previamente mencionadas, se puede definir el función QUBO como lo indica la Ecuación (2.43).

$$\begin{aligned}
 H = & - \sum_{ij} [c^P (\mathcal{P}_i^V - \mathcal{P}_j^C) + c^A (\mathcal{A}_{ij}^V - \mathcal{A}_{jj}^C)] x_{ij} \\
 & + \lambda_1^R \sum_i \left(\sum_j x_{ij} - 0,5 \right)^2 + \lambda_2^R \sum_j \left(\sum_i x_{ij} - 0,5 \right)^2
 \end{aligned} \tag{2.43}$$

2.4.3. Comparación y Limitaciones de Enfoques de Resolución Clásica

El problema JRP es considerado como un caso particular del Problema de Asignación clásico [65], el cual ha sido objeto de un amplio estudio y para el que se han desarrollado solucionadores consolidados, tales como el método Húngaro o los enfoques basados en programación entera [66, 67]. Sin embargo, dichos solucionadores presentan limitaciones significativas al momento de incorporar *soft constraints* o restricciones blandas en las distintas variantes del Problema de Asignación, incluyendo el propio JRP. En estos casos, los solucionadores suelen incurrir en una sobrecarga computacional que reduce, e incluso puede llegar a eliminar, la eficiencia computacional del enfoque [23].

En este contexto, resulta fundamental establecer la distinción entre restricciones duras y blandas. Las restricciones duras corresponden a condiciones que toda solución candidato debe satisfacer estrictamente. Por el contrario, las restricciones blandas representan condiciones de carácter deseable, cuya satisfacción no es estrictamente obligatoria [21, 22]. La incorporación de este último tipo de restricciones permite brindar al modelo de una mayor flexibilidad, particularmente en escenarios en los que no resulta posible cumplir de manera simultánea con la totalidad de las restricciones duras del problema.

En este sentido, la relevancia del enfoque cuántico mediante QAOA radica en que su estructura variacional ofrece un marco para la incorporación simultánea y natural de restricciones duras y blandas [68]. Gracias a esta flexibilidad, QAOA no solo permite modelar con mayor fidelidad las condiciones inherentes a problemas como el JRP, sino que carece de las dificultades que enfrentan los solucionadores clásicos al manejar restricciones blandas dentro de problemas de optimización combinatoria.

Capítulo 3

Diseño de los Procedimientos

En el presente capítulo, se explicará el diseño y configuración de los procedimientos. Para comenzar, se define el flujo de trabajo principal encargado de realizar la muestra de ejecuciones de QAOA que implementan las instancias de JRP. Luego se definen tres procedimientos que postprocesan los resultados y brindan mayor información sobre la factibilidad de implementar JRP en QAOA.

3.1. Flujo de Trabajo Principal

El flujo de trabajo principal, que presenta un diseño conformado por cuatro capas de abstracción, se encarga de realizar la muestra de ejecuciones de QAOA para resolver instancias de JRP. Esta muestra es generada en función de un conjunto de configuraciones de entrada, donde cada individuo de la misma se corresponde a un conjunto de instancias de JRP implementadas en QAOA bajo una determinada configuración. En la Figura 3.1 se presenta el diseño propuesto, y cómo se conecta con el postprocesamiento y el análisis de resultados.

Cada capa introduce un nivel de abstracción adicional al flujo de trabajo. La capa de **muestreo de configuraciones** es responsable de generar los individuos de la muestra de salida, asegurándose de que cada uno de ellos corresponda a una única configuración de entrada. Cada individuo puede ser considerado una submuestra en sí misma, ya que se componen de la implementación en QAOA de cinco instancias diferentes de JRP. La generación de los individuos de estas submuestras es gestionada por la capa de **muestreo de instancias**, donde cada individuo corresponde a la resolución de una de las cinco instancias JRP. La estructura de estas capas se presenta en las Figuras 3.2 y 3.3 respectivamente.

La tercer capa, denominada **analizador de rendimiento algorítmico** y presentada en la Figura 3.4, es responsable de resolver una instancia JRP, procesar sus resultados y computar sus métricas de rendimiento algorítmico. Es decir, las métricas encargadas de cuantificar la calidad de las soluciones obtenidas. Las mismas son almacenadas, junto a otros datos propios de la implementación, en una de las submuestras. Finalmente, se introducen el **solucionador QAOA** y el **solucionador por fuerza bruta**. El primero de ellos implementa, configura y ejecuta el algoritmo QAOA, mientras que el segundo emplea una búsqueda por fuerza bruta para identificar la solución óptima del problema. El diseño del solucionador QAOA se presenta en la Figura 3.5.

Es importante destacar que el solucionador por fuerza bruta se utiliza como parte del cálculo de la razón de aproximación, una de las métricas de rendimiento algorítmico que será presentada en la Sección 3.1.2. Sin embargo, la ejecución eficiente de este solucionador es factible únicamente debido a que las instancias de problema consideradas en esta tesis son de tamaño reducido, lo que posibilita explorar exhaustivamente el espacio de soluciones en tiempos de cómputo razonables. Sin embargo, si se decidiera extender la evaluación a instancias de mayor tamaño, este enfoque se volvería impracticable debido al crecimiento exponencial del espacio de búsqueda, imposibilitando así el cálculo exacto de la razón de aproximación mediante fuerza bruta.

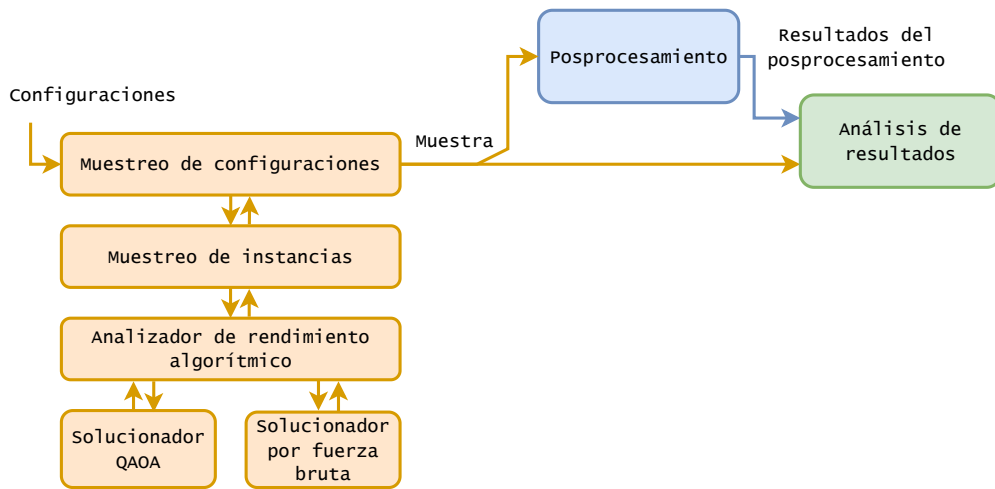


Figura 3.1: Diseño de cuatro capas del flujo de trabajo principal, conectado con el postprocesamiento y el análisis de resultados.

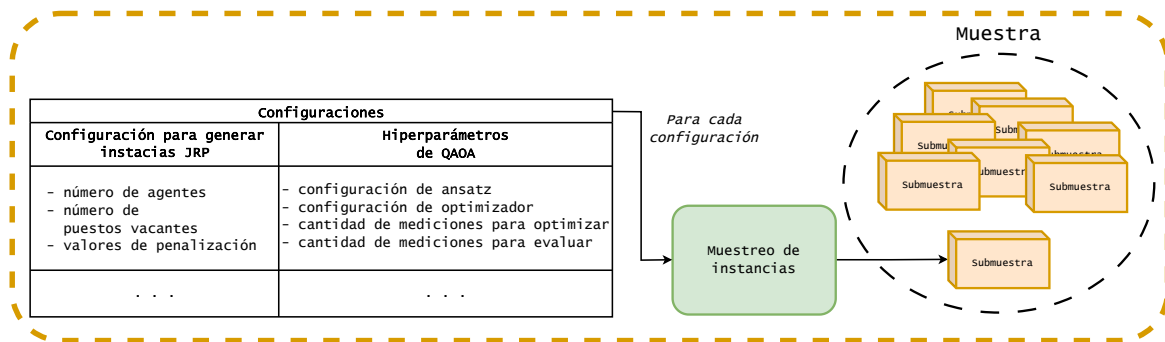


Figura 3.2: Diseño de la primera capa: Muestreo de configuraciones.

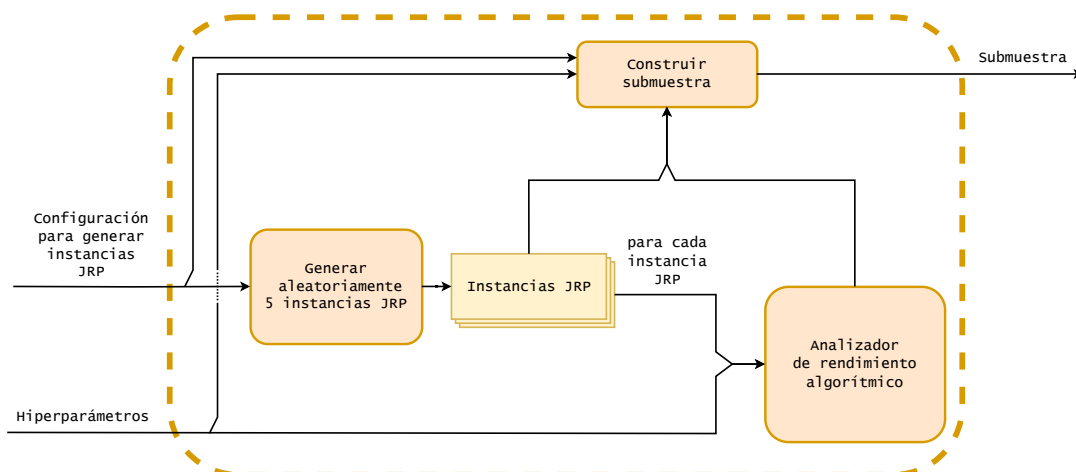


Figura 3.3: Diseño de la segunda capa: Muestreo de instancias.

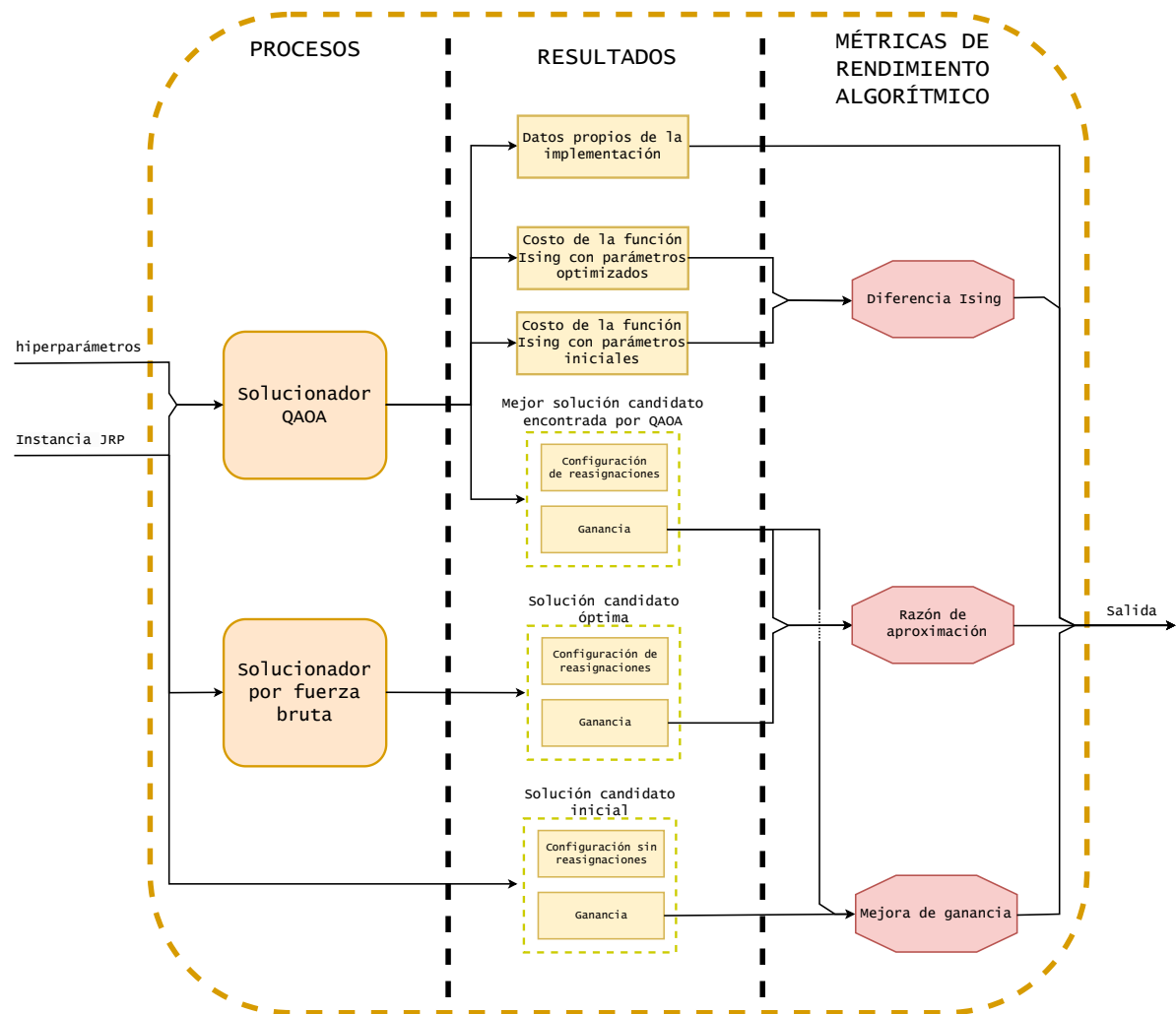


Figura 3.4: Diseño de la tercer capa: Analizador de rendimiento algorítmico.

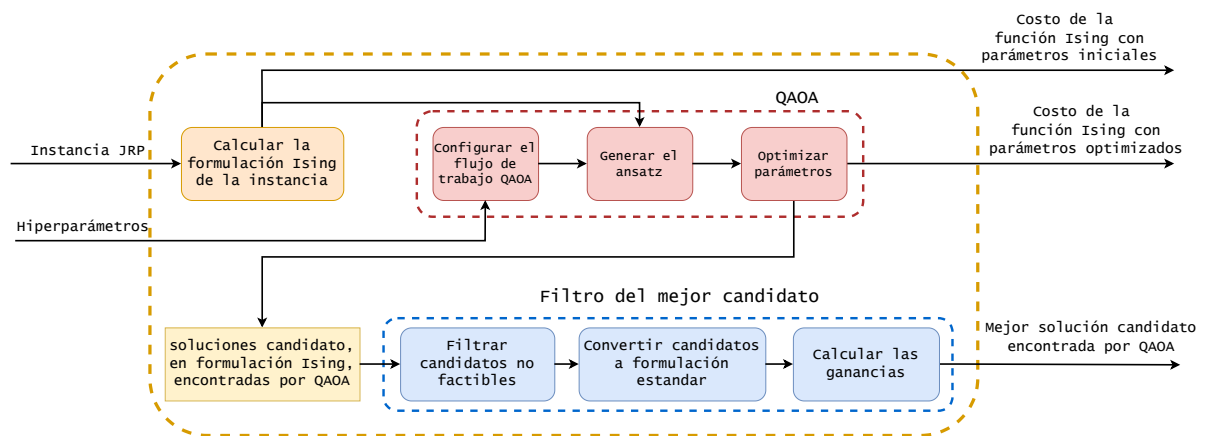


Figura 3.5: Diseño de uno de los componentes de la cuarta capa: Solucionador QAOA

ID	trabajadores	vacantes	p
(5,3,3)	5	3	3
(5,3,4)	5	3	4
(5,3,5)	5	3	5
(3,5,3)	3	5	3
(3,5,4)	3	5	4
(3,5,5)	3	5	5
(6,3,3)	6	3	3
(6,3,4)	6	3	4
(6,3,5)	6	3	5
(3,6,3)	3	6	3
(3,6,4)	3	6	4

ID	trabajadores	vacantes	p
(3,6,5)	3	6	5
(5,4,3)	5	4	3
(5,4,4)	5	4	4
(5,4,5)	5	4	5
(4,5,3)	4	5	3
(4,5,4)	4	5	4
(4,5,5)	4	5	5
(4,4,3)	4	4	3
(4,4,4)	4	4	4
(4,4,5)	4	4	5

Tabla 3.1: Tablas con las 21 configuraciones de entrada utilizadas para el flujo de trabajo principal.

3.1.1. Configuraciones de Entrada

Una configuración de entrada es un conjunto de hiperparámetros inicializados que definen como se comporta el flujo de trabajo principal. Como su nombre indica, son la entrada a este procedimiento. Se definen un total de 21 configuraciones de entrada, en las que la mayoría de los hiperparámetros se mantienen fijos con el fin de asegurar un tamaño de muestra asequible, teniendo en cuenta las limitaciones del hardware. Consecuentemente, solo un subconjunto de los hiperparámetros ha sido seleccionado para su análisis comparativo, el cual se describirá posteriormente.

En todas las configuraciones, los coeficientes de ganancia se establecen en 1, garantizando que tanto la afinidad como la prioridad de los trabajos tengan la misma importancia. Por otro lado, los coeficientes de penalización se configuran como $\lambda_1^R = 2,5$ y $\lambda_2^R = 3$. Como se indicó en la Sección 2.4.2, estos valores fueron determinados por medio de la utilización de heurísticas a partir de pruebas informales realizadas sobre instancias pequeñas de cuatro y seis qubits. Si bien se espera que estos valores resulten adecuados para instancias de mayor tamaño, no es posible demostrar formalmente que ésto sea así.

Para la configuración de QAOA, se utiliza una inicialización de los parámetros variacionales TQA, técnica presentada en Sección 2.3.3. Como optimizador clásico, se utiliza el método de Powell [69], con un máximo de 10.000 iteraciones y una tolerancia de 0,01. En cada iteración de optimización, se realizan 10.000 mediciones del ansatz para calcular el valor esperado, mientras que en la etapa de evaluación solo se efectúan 20 mediciones. Esta etapa de evaluación permite determinar cuántas soluciones candidatas serán encontradas por QAOA una vez finalizada la optimización. Es importante destacar que dichas soluciones deben superar el filtro de mejor candidato, detallado en la Figura 3.5, para poder ser consideradas como soluciones válidas.

Finalmente, se definen aquellos hiperparámetros que son explorados durante el flujo de trabajo principal. Éstos incluyen la cantidad de trabajadores, la cantidad de trabajos vacantes y el hiperparámetro p de QAOA. En la Tabla 3.1 se presentan estos hiperparámetros distribuidos en 21 configuraciones distintas, cada una identificada por la tupla $(trabajadores, puestosVacantes, p)$. Dado que cada configuración genera una submuestra de cinco instancias del problema JRP implementadas en QAOA, la muestra de salida se conforma de un total de $21 \times 5 = 105$ instancias JRP analizadas.

La selección del hiperparámetro $p \in \{3, 4, 5\}$ para el muestreo se justifica a partir de hallazgos establecidos sobre el rendimiento del algoritmo y las capacidades del hardware. Particularmente, se sabe que el entrenamiento de QAOA se satura en una profundidad $p^* = n$, lo que significa que incrementar la profundidad más allá del número de qubits n no produce mejoras adicionales [70]. Una instancia de JRP tiene un ancho de circuito de $trabajadores \times puestosVacantes = n$,

resultando en puntos de saturación alrededor de $p^* \in \{15, 16, 18, 20\}$.

Por lo tanto, elegir valores de hiperparámetro $p \in \{3, 4, 5\}$ debería permitir observar mejoras significativas en el rendimiento sin alcanzar las limitaciones de saturación. Además, estas profundidades están directamente respaldadas por estudios existentes. Se han realizado pruebas numéricas de QAOA para sistemas de hasta $N = 20$ qubits, confirmando la relevancia de estudiar el rendimiento en estas escalas de tamaño de problema [71].

Además, un estudio reciente entre múltiples computadoras cuánticas utilizando el protocolo *Linear Ramp* QAOA (LR-QAOA) incluyó instancias de 56 y 28 qubits con profundidades tan bajas como $p = 3$, considerándolas informativas y relevantes para evaluar y comparar diferentes proveedores de hardware [72].

3.1.2. Métricas de Rendimiento Algorítmico

En la Figura 3.4 se presentan las métricas que permiten estudiar el rendimiento algorítmico de QAOA para JRP. La primera de ellas, de elaboración propia, es la **diferencia Ising**, definido como la diferencia entre el valor esperado del ansatz cuando se utilizan los parámetros variacionales optimizados y cuando se emplean los parámetros inicializados con TQA. Esta diferencia no está operada por valor absoluto, por lo tanto, conserva su signo. Ésto implica que un signo $+$ o $-$ refleja la tendencia creciente o decreciente de la diferencia. En particular, una diferencia con signo $-$ sugiere que la optimización de los parámetros conduce a un valor esperado menor (interpretado como costo), lo que indicaría el correcto funcionamiento del algoritmo desde una perspectiva teórica.

Luego, se introduce la **razón de aproximación** [45], que se define como un valor entre 0 y 1 que indica cuán aproximada es una solución obtenida respecto a la solución óptima. A más cercano esté el valor a 1, más aproximada es la solución obtenida. Para un problema de maximización, la razón de aproximación R se puede calcular como lo indica la Ecuación (3.1), donde $f(x)$ es la ganancia para una solución obtenida x y $\max_x f(x)$ es la ganancia para la solución óptima. Esta métrica es ampliamente utilizada en la investigación de algoritmos aproximados para problemas de optimización y permite evaluar la calidad de las soluciones obtenidas con QAOA.

$$\frac{f(x)}{\max_x f(x)} \geq R \quad (3.1)$$

Por último, se define la **mejora de ganancia**, también de elaboración propia, una métrica que cuantifica el impacto de los resultados obtenidos con QAOA en la productividad de una industria u organización. Se calcula como el incremento porcentual entre la ganancia de la solución obtenida con QAOA y la ganancia de la solución que implica no realizar reasignaciones. Esta métrica es complementaria a la razón de aproximación, ya que alcanzar soluciones cercanas a la óptima no siempre implica un impacto significativo en la productividad. En algunos casos, la aplicación de QAOA puede no justificar su uso si los beneficios en términos de productividad son marginales, incluso cuando las soluciones son altamente aproximadas.

3.2. Procedimientos de Postprocesamiento

Tras haber ejecutado el flujo de trabajo principal y haber obtenido sus resultados, se ejecutan un conjunto de procedimientos de postprocesamiento que brindan mayor información sobre la factibilidad de aplicar QAOA para JRP.

Entre éstos, se encuentran dos procedimientos: la generación de curvas de mediciones requeridas y de probabilidad de medición, que cuantifican la relación entre la razón de aproximación esperada por el usuario y la cantidad de mediciones requeridas; y la aplicación de Aprendizaje por Transferencia, técnica presentada en la Sección 2.3.4 que busca analizar la factibilidad de reutilizar parámetros variacionales ya optimizados para una instancia JRP en otra diferente.

3.2.1. Generación de Curvas de Mediciones Requeridas y Probabilidad de Medición

Una vez optimizado el ansatz para generar soluciones aproximadas de una instancia JRP, corresponde realizar un análisis más profundo de la relación entre la razón de aproximación esperada por el usuario y la cantidad de mediciones requeridas para su obtención. Con este objetivo, se define un procedimiento que permite generar dos conjuntos de curvas de elaboración propia denominadas **curvas de mediciones requeridas** y **curvas de probabilidad de medición**. El primer conjunto se utiliza para cuantificar el incremento en la cantidad de mediciones requeridas a medida que se busca una mejor aproximación a la solución óptima. Mientras que el segundo, considerando una cantidad fija de mediciones, cuantifica la disminución en la probabilidad de obtener soluciones candidatas a medida que su razón de aproximación incrementa.

Por otro lado, una curva de mediciones requeridas se define como una función en un plano, donde el eje X corresponde a la mínima razón de aproximación esperada con una probabilidad media de 0,9, y el eje Y indica la cantidad de mediciones requeridas para cumplir tales condiciones. A su vez, una mínima razón de aproximación x está dada por la la Función de Distribución Acumulativa Complementaria (FDAC), definida en la Ecuación (3.2).

$$\text{mínima razón de aproximación } x = \bar{F}_X(x) = P(X \geq x) \quad (3.2)$$

Es importante notar que el eje X involucra una probabilidad media 0,9, lo que indica que incluso aunque se realicen la cantidad de mediciones estipuladas por la curva, existe una probabilidad media de 0,1 de no obtener una solución que cumpla la aproximación esperada. Ésto sucede debido a la naturaleza probabilística del algoritmo. El valor de 0,9 fue configurado heurísticamente, ya que se lo considera lo suficientemente alto como para asegurar con muy alta probabilidad que se obtiene una solución lo suficientemente aproximada.

Además, al hablar de una probabilidad media, se da a entender que existe una varianza asociada a dicha estimación, es decir, que se encuentra definida dentro de un intervalo de confianza. Para su cálculo, se utilizó un nivel de confianza de 0,95; un valor estándar y ampliamente recomendado para obtener estimaciones confiables sin una varianza excesiva. El intervalo de confianza fue calculado a partir de una muestra compuesta por 1.000 conjuntos de 10.000 mediciones del ansatz con los parámetros optimizados, lo que garantiza una base estadística robusta para la estimación de la probabilidad media y su variabilidad.

En cuanto a lo anterior, es posible calcular una curva de mediciones requeridas para cada uno de los 105 individuos de la muestra de salida del flujo de trabajo principal. Aun así, ésto resultaría en un conjunto excesivamente grande de curvas, lo que dificultaría el análisis de patrones entre las mismas. Para simplificar el estudio, se agrupan las curvas en función del tamaño de su instancia JRP y de su valor del hiperparámetro p , para finalmente calcular una curva media para cada categoría. Con ésto, se obtienen dos conjuntos diferentes de curvas de mediciones requeridas, siendo el primero un conjunto de cuatro curvas (correspondientes a instancias de 15, 16, 18 y 20 qubits) y el segundo uno de tres curvas (correspondientes a $p = 3$, $p = 4$ y $p = 5$).

Por otro lado, una curva de probabilidad de medición también se define como una función en un plano, donde el eje X corresponde a la mínima razón de aproximación esperada en alguna de las 100 mediciones hechas, y el eje Y indica la probabilidad de cumplir tales condiciones. Para estas curvas, también se trabaja con intervalos de confianza para la estimación de la probabilidad, por lo que cada punto en el eje X corresponde a un intervalo de confianza para la probabilidad del eje Y . Al igual que con las curvas de mediciones requeridas, se puede calcular una curva de probabilidad de medición para cada uno de los 105 individuos de la muestra de salida del flujo de trabajo principal, y se promedian en función de su tamaño de instancia JRP y de su valor del hiperparámetro p .

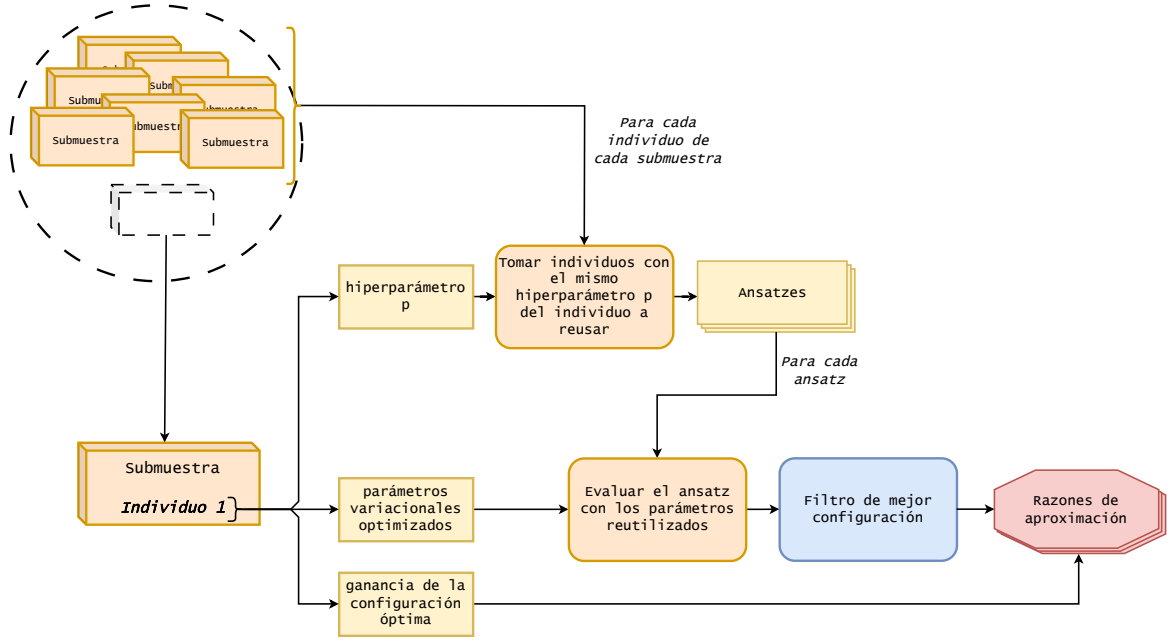


Figura 3.6: Aplicación de Aprendizaje por Transferencia.

3.2.2. Aplicación de Aprendizaje por Transferencia

En el procedimiento que se desarrolla a continuación se implementa el Aprendizaje por Transferencia, introducido previamente en la Sección 2.3.4. El objetivo es analizar si el problema JRP es adecuado para la aplicación de esta técnica, con el fin de reducir el costo asociado a múltiples optimizaciones de parámetros en distintas instancias del problema. Para facilitar el análisis, se restringe a aplicar Aprendizaje por Transferencia solo entre aquellos ansatzes que comparten el mismo valor de hiperparámetro p . Además, no se realiza una segunda ronda de optimización sobre los parámetros reutilizados en el segundo ansatz, sino que se miden directamente soluciones candidatas. Cabe destacar que, si bien esta técnica puede ser aplicada entre ansatzes con diferentes valores de p y permite una segunda ronda de optimización para refinar los resultados, dichas variantes no son abordadas en esta tesis.

Como se explicó previamente en la Sección 3.1, la muestra de salida generada por el flujo de trabajo principal está compuesta por 21 submuestras, cada una conformada por cinco instancias del problema JRP ejecutadas mediante QAOA. Por lo tanto, la muestra total consta de $21 \times 5 = 105$ instancias JRP.

La aplicación del Aprendizaje por Transferencia, cuyo diseño se muestra en la Figura 3.6, comienza con la selección del primer individuo de alguna de las 21 submuestras. Este individuo contiene los parámetros variacionales optimizados que se reutilizarán. Luego, entre los 104 individuos restantes, se seleccionan aquellos ansatzes cuyo hiperparámetro p coincide con el del individuo seleccionado. En base a las configuraciones establecidas en la Tabla 3.1, existen 34 ansatzes que satisfacen esta condición, independientemente del individuo seleccionado.

Cada uno de estos ansatzes es configurado con los parámetros reutilizados, se realiza un conjunto de mediciones, se extraen las mejores soluciones candidatas y se calculan sus respectivas razones de aproximación. Este procedimiento se repite para el primer individuo de cada una de las 21 submuestras de la muestra de salida del flujo de trabajo principal, lo que da como resultado una nueva muestra compuesta por $21 \times 34 = 714$ razones de aproximación generadas mediante Aprendizaje por Transferencia.

En el siguiente capítulo, se presentarán las herramientas utilizadas y el entorno de ejecución seleccionado para la implementación de los procedimientos. Además, se detallarán y ejemplifica-

rán las estructuras de datos y secciones de código principales que forman parte de la implementación.

Capítulo 4

Implementación de los Procedimientos

A continuación, se detallan los aspectos relevantes de la implementación de los procedimientos presentados en el Capítulo 3. La Sección 4.1 detalla las herramientas utilizadas, mientras que la Sección 4.2 presenta el entorno de ejecución. Luego, la Sección 4.3 define las estructuras de datos principales utilizadas para el almacenamiento de los resultados. Finalmente, la Sección 4.4 detalla algunas de las partes de código más importantes.

4.1. Herramientas Utilizadas

Las herramientas que se presentarán a continuación se utilizan para implementar la carga de trabajo de los procedimientos, es decir, la realización del muestreo, el calculo de métricas de rendimiento algorítmico, la implementación de QAOA, la operación de estructuras de datos, entre otras tareas.

Esta tesis tiene un enfoque experimental, es decir, involucra la ejecución de procedimientos que generen resultados para un posterior análisis de los mismos. Para facilitar este proceso y aprovechar la amplia disponibilidad de bibliotecas orientadas al análisis de datos, se optó por utilizar como entorno de trabajo el ecosistema Anaconda en combinación con Jupyter Notebook. En primer lugar, Anaconda es un ecosistema que facilita la creación de ambientes aislados que permiten instalar diferentes herramientas, como librerías de Python o R, sin que las mismas generen conflictos con el resto del sistema operativo [73]. Su uso es recurrente para la realización de proyectos de ciencia de datos y computación cuántica. Por otra parte, Jupyter Notebook es una de las herramientas disponibles en Anaconda. Se define como una aplicación de autoría de cuadernos, es decir, archivos conformados por celdas individuales que permiten ejecutar código, imprimir texto y otros elementos multimedia [74]. El código ejecutado en cada celda es interpretado individualmente, lo que lo vuelve una herramienta de gran utilidad para la implementación de experimentos y pruebas que requieran visualizar el estado intermedio de los datos.

Finalmente, para la programación de los procedimientos se selecciona el lenguaje Python ya que es ampliamente utilizado por la comunidad de la computación cuántica, y también, aquel que cuenta con una mayor cantidad de librerías para esta área. En esta tesis se utilizó Numpy y Pandas, que simplifican la manipulación de estructuras tensoriales [75, 76]; y Matplotlib y Seaborn, que permiten generar gráficos de visualización para datos numéricos [77, 78].

4.1.1. Comparación entre Herramientas para QAOA

En la comunidad de la computación cuántica, existen diversas herramientas para implementar QAOA que compiten con diversas prestaciones y ventajas. Entre éstas, se encuentran Qiskit, OpenQAOA y QRisP.

Qiskit¹ es un kit de desarrollo de software, o *Software Development Kit* (SDK), de código

¹Web de Qiskit (último acceso: 01/08/2025): <https://www.ibm.com/quantum/qiskit>

abierto orientado a computación cuántica y desarrollado por IBM en 2017 [79]. Proporciona una librería de programación, simuladores clásicos, interfaces de acceso a hardware cuántico, entre otras funcionalidades. Además, incorpora su propia implementación de QAOA, que también permite definir los hiperparámetros esenciales. Aun así, Qiskit implementa QAOA desde un nivel de abstracción muy bajo. Para la presente tesis, no se requiere una configuración a dicho nivel de abstracción del algoritmo. Por ello, y para facilitar la implementación y depuración del código, se determinó no utilizar este software.

Por otro lado, OpenQAOA² es una librería de código abierto creada por Entropica Labs en 2022 [80]. Su objetivo es simplificar la creación, configuración y ejecución del algoritmo QAOA. Cuenta con interfaces hacia otras herramientas relevantes en la comunidad, tales como Qiskit, PennyLane³ y Bracket⁴. Gracias a esto, permite utilizar los simuladores clásicos o hardware cuántico que ofrecen estos proveedores. Cuenta con nivel de abstracción mayor que Qiskit, ideal para el tipo de configuraciones que se necesitan en la presente tesis.

Finalmente, QRisp es un marco de trabajo de alto nivel para computación cuántica desarrollado en 2024 por Eclipse [81]. Esta herramienta implementa un conjunto de algoritmos, entre los que se encuentra QAOA. En comparación con OpenQAOA, el nivel de abstracción es similar, aunque proporciona ciertas configuraciones extras. De esta forma, lo vuelve una herramienta a considerar para la implementación. Aun así, la robustez o estabilidad del software y los aportes de la comunidad que lo sostiene, pueden no estar a la altura de OpenQAOA, que cuenta con más años de desarrollo. Ésto, nuevamente, puede dificultar el proceso de depuración del código, en caso de que sucedan problemas inesperados con la herramienta.

Los problemas mencionados de Qiskit y QRisp dieron lugar a decidir por OpenQAOA como la herramienta a utilizar para implementar el algoritmo QAOA. Ésta cuenta con los elementos necesarios para que se lleven a cabo los procedimientos exitosamente, mientras que facilita el proceso de codificación y depuración de errores.

4.1.2. Introducción a la Librería OpenQAOA

OpenQAOA fue utilizada como parte del solucionador QAOA, presentado previamente en la Figura 3.5 e implementado en el Código B.1. En este código, se observa que existe un conjunto de métodos predefinidos por OpenQAOA que reciben los hiperparámetros del algoritmo y se encargan de abstraer al programador de su configuración. Algunos de estos métodos son `set_circuit_properties` y `set_classical_optimizer`. Además, se utiliza el método `optimize` para automatizar la optimización de los parámetros variacionales, permitiendo que el programador acceda directamente a los resultados en `qaoa.result`.

OpenQAOA también cuenta con una estructura de resultados bien definida, que permite realizar una caracterización básica de lo sucedido en una ejecución de QAOA. La estructura general, implementada como un diccionario, se observa en el Código A.1. En la misma, se define el método de optimización clásica elegido, la estructura del Hamiltoniano de Costo, la cantidad de iteraciones del optimizador, las soluciones candidato más probables, la estructura `optimized` y el historial de costos de la función Ising. De estos datos, el más relevante es la estructura `optimized`, que muestra los resultados posteriores a la optimización clásica, entre éstos, los parámetros variacionales optimizados, el costo de función Ising logrado, una muestra preliminar de mediciones y la cantidad de iteraciones de optimizador necesarias para llegar a estos datos. Cabe destacar que alguno de los datos y métricas presentados en el Capítulo 3 no se encontraban directamente en esta estructura, y que por lo tanto, se realizó cómputo adicional para obtenerlos.

²Web de OpenQAOA (último acceso: 01/08/2025): <https://openqaoa.entropicalabs.com/>

³Web de PennyLane (último acceso: 01/08/2025): <https://pennylane.ai/>

⁴Web de Bracket (último acceso: 01/08/2025): <https://aws.amazon.com/es/braket/>

4.2. Entorno de Ejecución

El entorno de ejecución se refiere al hardware y módulos de software utilizados para ejecutar los procedimientos implementados. La gran carga de cómputo que requiere el muestreo a realizar vuelve inviable la ejecución de las pruebas en una computadora de uso personal. De manera ilustrativa, pruebas preliminares con instancias JRP de entre 15 y 20 qubits en QAOA arrojaron tiempos de ciclo de optimización de entre tres y cuatro horas por instancia, dependiendo del caso. Para una muestra de 105 instancias, esto implicaba un tiempo total estimado de entre 315 y 420 horas de ejecución continua.

Con el fin de solventar esta limitación, el Instituto Tecnológico de Castilla y León⁵ proporcionó acceso a un nodo de cómputo de alto rendimiento con la capacidad suficiente para ejecutar las pruebas. Gracias a este recurso, cada instancia pudo resolverse en un intervalo de entre 20 y 30 minutos.

El nodo de cómputo está basado en un servidor Tyan Thunder HX FT83AB7129, equipado con dos procesadores Intel Xeon Gold 6326 (32 núcleos y 64 hilos en total) y 512 GB de RAM ECC DDR4. Su almacenamiento está compuesto por dos SSD Micron 7450 de 1,92 TB en RAID 0. También incluye seis GPUs Nvidia A30, dos A100 y una H100, todas interconectadas mediante NVLink. La conectividad se completa con dos interfaces InfiniBand ConnectX-6 de 200 Gbps.

Además, como ya ha sido mencionado, la simulación de cómputo cuántico en hardware clásico requiere de módulos de software denominados simuladores de cómputo cuántico. Haciendo uso de la interfaz de conexión entre OpenQAOA y Qiskit, se utilizó uno de los simuladores proporcionados por Qiskit para las ejecuciones de QAOA.

4.3. Estructuras de Datos Principales

Las estructuras de datos principales son aquellas que encapsulan los resultados del flujo de trabajo principal, presentado en la Sección 3.1.

Como ya se explicó, la muestra de 105 ejecuciones de QAOA se subdivide en 21 submuestras de cinco ejecuciones de QAOA cada una. Estas submuestras son almacenadas en un archivo JSON individualmente, tal como lo indica el Código A.2. Estas estructuras comienzan guardando la información de la configuración de entrada que le corresponde, es decir, la configuración de las instancias de JRP y de los hiperparámetros de QAOA. Luego, cada individuo de la muestra, que representa una ejecución de QAOA con JRP, se conforma de la instancia JRP, las métricas de rendimiento algorítmico, la solución candidato óptima, la solución candidato obtenida por QAOA y la estructura de datos generada por OpenQAOA (que se obtiene de `qaoa.result`).

También se utilizan archivos JSON para almacenar los resultados de la aplicación de Aprendizaje por Transferencia, uno de los procedimientos de postprocesamiento presentados en la Sección 3.2.2. Su estructura se define como lo indica el Código A.3. El archivo empieza definiendo la instancia a reutilizar, es decir, el archivo JSON de la submuestra seleccionada y el índice de la instancia JRP cuyos parámetros optimizados serán tomados. Luego, se generan una subestructura por cada submuestra cuyo hiperparámetro p coincida con el de la instancia a reusar. En estas subestructuras se guardan las razones de aproximación para cada instancia JRP ejecutada con los parámetros de reutilización. También se almacena la solución candidato óptima y la encontrada por QAOA.

4.4. Secciones de Código Principales

A continuación, se detallarán algunas de las secciones de código más relevantes de la implementación. Para comenzar, el flujo de trabajo principal es invocado desde el cuaderno `experiment-`

⁵Web del Instituto Tecnológico de Castilla y León (último acceso: 01/08/2025): <https://itcl.es/en/>

`_sampleOptimization`⁶. En el mismo se definen las configuraciones de entrada, compuestas por las configuraciones para inicializar instancias de JRP y los hiperparámetros de QAOA, presentados en el Código B.2 y el Código B.3 respectivamente. Luego, como se muestra en el Código B.4, se codifica una estructura iterativa para combinar esas configuraciones entre sí. Aquí se hace uso de la clase `TestQAOASolver`⁷, que representa la lógica del flujo de trabajo principal.

De esta clase, se invocan los métodos `set_fixedInstances` y `sample_workflows_with_fixedInstances`, que permite generar las instancias JRP para posteriormente ejecutar el flujo de trabajo principal. En el Código B.5 se observa el segundo método mencionado.

El método `run_workflow` de esta misma clase, que se observa en el Código B.6, implementa gran parte de la lógica del analizador de rendimiento algorítmico. Comienza definiendo la solución inicial de la formulación estándar del problema, es decir, aquella en la que no hay reasignaciones, junto a la ganancia de la misma. Luego, se ejecuta la implementación del solucionador por fuerza bruta para obtener la solución óptima del problema. A continuación, se utiliza el solucionador QAOA para poder calcular el costo de la función Ising, preoptimización y postoptimización, junto a la mejor solución candidato encontrada por el algoritmo y su ganancia. Haciendo uso de estos resultados, se calculan las métricas de rendimiento algorítmico y otros resultados propios de la implementación, que son finalmente retornados.

Por otro lado, la aplicación de Aprendizaje por Transferencia es invocada desde el cuaderno `experiment_useOptimizedParameters`⁸. En el Código B.7, se observa la sección de código que define un `EvaluationQAOASolver`⁹ encargado de aplicar la lógica del postprocesamiento. Luego, haciendo uso del método `sample_workflows`, se reutilizan los parámetros variacionales optimizados del primer individuo de cada una de las 21 submuestras. La implementación de la clase `EvaluationQAOASolver` se presenta en el Código B.8.

⁶URL para acceder al cuaderno `experiment_sampleOptimization` (último acceso: 01/08/2025): https://github.com/AdrianoLusso/QuantumComputing_for_JobReassignmentProblem/blob/main/experiment_testQAOASolver/real/experiment_sampleOptimization.ipynb

⁷URL para acceder a la clase `TestQAOASolver` (último acceso: 01/08/2025): https://github.com/AdrianoLusso/QuantumComputing_for_JobReassignmentProblem/blob/main/functions/TestQAOASolver.py

⁸URL para acceder al cuaderno `experiment_useOptimizedParameters` (último acceso: 01/08/2025): https://github.com/AdrianoLusso/QuantumComputing_for_JobReassignmentProblem/blob/main/experiment_testQAOASolver/real/results/experiment_reuseOptimizedParameters/experiment_useOptimizedParameters.ipynb

⁹URL para acceder a la clase `EvaluationQAOASolver` (último acceso: 01/08/2025): https://github.com/AdrianoLusso/QuantumComputing_for_JobReassignmentProblem/blob/main/functions/EvaluationQAOASolver.py

Capítulo 5

Resultados y Discusiones

En este capítulo, se presentan y discuten los resultados de los procedimientos introducidos en el Capítulo 3. Desde la Sección 5.1 a la Sección 5.4 se detallan los resultados obtenidos por el flujo de trabajo principal. Particularmente, las métricas de rendimiento algorítmico: la razón de aproximación, diferencia Ising y mejora de ganancia; y la cantidad de iteraciones del optimizador, dato propio de la implementación hecha. Luego, la Sección 5.5 y la Sección 5.6 detallan los resultados de los postprocesamientos que generaron las curvas de mediciones requeridas, la curvas de probabilidad de medición y las razones de aproximación de la aplicación de Aprendizaje por Transferencia. Finalmente, la Sección 5.7 presenta un resumen de las principales conclusiones hechas en las secciones previas.

5.1. Razón de Aproximación

En la Figura 5.1 se observa la razón de aproximación de las instancias de JRP. Por cada una de las 21 configuraciones (*trabajadores, puestosVacantes, p*) se muestran las razones de aproximación de sus respectivas cinco instancias. A cada configuración le corresponde una categoría en el eje X , y cada índice de instancia se representa con una forma distinta para los puntos.

A partir de la figura anterior, se elabora la Figura 5.2, que presenta dos gráficos de cajas en los que se agrupan las mismas razones de aproximación según el tamaño del problema (cantidad de qubits utilizados) y el hiperparámetro p . Una de las categorías de cada gráfico representa al muestreo completo, indicando que para esa caja se utilizaron las 105 instancias JRP disponibles. Los gráficos de cajas serán utilizados de manera recurrente en este capítulo, ya que permiten visualizar diferentes estadísticas de un grupo de datos. Como su nombre indica, se conforman de una estructura de caja por cada categoría del eje X . Cada caja permite visualizar estadísticas del conjunto de datos asociados a cada categoría. La línea amarilla representa la media de los datos. El primer y tercer cuartil son representados con el límite inferior y superior de las cajas. Los valores mínimo y máximo se visualizan con el límite inferior y superior de las líneas sobresalientes a la cajas. Finalmente, los valores atípicos de la distribución son representados como puntos y se calculan siguiendo el método de Tukey [82]. Este procedimiento identifica como atípicos aquellos valores que se encuentran fuera del rango definido por los límites de la Ecuación (5.1) y Ecuación (5.2), donde Q_1 y Q_3 son el primer y tercer cuartil, respectivamente, y IQR es el rango intercuartílico.

$$\text{Límite inferior} = Q_1 - 1.5 \times IQR \quad (5.1)$$

$$\text{Límite superior} = Q_3 + 1.5 \times IQR \quad (5.2)$$

Al analizar los gráficos de cajas de la razón de aproximación, **los resultados son muy favorables**, con valores que oscilan principalmente entre 0,85 y 0,97, y una media cercana a 0,9. Estos altos valores indican que el algoritmo ha sido efectivo en la aproximación de la solución óptima para JRP en la mayoría de los casos. Además, se observa una ligera tendencia

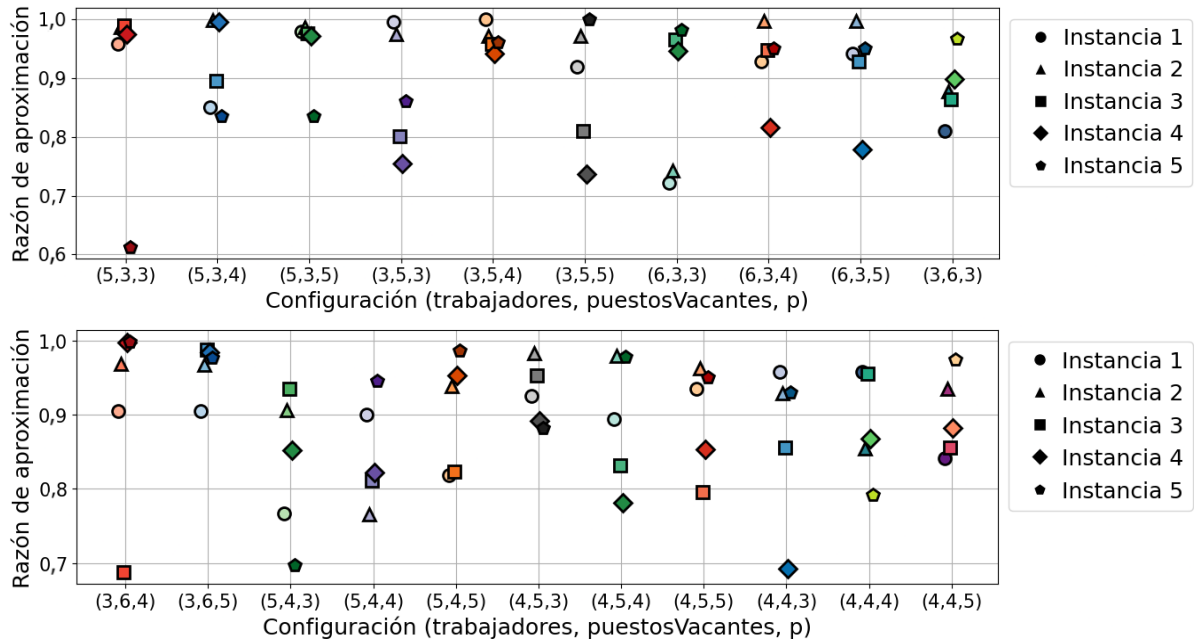


Figura 5.1: Razón de aproximación de las 105 instancias de JRP.

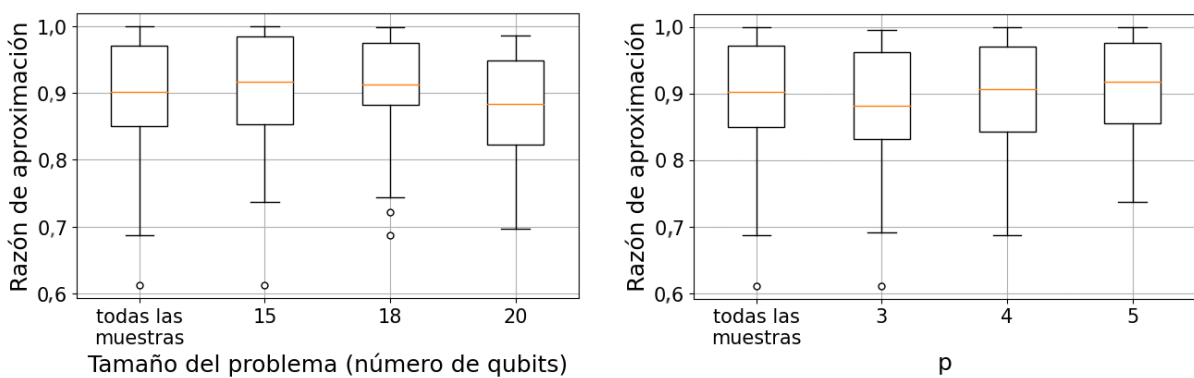


Figura 5.2: Estadísticas de razón de aproximación agrupando por tamaño del problema a la izquierda y por hiperparámetro p a la derecha.

a la disminución de la razón de aproximación conforme aumenta el tamaño del problema. Esta tendencia es esperable, ya que un mayor tamaño del problema introduce un aumento exponencial del espacio de búsqueda, lo que puede dificultar la obtención de soluciones cercanas al óptimo. En particular, se nota una mayor caída en la razón de aproximación al aumentar el número de qubits de 18 a 20, en comparación con el cambio de 15 a 18 qubits. Este comportamiento sugiere que **la caída de la razón de aproximación sigue una relación de orden superior al lineal con el tamaño del problema**. Sin embargo, para confirmar esta observación, sería necesario analizar tamaños de problema aun mayores.

Otro hallazgo interesante es que la razón de aproximación media aumenta conforme lo hace el hiperparámetro p . Este comportamiento está alineado con las expectativas, en las que una mayor profundidad en el ansatz y una cantidad adecuada de iteraciones del optimizador clásico resultaría en mejores soluciones, lo que proporciona **evidencia de que el algoritmo esta minimizando exitosamente el valor esperado del ansatz**. Además, también se observa una reducción en la varianza de la razón de aproximación junto al incremento de p . Ésto sugiere que, al seleccionar un valor de p suficientemente grande, no solo se mejora la calidad de las soluciones obtenidas, sino que también **se incrementa la robustez del algoritmo** al realizar distintos muestreos de soluciones candidato. Esto último tiene implicaciones prácticas importantes. En un entorno real, la misma instancia de problema JRP puede necesitar ser resuelta en diferentes momentos a lo largo de un período prolongado (por ejemplo, meses o años). En este caso, un valor adecuado del hiperparámetro p asegura que entre diferentes muestreos se obtengan razones de aproximación lo suficientemente parecidas entre sí, debido a la baja variabilidad de su distribución.

5.2. Diferencia Ising

En la Figura 5.3 se observa la diferencia Ising de las instancias de JRP. Por cada una de las 21 configuraciones (*trabajadores, puestosVacantes, p*) se muestran las diferencias para sus respectivas cinco instancias. A cada configuración le corresponde una categoría en el eje X , y a cada índice de instancia una forma distinta para los puntos. Como se detalló en la Sección 3.1.2, esta métrica se define como la diferencia entre el valor esperado del ansatz cuando se utilizan los parámetros variacionales optimizados y cuando se emplean los parámetros inicializados con TQA. Diferencias de Ising en valores negativos indican un funcionamiento correcto de QAOA desde el apartado teórico.

A partir de la figura anterior, se elabora la Figura 5.4, que presenta dos gráficos de cajas en los que se agrupan las mismas diferencias Ising según el tamaño del problema (cantidad de qubits utilizados) y el hiperparámetro p . Una de las categorías del gráfico representa al muestreo total, indicando que para esa caja se utilizaron las 105 instancias JRP disponibles.

Los resultados son coherentes con los hallazgos previos de la razón de aproximación ya que las diferencias Ising se mantienen en altos valores negativos, es decir, valores negativos lejanos al 0. Ésto vuelve a sugerir que **el algoritmo logra minimizar el valor esperado del ansatz**.

Al analizar las distribuciones agrupadas por tamaño de problema, se observa que las diferencias Ising se alejan consistentemente de cero en dirección negativa a medida que crece el tamaño del problema. En una primera hipótesis, se podría suponer que ésto está relacionado con el número de reasignaciones posibles entre diferentes instancias JRP. Las instancias más grandes probablemente tienen más trabajos vacantes, lo que proporciona más reasignaciones potenciales que disminuyen el costo Ising tanto de la solución óptima como de sus soluciones aproximadas. Si ésto es cierto, este fenómeno **no debe interpretarse como un factor significativo** que influye en la capacidad de QAOA para encontrar soluciones más aproximadas en instancias más grandes. En cambio, refleja la tendencia de las instancias más grandes a tener soluciones óptimas con menores costos Ising, causando una mayor distancia respecto al costo Ising inicial.

Una segunda hipótesis sugiere que el crecimiento exponencial del espacio de búsqueda, a medida que aumenta el tamaño del problema, podría hacer que la inicialización TQA comience

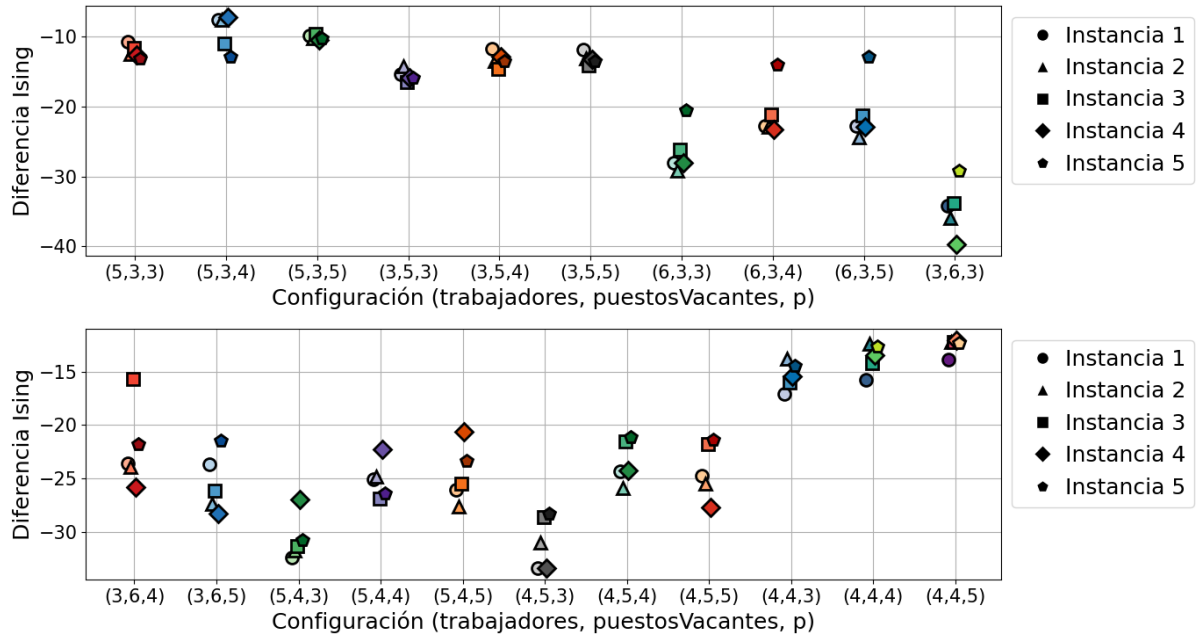


Figura 5.3: Diferencia Ising de las 105 instancias de JRP.

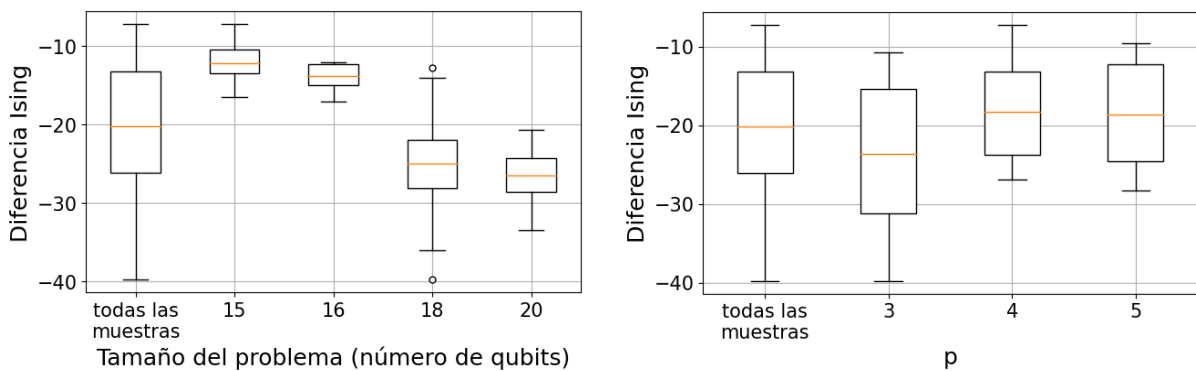


Figura 5.4: Estadísticas de diferencia Ising agrupando por tamaño del problema en la figura izquierda y por hiperparámetro p en la figura derecha.

con parámetros variacionales peores, es decir, más alejados de los parámetros óptimos efectivos. Esto, a su vez, podría dar lugar a que la diferencia Ising incremente en la dirección negativa y resulte, probablemente, en una **mayor cantidad de iteraciones de optimización**. Tanto la primera como la segunda hipótesis podrían ocurrir simultáneamente. Sin embargo, para evaluar las mismas, sería necesaria una comparación más detallada de los costos Ising que contribuyen a las diferencias Ising observadas.

Otro factor importante que puede influir en el éxito del método es la variabilidad de las diferencias Ising a través de los diferentes tamaños de problema. Se pudo observar que la distribución para las instancias de 18 qubits es más dispersa en comparación con la distribución para las instancias de 16 qubits. Esto podría estar relacionado con la configuración del número de trabajadores y trabajos vacantes. Para las instancias de 18 qubits, se establecieron dos configuraciones: una con seis trabajadores y tres trabajos vacantes (denominada $(6, 3, p)$) y la otra con la configuración inversa (denominada $(3, 6, p)$). Para las instancias de 16 qubits, solo se estableció una configuración con cuatro trabajadores y cuatro trabajos. Es posible que el cambio entre las configuraciones $((6, 3, p))$ y $((3, 6, p))$ conduzca a diferentes diferencias Ising, las cuales no pueden generalizarse únicamente en función del tamaño del problema. Si esto es cierto, podría impactar en las razones de aproximación logradas, de modo que las instancias de una configuración podrían generar mayores razones de aproximación que las de la otra. Este hallazgo resalta la **importancia de realizar una comparación por cada configuración** al evaluar el rendimiento de JRP en QAOA, ya que considerar únicamente el tamaño del problema podría llevar a conclusiones erróneas, especialmente a medida que la cantidad de trabajadores y de trabajos vacantes se vuelve más desbalanceada.

Respecto al hiperparámetro p , se esperaba que la diferencia Ising aumentara en dirección negativa a medida que p aumentara, con la esperanza de que esto indicara que QAOA estaba encontrando mejores soluciones. Sin embargo, la tendencia observada fue opuesta. Tras un análisis más detallado, una hipótesis inicial atribuye este comportamiento al método utilizado para inicializar los parámetros variacionales. Como se detalló anteriormente en la Sección 2.3.3, se empleó una inicialización TQA, que surge de una discretización desde el Templado Cuántico hacia el ansatz de QAOA. En trabajos previos [59], se menciona que esta conversión entre el Templado Cuántico y QAOA, junto con la universalidad del Templado Cuántico para $T \rightarrow \infty$, proporciona respaldo teórico para afirmar la universalidad de QAOA para $p \rightarrow \infty$. A partir de lo anterior, en un escenario ideal donde $p \rightarrow \infty$, el ansatz de QAOA no requeriría ninguna optimización, ya que la inicialización TQA correspondería a los parámetros óptimos. Sin embargo, en implementaciones prácticas, cuanto más se aleje p del límite ideal de infinito, mayor será la distancia entre los parámetros óptimos y los inicializados. Esto explica la tendencia observada, en la cual, para valores más grandes de p , que se aproximan al ideal de profundidad infinita, las diferencias Ising también se acercan al valor 0. En otras palabras, el resultado del análisis actual destaca la **fortaleza de la inicialización basada en TQA en combinación con valores mayores de p** , no solo para mejorar la calidad de la solución, sino también para lograr una inicialización más cercana a los parámetros óptimos.

5.3. Cantidad de Iteraciones del Optimizador

En la Figura 5.5 se observa la cantidad de iteraciones del optimizador para las instancias de JRP. Por cada una de las 21 configuraciones (*trabajadores, puestosVacantes, p*) se muestran la cantidad de iteraciones para sus respectivas cinco instancias. A cada configuración le corresponde una categoría en el eje X , y a cada índice de instancia una forma distinta para los puntos.

A partir de la figura anterior, se elabora la Figura 5.6, que presenta un gráfico de cajas en el que se agrupan la cantidad de evaluaciones según el hiperparámetro p . Una de las categorías de cada gráfico representa al muestreo total, indicando que para esa caja se utilizaron las 105 instancias de JRP disponibles.

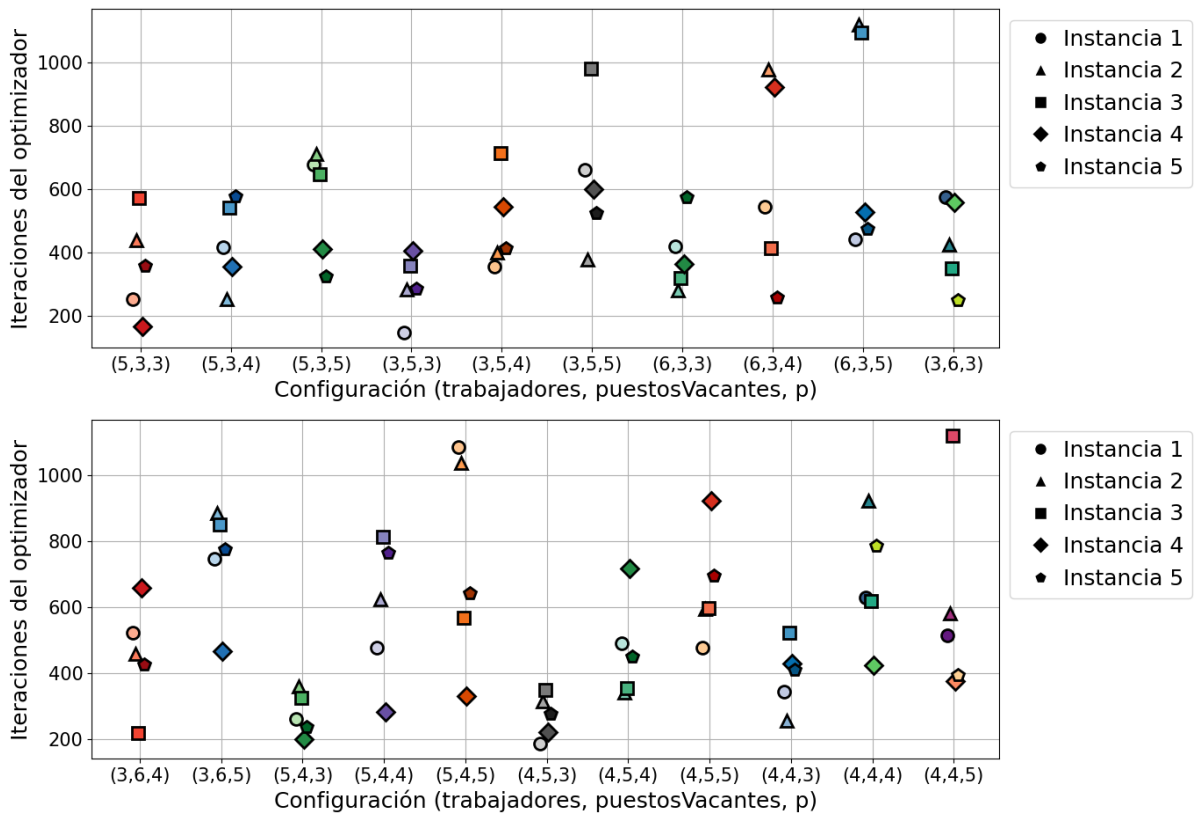


Figura 5.5: Cantidad de iteraciones del optimizador para las 105 instancias de JRP.

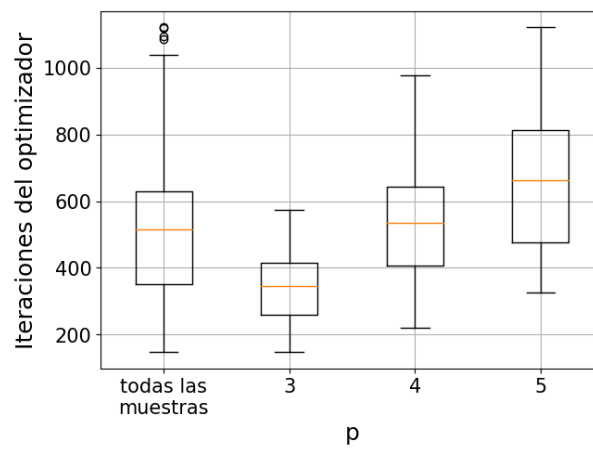


Figura 5.6: Estadísticas de la cantidad de iteraciones del optimizador agrupando por hiperparámetro p .

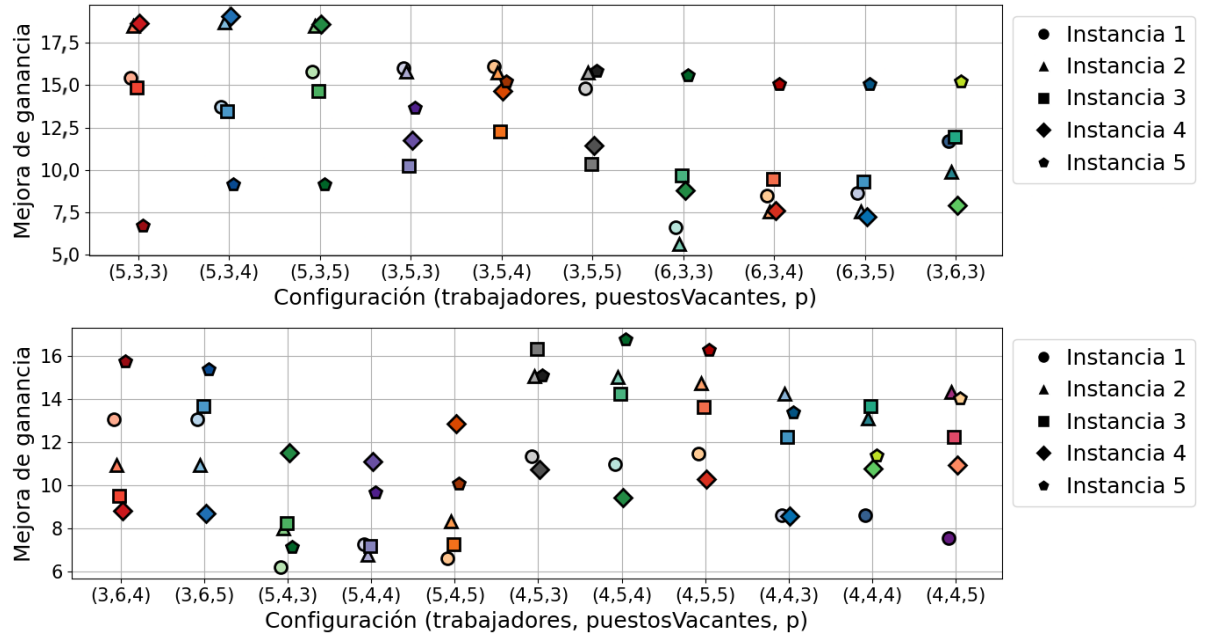


Figura 5.7: Mejora de ganancia de las 105 instancias de JRP.

Todas las instancias se optimizaron en un número de iteraciones significativamente menor que el máximo establecido de 2.000. Éste hecho indica que **el corte de optimización se alcanzó al encontrar un mínimo local** y no por la limitación introducida en el número de iteraciones. La cantidad media de iteraciones necesarias para converger, así como la varianza observada entre las distintas instancias, aumentan a medida que el valor del hiperparámetro p se incrementa. Éste comportamiento es teóricamente esperado, ya que un mayor valor de p implica una mayor cantidad de parámetros a optimizar, lo que naturalmente demanda más iteraciones para alcanzar una solución óptima.

El aumento en la cantidad de iteraciones conforme crece p también está relacionado con uno de los comportamientos descritos en Sección 5.2. Recapitulando el mismo, se discutió que a medida que p crece, los parámetros variacionales iniciales tienden a estar más cercanos de los parámetros óptimos. Sin embargo, como también se introdujo en esta sección, se incrementa la cantidad de parámetros a optimizar, lo que añade complejidad al proceso de optimización. Este fenómeno crea un *trade-off* donde, por un lado, se debe buscar que los parámetros iniciales estén lo suficientemente cerca de los óptimos, pero por otro lado, es necesario considerar que la optimización de un mayor número de parámetros requerirá un mayor número de iteraciones. Por lo tanto, surge la necesidad de realizar un **análisis de equilibrio** que permita **elegir un valor de p que minimice tanto la cantidad de iteraciones del optimizador clásico como la calidad de las soluciones medidas**.

5.4. Mejora de Ganancia

En la Figura 5.7 se observa la mejora de ganancia de las instancias de JRP. Por cada una de las 21 configuraciones (*trabajadores, puestosVacantes, p*) se muestran las mejoras para sus respectivas cinco instancias. A cada configuración le corresponde una categoría en el eje X , y a cada índice de instancia una forma distinta para los puntos.

A partir de la figura anterior, se elabora la Figura 5.8, que presenta un gráfico de cajas en el que se agrupan las mismas mejoras de ganancia según el tamaño del problema (cantidad de qubits utilizados). Una de las categorías de cada gráfico representa al muestreo total, indicando que para esa caja se utilizaron las 105 instancias de JRP disponibles.

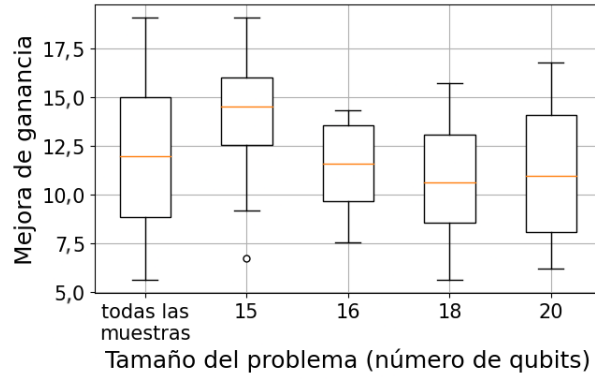


Figura 5.8: Estadísticas de diferencia de ganancia de mejora de ganancia agrupando por tamaño del problema.

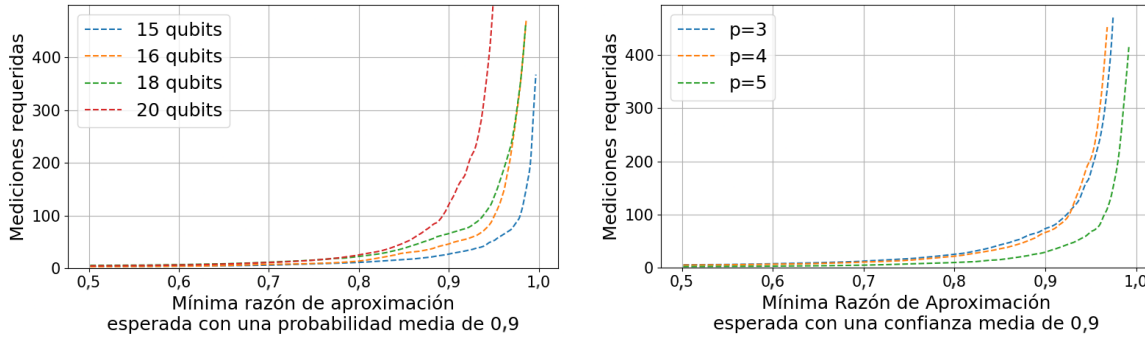


Figura 5.9: Mediciones requeridas para obtener una determinada razón de aproximación con alta probabilidad, agrupando por tamaño del problema en el gráfico izquierdo y por hiperparámetro p en el derecho.

Los resultados muestran que, en un contexto industrial o empresarial, la aplicación de QAOA en JRP logra una mejora general en la ganancia. La misma oscila entre el 9% y el 15%, y tiene una **mejora promedio del 12%**. Además, se observa que a medida que aumenta el tamaño del problema, la mejora en la ganancia muestra una tendencia a la disminución. Esta observación coincide con los análisis previos que comparan el tamaño del problema en relación con la razón de aproximación y la diferencia Ising, lo que refuerza aun más el impacto de la complejidad del problema en el rendimiento de QAOA.

5.5. Curvas de Mediciones Requeridas y Probabilidad de Medición

En la Figura 5.9 se observan las mediciones requeridas para obtener una determinada razón de aproximación con una probabilidad de éxito media de 0,9 y confianza de 0,95. En el gráfico izquierdo, se promediaron las curvas agrupando por tamaño del problema, mientras que en el gráfico derecho se promediaron agrupando por p .

Las curvas obtenidas muestran un incremento exponencial, lo que destaca una de las **principales limitaciones del algoritmo**. En particular, lograr una aproximación más precisa con alta probabilidad de éxito requiere un número exponencial de mediciones en el ansatz, lo que limita la eficiencia del algoritmo.

Además, se observa que las curvas se desplazan hacia la izquierda conforme aumenta el tamaño del problema, indicando que se requieren más mediciones para obtener una aproximación similar.

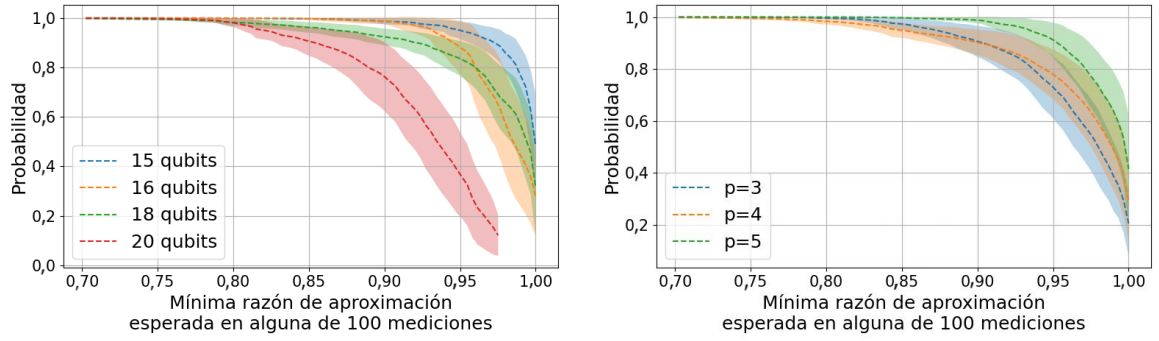


Figura 5.10: Probabilidad de obtener una determinada razón de aproximación en alguna de las 100 mediciones, promediando las curvas al agrupar por tamaño del problema en el gráfico izquierdo y por hiperparámetro p en el derecho.

Por otro lado, al aumentar el valor de p , las curvas se trasladan hacia la derecha, lo que sugiere un comportamiento inverso. Estas **observaciones están alineadas con discusiones previas**, según el análisis sobre las razones de aproximación y las diferencias Ising, donde se explicó que un mayor p mejora la calidad de las soluciones, mientras que un mayor tamaño de problema dificulta su búsqueda.

Es importante destacar que las diferencias en los traslados de las curvas, cuando se agrupan por tamaño de problema, no siguen una tendencia lineal clara. Además, dado que solo se analizaron cuatro, no es posible identificar de manera concluyente la forma exacta de este traslado. Un posible enfoque para futuras investigaciones sería probar con problemas de mayor tamaño y analizar si hay una tendencia clara en el desplazamiento de la curva a medida que el tamaño del problema crece. Asimismo, podría ser útil realizar un análisis similar para el hiperparámetro p .

Un patrón interesante emerge al analizar los resultados para problemas de 20 qubits. En estos casos, la curva posee una asíntota en una razón de aproximación de 0,95, lo que sugiere que aumentar **el tamaño del problema, reduce la razón de aproximación máxima** que se puede obtener con alta probabilidad de éxito, generando una asíntota vertical. Además, cuando el valor de p se configura en 3 o 4, la razón de aproximación nunca llega a 1, lo que indica que un p **relativamente bajo también restringe la máxima razón de aproximación**. En un entorno real, donde el tamaño del problema es intrínseco a la instancia a resolver y no se puede modificar, se pueden sugerir dos opciones para mitigar este inconveniente: por un lado, aumentar el valor de p para mejorar la aproximación, o por otro, reducir la probabilidad de éxito de la razón de aproximación buscada, lo que puede implicar aceptar una menor precisión en la solución.

En la Figura 5.10, se muestra la probabilidad de obtener una razón de aproximación X en alguna de las 100 mediciones con una confianza de 0,95. La probabilidad media se representa mediante una línea punteada, mientras que su margen de error se visualiza como una banda de error. En el gráfico izquierdo se promediaron las curvas agrupando por tamaño del problema, y en el gráfico derecho se promediaron agrupando por hiperparámetro p .

Los resultados obtenidos en la curva de probabilidad de medición son consistentes con los observados en la Figura 5.9. En particular, se observa que la probabilidad de medición disminuye a un ritmo aproximadamente logarítmico a medida que aumentan las razones de aproximación, tanto cuando se incrementa el tamaño del problema como cuando se disminuye el valor del hiperparámetro p . Este comportamiento sugiere la existencia de una razón de aproximación a partir de la cual la probabilidad de medición decrece a un ritmo tan pronunciado que se vuelve intolerable. Este hallazgo es complementario a los incrementos exponenciales observados en la figura previa, que refleja una razón de aproximación a partir de la cual se necesita una cantidad exagerada de mediciones para asegurar su aparición.

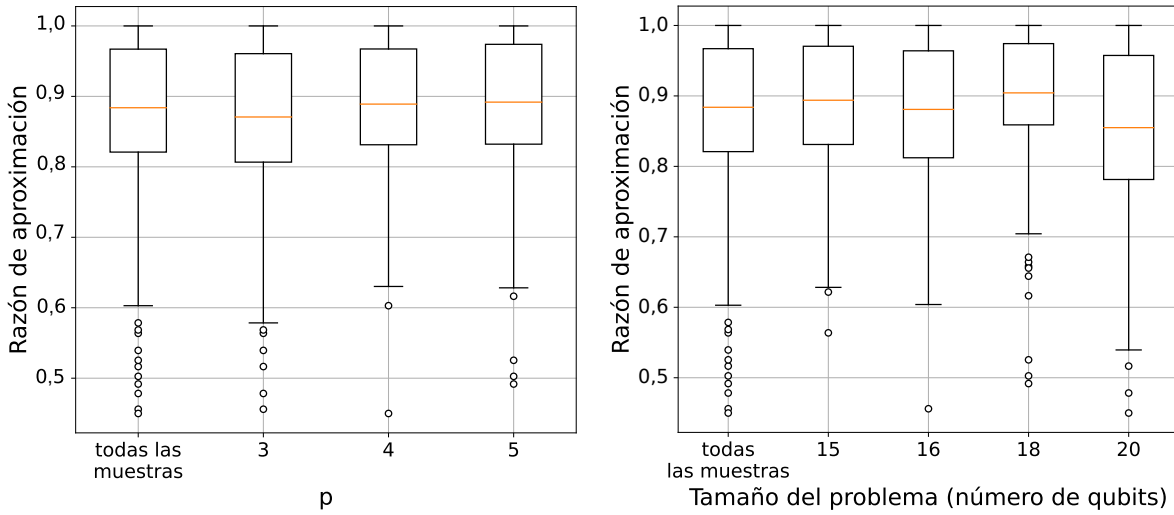


Figura 5.11: Razón de aproximación de las 714 aplicaciones de Aprendizaje por Transferencia.

5.6. Razón de Aproximación de Aprendizaje por Transferencia

En la Figura 5.11 se observa la razón de aproximación de las aplicaciones de Aprendizaje por Transferencia según el tamaño del problema (cantidad de qubits) y el hiperparámetro p . Una de las categorías de cada gráfico representa el muestreo completo, indicando que para esa caja se utilizaron las 714 aplicaciones de Aprendizaje por Transferencia.

La aplicación de Aprendizaje por Transferencia ha demostrado resultados notablemente positivos. A pesar de no haberse implementado una segunda ronda de optimización posterior a la transferencia de parámetros, se observaron razones de aproximación que oscilaron principalmente entre 0,82 y 0,97, con una media cercana a 0,88. Este desempeño sugiere que el enfoque puede considerarse prometedor, ya que las soluciones obtenidas son relativamente cercanas a las reportadas en la Sección 5.1, donde no se aplica Aprendizaje por Transferencia. En consecuencia, se evidencia el **potencial de esta técnica para acelerar procesos de resolución en nuevas instancias del problema**, reduciendo considerablemente el costo computacional asociado al proceso de optimización.

Sin embargo, el análisis detallado de la distribución de las razones de aproximación revela ciertos matices importantes. Si bien el cuartil superior y el valor máximo permanecen similares respecto a los obtenidos sin transferencia, la media experimenta una degradación leve de aproximadamente 0,02 y el cuartil inferior de aproximadamente 0,03. Este impacto limitado sugiere que, en promedio, el enfoque sigue siendo competitivo. No obstante, se observa un deterioro más pronunciado en el valor mínimo, alcanzando este último un valor de 0,6, con la presencia de valores atípicos incluso por debajo de 0,5. Esta dispersión hacia regiones de menor calidad en la solución es coherente con las limitaciones esperables de la técnica: al reutilizar parámetros optimizados para una instancia distinta, sin una segunda optimización, es razonable esperar cierta pérdida de desempeño. Lo relevante de este patrón es que la degradación no implica un desplazamiento uniforme de toda la distribución hacia valores más bajos, sino que **aumenta la variabilidad de los resultados hacia menores razones de aproximación**.

Tal como se esperaba para estos casos, el aumento del hiperparámetro p con una cantidad adecuada de iteraciones del optimizador clásico contribuyó a mejorar la calidad de las soluciones y a reducir la varianza de las razones de aproximación. Sin embargo, este efecto resulta menos marcado que en el escenario en el cual los parámetros han sido optimizados específicamente para la instancia en cuestión.

5.7. Resumen de las Discusiones

Las razones de aproximación obtenidas en este trabajo de tesis muestran un desempeño favorable, con valores que oscilan principalmente entre 0,85 y 0,97, y una media cercana a 0,9. Estos resultados son coherentes con las diferencias Ising que presentan valores negativos alejados de cero, así como con las mejoras de ganancia promedio del 12%. Ésto indica que el algoritmo QAOA está funcionando correctamente tanto a nivel teórico como práctico, alcanzando soluciones aproximadas de calidad en la resolución del Problema de Reasignación de Puestos de Trabajo.

Se observó que el aumento del hiperparámetro p y el incremento en el tamaño del problema siguen una tendencia coherente en todas las métricas analizadas. A medida que se aumenta el valor de p , se mejoran los resultados obtenidos, lo que indica que mayores valores de p contribuyen a una mejor calidad en las soluciones. Por otro lado, el crecimiento del tamaño del problema presenta un desafío adicional, ya que dificulta la búsqueda de soluciones aproximadas, lo que refleja la mayor complejidad asociada a problemas de mayor tamaño.

Una observación relevante es que el tamaño del problema no es el único factor que afecta el rendimiento del algoritmo. Es crucial profundizar más allá del tamaño y analizar el desbalance entre la cantidad de trabajadores y puestos vacantes en cada instancia particular. Instancias de tamaño similar, pero con diferentes distribuciones de estos elementos, pueden tener un comportamiento distinto en las métricas, lo que sugiere que este aspecto también influye en la calidad de las soluciones obtenidas.

La inicialización TQA se destacó como un factor importante en el desempeño del algoritmo. Se hipotetiza que, a medida que aumenta el valor de p , la fortaleza de los parámetros inicializados en TQA mejora, lo que contribuye positivamente a la optimización. Sin embargo, se debe tener en cuenta que, si bien un mayor valor de p mejora la calidad de las soluciones, también existe un *trade-off* en términos de cantidad de iteraciones de optimización, ya que el aumento de p también conlleva un mayor número de parámetros variacionales a optimizar.

Una de las limitaciones más significativas del algoritmo radica en el incremento exponencial en la cantidad de mediciones necesarias para mejorar la calidad de las soluciones durante la evaluación del ansatz postoptimización. Este fenómeno representa un desafío importante para la eficiencia del algoritmo, ya que aumenta significativamente el costo computacional asociado con la búsqueda de soluciones de mayor calidad.

Por último, el uso del Aprendizaje por Transferencia resultó ser una técnica satisfactoria, con resultados que mostraron parámetros estadísticos de la razón de aproximación similares a aquellos obtenidos sin aplicar esta técnica. La razón de aproximación oscila principalmente entre 0,82 y 0,97, con una media cercana a 0,88. Esto sugiere que el Aprendizaje por Transferencia puede ser una herramienta útil para mejorar la eficiencia en la resolución de nuevas instancias del problema, reduciendo el costo computacional asociado con la optimización sin sacrificar considerablemente la calidad de las soluciones obtenidas.

Capítulo 6

Conclusiones

En esta tesis, se diseñó y codificó un conjunto de procedimientos que implementaron el problema JRP haciendo uso de QAOA. En base a una muestra de 105 instancias ejecutadas en el algoritmo, se realizó un análisis cuantitativo sobre el rendimiento algorítmico del método de aproximación estudiado, analizando métricas como la razón de aproximación, la mejora de ganancia y las curvas de mediciones requeridas. Además, se estableció un marco teórico que permite introducir los conceptos básicos de la computación cuántica.

Se logró analizar exitosamente el comportamiento de QAOA para el problema planteado, permitiendo observar tanto sus ventajas como sus limitaciones. El análisis hecho indica razones de aproximación que oscilan principalmente entre 0,85 y 0,97 y con una media cercana a 0,9, junto a una mejora de ganancia promedio del 12%. Ésto indicó un correcto funcionamiento del algoritmo tanto a nivel teórico como en su impacto práctico. Además, se confirmó en reiteradas observaciones como el incremento del hiperparámetro p permite aumentar la calidad de las soluciones obtenidas, mientras que el incremento del tamaño de la instancia del problema la empeora.

Por otro lado, se presentó como una limitación significativa al incremento exponencial en la cantidad de mediciones necesarias para mejorar la calidad de las soluciones durante la evaluación del ansatz postoptimización. Además, se observó un Aprendizaje por Transferencia exitoso al reutilizar parámetros variacionales optimizados entre diferentes instancias del problema, logrando razones de aproximación entre 0,82 y 0,97, con una media cercana a 0,88.

Con todo lo explicado anteriormente, se logró cumplir con el objetivo general de esta tesis, a saber: **realizar las pruebas correspondientes de QAOA para el problema JRP, y en base a los resultados, analizar su rendimiento respecto a la calidad de las soluciones obtenidas y su robustez ante diferentes instancias.**

Particularmente, en la Sección 2.4.3 se especificó la formulación matemática de JRP, mientras que en la Sección 2.3 se presentó QAOA. Esto permitió incorporar al problema y al algoritmo en el diseño del Capítulo 3 e implementación del Capítulo 4, logrando así el objetivo **O1. Implementar JRP en QAOA**. Luego, en el Código B.2, se definieron las configuraciones específicas para los procedimientos diseñados, logrando el objetivo **O2. Determinar un conjunto de hiperparámetros para el algoritmo y problema seleccionados**. A partir del diseño e implementación de los mencionados Capítulo 3 y Capítulo 4, se cumplió el objetivo **O3. Realizar un muestreo de instancias de JRP en QAOA**. Finalmente, en la Capítulo 5, se estudiaron los resultados obtenidos, permitiendo cumplir **O4. Comparar los resultados respecto a las métricas que describen el rendimiento algorítmico, para discutir sobre los mismos**.

La principal limitación de esta tesis es referida a la falta de hardware especializado, es decir, de dispositivos cuánticos reales. Para solventar ésto, se empleó simulación de cómputo cuántico en hardware clásico provisto por el Instituto Tecnológico de Castilla y León. Consecuentemente, el uso plataformas clásicas para emular el comportamiento de sistemas cuánticos implica importantes restricciones computacionales. En particular, el crecimiento exponencial del espacio de

estados en función de la cantidad de qubits impuso un límite práctico al tamaño de las instancias simuladas.

El análisis se restringió a un conjunto de 105 instancias, abarcando entre 15 y 20 qubits. Si bien esta escala permite realizar un estudio preliminar del rendimiento del algoritmo seleccionado, no es suficiente para evaluar con precisión su viabilidad en contextos industriales, donde los problemas tienden a requerir una escala mucho mayor.

A pesar de estas restricciones, los resultados obtenidos permiten vislumbrar el potencial de los algoritmos cuánticos, al tiempo que evidencian los desafíos actuales en cuanto a accesibilidad y escalabilidad en el campo de la computación cuántica.

6.1. Trabajos Futuros

Existen dos direcciones principales que pueden guiar trabajos futuros. La primera consiste en profundizar el análisis de los resultados actuales mediante el uso de técnicas estadísticas y métodos de minería de datos. Este enfoque permitiría extraer información más detallada sobre el desempeño del algoritmo, por ejemplo, la extrapolación de tendencias de rendimiento hacia instancias de mayor tamaño. Esta línea permitiría aprovechar al máximo los datos ya generados sin incurrir en nuevos costos computacionales significativos.

La segunda dirección (ideal, pero actualmente limitada por los costos) apunta a explorar alternativas de ejecución sobre hardware cuántico real. Ejecutar instancias de mayor tamaño en dispositivos cuánticos permitiría no solo evaluar el desempeño del algoritmo en escenarios más realistas, sino también analizar el efecto del ruido inherente a los dispositivos cuánticos actuales. En este contexto, resultaría de particular interés experimentar con técnicas de mitigación y supresión de errores, con el fin de determinar cuáles son las más efectivas para el tipo de problema abordado. Ésto podría aportar una perspectiva valiosa sobre la robustez de las soluciones cuánticas frente a las limitaciones del hardware real.

Adicionalmente, se pueden combinar nuevas estrategias con QAOA para resolver el problema JRP. Por ejemplo, usando un enfoque de segmentación del problema basado en divide-y-vencerás, la cual descompone el problema original en subinstancias más pequeñas, facilitando así su resolución y reduciendo significativamente el espacio de búsqueda [20].

Otra línea prometedora consiste en experimentar con variantes del algoritmo QAOA clásico. Entre estas se encuentran QAOA Recursivo, que ha mostrado mejoras sustanciales respecto al QAOA tradicional en la resolución del problema del corte máximo [83], y GM-QAOA, una variante que incorpora operadores de mezcla inspirados en el algoritmo de Grover [84]. La implementación de estas variantes requerirá investigar herramientas y bibliotecas adicionales, ya que actualmente OpenQAOA ofrece únicamente soporte funcional para la versión clásica del algoritmo.

Finalmente, cabe recordar el interrogante presentado en la Sección 5.5 respecto al comportamiento de las curvas al agruparlas por tamaño de problema. Si bien se observa un traslado en las curvas, éste no parece seguir una tendencia lineal clara. Dado que solo se analizaron cuatro tamaños de problema distintos, no es posible identificar con certeza la forma exacta de este desplazamiento. Un enfoque posible para futuras investigaciones sería extender el análisis a instancias de mayor tamaño, con el objetivo de determinar si existe una tendencia sistemática en el corrimiento de la curva a medida que el tamaño del problema aumenta.

6.2. Publicaciones

Durante el desarrollo de esta tesis, se realizó la siguiente publicación en base a la misma:

- “*Benchmarking QAOA on the Job Reassignment Problem: An Empirical Analysis*” Adriano Lusso, Christian Nelson Gimenez y Alejandro Mata Ali.

Jornadas Argentinas de Informática e Investigación Operativa (JAIIO 54) - Argentinean Symposium on Quantum Computing (ASQC). Buenos Aires, Agosto del 2025.

Además, de la publicación anterior, se obtuvo el premio a "**Mejor Trabajo del Simposio**", del que surge la siguiente publicación extendida:

- *“Benchmarking QAOA on the Job Reassignment Problem: An Empirical Analysis using Transfer Learning and TQA Initialisation”* Adriano Lusso, Christian Nelson Gimenez y Alejandro Mata Ali.
Electronic Journal of SADIO. (En prensa al día de la fecha).

Referencias Bibliográficas

- [1] Gregg Jaeger. «Classical and quantum computing». En: *Quantum Information: An Overview* (2007), págs. 203-217 (vid. [pág. 1](#)).
- [2] Fernando Peres y Mauro Castelli. «Combinatorial optimization problems and metaheuristics: Review, challenges, design, and development». En: *Applied sciences* 11.14 (2021), [pág. 6449](#) (vid. [pág. 1](#)).
- [3] M.A. Nielsen e I.L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010 (vid. págs. [1](#), [5-7](#), [9-12](#)).
- [4] Mark Moore y Ajit Narayanan. «Quantum-inspired computing». En: *Dept. Comput. Sci., Univ. Exeter, Exeter, UK* (1995) (vid. [pág. 1](#)).
- [5] Gennaro De Luca. «A survey of NISQ era hybrid quantum-classical machine learning research». En: *Journal of Artificial Intelligence and Technology* 2.1 (2022), págs. 9-15 (vid. [pág. 1](#)).
- [6] Frank Gaitan. *Quantum error correction and fault tolerant quantum computing*. CRC Press, 2008 (vid. [pág. 1](#)).
- [7] T. Radtke y Stephan Fritzsche. «Simulation of n-qubit quantum systems. I. Quantum registers and quantum gates». En: *Computer Physics Communications* 173.1-2 (2005), págs. 91-113 (vid. págs. [1](#), [14](#)).
- [8] Juneseo Lee, Alicia B. Magann, Herschel A. Rabitz y Christian Arenz. «Progress toward favorable landscapes in quantum combinatorial optimization». En: *Physical Review A* 104.3 (2021), [pág. 032401](#) (vid. [pág. 1](#)).
- [9] Maxime Dupont, Bram Evert, Mark J. Hodson, Bhuvanesh Sundar, Stephen Jeffrey, Yuki Yamaguchi, Dennis Feng, Filip B. Maciejewski, Stuart Hadfield, M. Sohaib Alam et al. «Quantum-enhanced greedy combinatorial optimization solver». En: *Science Advances* 9.45 (2023) (vid. [pág. 1](#)).
- [10] Edward Farhi, Jeffrey Goldstone y Sam Gutmann. «A quantum approximate optimization algorithm». En: *arXiv preprint arXiv:1411.4028* (2014) (vid. págs. [2](#), [17](#), [18](#)).
- [11] Satoshi Morita e Hidetoshi Nishimori. «Mathematical foundation of quantum annealing». En: *Journal of Mathematical Physics* 49.12 (2008) (vid. págs. [2](#), [19](#)).
- [12] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik y Jeremy L. O'Brien. «A variational eigenvalue solver on a photonic quantum processor». En: *Nature communications* 5.1 (2014), [pág. 4213](#) (vid. [pág. 2](#)).
- [13] Mari Carmen Bañuls. «Tensor network algorithms: A route map». En: *Annual Review of Condensed Matter Physics* 14.1 (2023), págs. 173-191 (vid. [pág. 2](#)).
- [14] Román Orús. «A practical introduction to tensor networks: Matrix product states and projected entangled pair states». En: *Annals of physics* 349 (2014), págs. 117-158 (vid. [pág. 2](#)).

- [15] Walid Ben-Ameur, Ali Ridha Mahjoub y José Neto. «The maximum cut problem». En: *Paradigms of Combinatorial Optimization: Problems and New Approaches* (2014), págs. 131-172 (vid. pág. 2).
- [16] Harvey M. Salkin y Cornelis A. De Kluyver. «The knapsack problem: a survey». En: *Naval Research Logistics Quarterly* 22.1 (1975), págs. 127-144 (vid. pág. 2).
- [17] Kimleang Kea, Chansreynich Huot y Youngsun Han. «Leveraging Knapsack QAOA Approach for Optimal Electric Vehicle Charging». En: *IEEE Access* (2023) (vid. pág. 2).
- [18] Abhishek Awasthi, Francesco Bär, Joseph Doetsch, Hans Ehm, Marvin Erdmann, Maximilian Hess, Johannes Klepsch, Peter A. Limacher, Andre Luckow, Christoph Niedermeier et al. «Quantum computing techniques for multi-knapsack problems». En: *Science and Information Conference*. Springer. 2023, págs. 264-284 (vid. pág. 2).
- [19] Raphael Pfister, Gunnar Schubert y Markus Kröll. «Transfer of Logistics Optimizations to Material Flow Resource Optimizations using Quantum Computing». En: *Procedia Computer Science* 232 (2024), págs. 32-42 (vid. pág. 2).
- [20] Iñigo Perez Delgado, Beatriz García Markaida, Alejandro Mata Ali y Aitor Moreno Fdez de Leceta. «Qubo resolution of the job reassignment problem». En: *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*. IEEE. 2023, págs. 4847-4852 (vid. págs. 2, 19, 20, 48).
- [21] Pedro Meseguer, Francesca Rossi y Thomas Schiex. «Soft constraints». En: *Foundations of Artificial Intelligence*. Vol. 2. Elsevier, 2006, págs. 281-328 (vid. págs. 2, 21).
- [22] Ali Hmer y Malek Mouhoub. «Teaching assignment problem solver». En: *International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems*. Springer. 2010, págs. 298-307 (vid. págs. 2, 21).
- [23] David Cohen, Martin Cooper, Peter Jeavons y Andrei Krokhin. «A maximal tractable class of soft constraints». En: *Journal of Artificial Intelligence Research* 22.1 (2004), págs. 1-22 (vid. págs. 2, 21).
- [24] Stuart Hadfield, Zhihui Wang, Eleanor G Rieffel, Bryan O’Gorman, Davide Venturelli y Rupak Biswas. «Quantum approximate optimization with hard and soft constraints». En: *Proceedings of the Second International Workshop on Post Moores Era Supercomputing*. 2017, págs. 15-21 (vid. pág. 2).
- [25] Paul Adrien Maurice Dirac. «A new notation for quantum mechanics». En: *Mathematical proceedings of the Cambridge philosophical society*. Vol. 35. 3. Cambridge University Press. 1939, págs. 416-418 (vid. pág. 5).
- [26] Fortunato Tito Arecchi, Eric Courtens, Robert Gilmore y Harry Thomas. «Atomic coherent states in quantum optics». En: *Physical Review A* 6.6 (1972), pág. 2211 (vid. pág. 6).
- [27] Dan J. Shepherd. «On the role of Hadamard gates in quantum circuits». En: *Quantum Information Processing* 5 (2006), págs. 161-177 (vid. pág. 8).
- [28] Eleanor G. Rieffel y Wolfgang H. Polak. *Quantum computing: A gentle introduction*. MIT press, 2011 (vid. pág. 10).
- [29] Amira Abbas, Andris Ambainis, Brandon Augustino, Andreas Bärtshi, Harry Buhrman, Carleton Coffrin, Giorgio Cortiana, Vedran Dunjko, Daniel J. Egger, Bruce G. Elmegreen et al. «Challenges and opportunities in quantum optimization». En: *Nature Reviews Physics* (2024), págs. 1-18 (vid. pág. 13).
- [30] L. Wasserman. *All of statistics: A concise course in statistical inference*. 2013 (vid. pág. 13).
- [31] He-Liang Huang, Dachao Wu, Daojin Fan y Xiaobo Zhu. «Superconducting quantum computing: a review». En: *Science China Information Sciences* 63.8 (2020), pág. 180501 (vid. pág. 14).

- [32] Jan Benhelm, Gerhard Kirchmair, Christian F Roos y Rainer Blatt. «Towards fault-tolerant quantum computing with trapped ions». En: *Nature Physics* 4.6 (2008), págs. 463-466 (vid. pág. 14).
- [33] David S Weiss y Mark Saffman. «Quantum computing with neutral atoms». En: *Physics Today* 70.7 (2017), págs. 44-50 (vid. pág. 14).
- [34] Catherine C McGeoch. «Theory versus practice in annealing-based quantum computing». En: *Theoretical Computer Science* 816 (2020), págs. 169-183 (vid. pág. 14).
- [35] Davide Castelvecchi. «IBM releases first-ever 1,000-qubit quantum chip». En: *Nature* 624.7991 (2023), págs. 238-238 (vid. pág. 14).
- [36] Yirong Jin. «Google's "Willow" quantum processor: New RCS record and first error correction below the surface code threshold». En: *The Innovation* (2025) (vid. pág. 14).
- [37] and others. «Computational Power of Random Quantum Circuits in Arbitrary Geometries». En: *Physical Review X* 15.2 (2025), pág. 021052 (vid. pág. 14).
- [38] *IonQ Forte: The First Software-Configurable Quantum Computer*. <https://ionq.com/resources/ionq-forte-first-configurable-quantum-computer>. Accedido: 28-09-2025. 2024 (vid. pág. 14).
- [39] Jonathan Wurtz, Alexei Bylinskii, Boris Braverman, Jesse Amato-Grill, Sergio H Cantu, Florian Huber, Alexander Lukin, Fangli Liu, Phillip Weinberg, John Long et al. «Aquila: QuEra's 256-qubit neutral-atom quantum computer». En: *arXiv preprint arXiv:2306.11727* (2023) (vid. pág. 14).
- [40] Dennis Willsch, Madita Willsch, Carlos D Gonzalez Calaza, Fengping Jin, Hans De Raedt, Marika Svensson y Kristel Michielsen. «Benchmarking Advantage and D-Wave 2000Q quantum annealers with exact cover problems». En: *arXiv preprint arXiv:2105.02208* (2021) (vid. pág. 14).
- [41] Salonik Resch y Ulya R Karpuzcu. «Benchmarking quantum computers and the impact of quantum noise». En: *ACM Computing Surveys (CSUR)* 54.7 (2021), págs. 1-35 (vid. pág. 14).
- [42] Benedikt Fauseweh. «Quantum many-body simulations on digital quantum computers: State-of-the-art and future challenges». En: *Nature Communications* 15.1 (2024), pág. 2123 (vid. pág. 14).
- [43] Maram Assi y Ramzi A. Haraty. «A survey of the knapsack problem». En: *2018 International Arab Conference on Information Technology (ACIT)*. IEEE. 2018, págs. 1-6 (vid. pág. 15).
- [44] Sunita Kumawat, Chanchal Dudeja y Pawan Kumar. «An extensive review of shortest path problem solving algorithms». En: *2021 5th International Conference on Intelligent Computing and Control Systems (ICICCS)*. IEEE. 2021, págs. 176-184 (vid. pág. 15).
- [45] Jaeho Choi y Joongheon Kim. «A Tutorial on Quantum Approximate Optimization Algorithm (QAOA): Fundamentals and Applications». En: *2019 International Conference on Information and Communication Technology Convergence (ICTC)*. 2019, págs. 138-142. DOI: [10.1109/ICTC46691.2019.8939749](https://doi.org/10.1109/ICTC46691.2019.8939749) (vid. págs. 15, 27).
- [46] Fred Glover, Gary Kochenberger y Yu Du. «A tutorial on formulating and using QUBO models». En: *arXiv preprint arXiv:1811.11538* (2018) (vid. pág. 15).
- [47] Gary Kochenberger, Jin-Kao Hao, Fred Glover, Mark Lewis, Zhipeng Lü, Haibo Wang y Yang Wang. «The unconstrained binary quadratic programming problem: a survey». En: *Journal of combinatorial optimization* 28 (2014), págs. 58-81 (vid. pág. 15).

- [48] Mashiyat Zaman, Kotaro Tanahashi y Shu Tanaka. «PyQUBO: Python library for mapping combinatorial optimization problems to QUBO form». En: *IEEE Transactions on Computers* 71.4 (2021), págs. 838-850 (vid. pág. 15).
- [49] Ernst Ising. «Contribution to the theory of ferromagnetism». En: *Z. Phys* 31.1 (1925), págs. 253-258 (vid. pág. 16).
- [50] Andrew Lucas. «Ising formulations of many NP problems». En: *Frontiers in physics* 2 (2014), pág. 74887 (vid. pág. 16).
- [51] Kostas Blekos, Dean Brand, Andrea Ceschini, Chiao-Hui Chou, Rui-Hao Li, Komal Pandya y Alessandro Summer. «A review on quantum approximate optimization algorithm and its variants». En: *Physics Reports* 1068 (2024), págs. 1-66 (vid. págs. 17, 18).
- [52] Jaeho Choi y Joongheon Kim. «A tutorial on quantum approximate optimization algorithm (QAOA): Fundamentals and applications». En: *2019 international conference on information and communication technology convergence (ICTC)*. IEEE. 2019, págs. 138-142 (vid. págs. 17, 18).
- [53] Michael JD Powell. *A direct search optimization method that models the objective and constraint functions by linear interpolation*. Springer, 1994 (vid. pág. 18).
- [54] John A Nelder y Roger Mead. «A simplex method for function minimization». En: *The computer journal* 7.4 (1965), págs. 308-313 (vid. pág. 18).
- [55] Diederik P. Kingma y Jimmy Ba. «Adam: A method for stochastic optimization». En: *arXiv preprint arXiv:1412.6980* (2014) (vid. pág. 18).
- [56] Roger Fletcher. «A new approach to variable metric algorithms». En: *The computer journal* 13.3 (1970), págs. 317-322 (vid. pág. 18).
- [57] Michael J.D. Powell. «An efficient method for finding the minimum of a function of several variables without calculating derivatives». En: *The computer journal* 7.2 (1964), págs. 155-162 (vid. pág. 18).
- [58] Aidan Pellow-Jarman, Shane McFarthing, Ilya Sinayskiy, Daniel K. Park, Anban Pillay y Francesco Petruccione. «The effect of classical optimizers and Ansatz depth on QAOA performance in noisy devices». En: *Scientific reports* 14.1 (2024), pág. 16011 (vid. pág. 18).
- [59] Stefan H. Sack y Maksym Serbyn. «Quantum annealing initialization of the quantum approximate optimization algorithm». En: *quantum* 5 (2021), pág. 491 (vid. págs. 19, 39).
- [60] J.A. Montañez-Barrera, Dennis Willsch y Kristel Michielsen. «Transfer learning of optimal QAOA parameters in combinatorial optimization». En: *Quantum Information Processing* 24.5 (2025), pág. 129 (vid. pág. 19).
- [61] Fernando G.S.L. Brandao, Michael Broughton, Edward Farhi, Sam Gutmann y Hartmut Neven. «For fixed control parameters the quantum approximate optimization algorithm's objective function value concentrates for typical instances». En: *arXiv preprint arXiv:1812.04170* (2018) (vid. pág. 19).
- [62] Alexey Galda, Xiaoyuan Liu, Danylo Lykov, Yuri Alexeev e Ilya Safro. «Transferability of optimal QAOA parameters between random graphs». En: *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE. 2021, págs. 171-180 (vid. pág. 19).
- [63] Ruslan Shaydulin, Phillip C. Lotshaw, Jeffrey Larson, James Ostrowski y Travis S. Humble. «Parameter transfer for quantum approximate optimization of weighted maxcut». En: *ACM Transactions on Quantum Computing* 4.3 (2023), págs. 1-15 (vid. pág. 19).
- [64] Francesco Aldo Venturelli, Sreetama Das y Filippo Caruso. «Investigating layer-selective transfer learning of QAOA parameters for Max-Cut problem». En: *arXiv preprint arXiv:2412.21071* (2024) (vid. pág. 19).

- [65] David W Pentico. «Assignment problems: A golden anniversary survey». En: *European Journal of Operational Research* 176.2 (2007), págs. 774-793 (vid. pág. 21).
- [66] Harold W Kuhn. «The Hungarian method for the assignment problem». En: *Naval research logistics quarterly* 2.1-2 (1955), págs. 83-97 (vid. pág. 21).
- [67] Christos H Papadimitriou y Kenneth Steiglitz. *Combinatorial optimization: algorithms and complexity*. Courier Corporation, 1998 (vid. pág. 21).
- [68] Amira Abbas, Andris Ambainis, Brandon Augustino, Andreas Bärtzchi, Harry Buhrman, Carleton Coffrin, Giorgio Cortiana, Vedran Dunjko, Daniel J. Egger, Bruce G. Elmegreen et al. «Challenges and opportunities in quantum optimization». En: *Nature Reviews Physics* 6.12 (oct. de 2024), págs. 718-735. ISSN: 2522-5820. DOI: [10.1038/s42254-024-00770-9](https://doi.org/10.1038/s42254-024-00770-9). URL: <http://dx.doi.org/10.1038/s42254-024-00770-9> (vid. pág. 21).
- [69] M. J. D. Powell. «An efficient method for finding the minimum of a function of several variables without calculating derivatives». En: *The Computer Journal* 7.2 (ene. de 1964), págs. 155-162. ISSN: 0010-4620. DOI: [10.1093/comjnl/7.2.155](https://doi.org/10.1093/comjnl/7.2.155) (vid. pág. 26).
- [70] Ernesto Campos, Daniil Rabinovich, Vishwanathan Akshay y J Biamonte. «Training saturation in layerwise quantum approximate optimization». En: *Physical Review A* 104.3 (2021), pág. L030401 (vid. pág. 26).
- [71] Murphy Yuezhen Niu, Sirui Lu e Isaac L Chuang. «Optimizing qaoa: Success probability and runtime dependence on circuit depth». En: *arXiv preprint arXiv:1905.12134* (2019) (vid. pág. 27).
- [72] JA Montanez-Barrera, Kristel Michielsen y David E Bernal Neira. «Evaluating the performance of quantum process units at large width and depth». En: *arXiv preprint arXiv:2502.06471* (2025) (vid. pág. 27).
- [73] *Anaconda Software Distribution*. Ver. 2-2.4.0. 2020. URL: <https://docs.anaconda.com/> (vid. pág. 31).
- [74] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay et al. «Jupyter Notebooks—a publishing format for reproducible computational workflows». En: *Positioning and power in academic publishing: Players, agents and agendas*. IOS press, 2016, págs. 87-90 (vid. pág. 31).
- [75] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith et al. «Array programming with NumPy». En: *Nature* 585.7825 (sep. de 2020), págs. 357-362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2) (vid. pág. 31).
- [76] Wes McKinney et al. «Data structures for statistical computing in python». En: *Proceedings of the 9th Python in Science Conference*. Vol. 445. Austin, TX. 2010, págs. 51-56 (vid. pág. 31).
- [77] J. D. Hunter. «Matplotlib: A 2D graphics environment». En: *Computing in Science & Engineering* 9.3 (2007), págs. 90-95. DOI: [10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55) (vid. pág. 31).
- [78] Michael L. Waskom. «seaborn: statistical data visualization». En: *Journal of Open Source Software* 6.60 (2021), pág. 3021. DOI: [10.21105/joss.03021](https://doi.org/10.21105/joss.03021) (vid. pág. 31).
- [79] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross et al. *Quantum computing with Qiskit*. 2024. DOI: [10.48550/arXiv.2405.08810](https://doi.org/10.48550/arXiv.2405.08810) (vid. pág. 32).
- [80] Vishal Sharma, Nur Shahidee Bin Saharan, Shao-Hen Chiew, Ezequiel Ignacio Rodríguez Chiacchio, Leonardo Disilvestro, Tommaso Federico Demarie y Ewan Munro. «OpenQAOA—An SDK for QAOA». En: *arXiv preprint arXiv:2210.08695* (2022) (vid. pág. 32).

- [81] Raphael Seidel, Sebastian Bock, René Zander, Matic Petrič, Niklas Steinmann, Nikolay Tcholtchev y Manfred Hauswirth. «Qrisp: A framework for compilable high-level programming of gate-based quantum computers». En: *arXiv preprint arXiv:2406.14792* (2024) (vid. pág. 32).
- [82] «Exploratory Data Analysis». En: *The Concise Encyclopedia of Statistics*. New York, NY: Springer New York, 2008, págs. 192-194. ISBN: 978-0-387-32833-1. DOI: [10.1007/978-0-387-32833-1_136](https://doi.org/10.1007/978-0-387-32833-1_136). URL: https://doi.org/10.1007/978-0-387-32833-1_136 (vid. pág. 35).
- [83] Eunok Bae y Soojoon Lee. «Recursive QAOA outperforms the original QAOA for the MAX-CUT problem on complete graphs». En: *Quantum Information Processing* 23.3 (2024), pág. 78 (vid. pág. 48).
- [84] Andreas Bartschi y Stephan Eidenbenz. «Grover mixers for QAOA: Shifting complexity from mixer design to state preparation». En: *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE. 2020, págs. 72-82 (vid. pág. 48).

Apéndice A

Estructuras de datos de la implementación

```

1  { "method": "powell" ,
2    "cost_hamiltonian": { "n_qubits": 15,
3                          "terms": [
4                            {
5                              "qubit_indices": [0, 1],
6                              "pauli_str": "ZZ",
7                              "phase": 1
8                            },
9                            . . . ]
10                   },
11    "evals":{ "number_of_evals":254,
12             "jac_evals": 0,
13             "qfim_evals": 0
14           } ,
15    "most_probable_states": { "solutions_bitstrings": ["000000001000010"],
16                           "bitstring_energy": -7.105427357601002e-15
17                           },
18    "optimized":{
19      "angles": [0.531762424122, . . . ],
20      "cost": -3.392883 ,
21      "measurement_outcomes": {"0100111001":183, . . . },
22      "job_id": "7dca70d6-1020-4c29-b714-65bc2e243bbc" ,
23      "eval_number": 187 ,
24    },
25    "cost_history": [ 7.099445, . . . ],
26  }

```

Código A.1: Ejemplo de diccionario de resultados generado luego de utilizar OpenQAOA.

```

1  {
2    "jrp_configuration": {
3      "num_agents": 5,
4      "num_vacnJobs": 3,
5      "penalty1": 2.5,
6      "penalty2": 3,
7      "control_restrictions": false
8    },
9    "circuit_configuration": {
10     "p": 3,
11     "param_type": "standard",
12     "init_type": "ramp",
13     "mixer_hamiltonian": "x"
14   },
15   "optimizer_configuration": {
16     "method": "Powell",
17     "maxfev": 10000,
18     "tol": 0.01,
19     "optimization_progress": true,
20     "cost_progress": true,
21     "parameter_log": true
22   },
23   "optimization_backend_configuration": {
24     "n_shots": 10000
25   },
26   "evaluation_backend_configuration": {
27     "n_shots": 20
28   },
29   "0":{
30     "instance":{ ... },
31     "approximation_ratio": 0.9576008273009305,
32     "standard_gain_difference": 9.26,
33     "ising_cost_difference": -10.70274,
34     "opt_standard_solution": [-1,-1,2,1,0],
35     "final_standard_solution": [1,-1,-1,0,2],
36     "result": { . . . }
37   },
38   . . .
39   "4": { . . . }
40 }

```

Código A.2: Ejemplo de archivo JSON que almacena una submuestra de cinco ejecuciones QAOA implementado JRP.

```
1  {
2    "reused_instance": {
3      "filename": "conf_jrpInitConf0_circuitConf0",
4      "instance": 0
5    },
6    "<nombre de archivo de submuestra con mismo hiperparametro p>": {
7      "0": {
8        "approximation_ratio": 0.9670818505338078,
9        "opt_standard_solution": [2,-1,0, 1,-1],
10       "final_standard_solution": [1,-1,-1,0,2]
11      },
12       . . . ,
13       "4": { . . . }
14     },
15     . . .
16   }
```

Código A.3: Ejemplo de archivo JSON que almacena una ejecución de aplicación de Aprendizaje por Transferencia.

Apéndice B

Métodos y funciones de la implementación

```

1  def __run_qaoa_workflow(self, jrp, circuit_configuration,
2  optimizer_configuration, optimization_backend_configuration,
3  evaluation_backend_configuration, device):
4  . . .
5  # create and configure the qaoa for optimization
6  qaoa = super().create_and_configure_qaoa(device, circuit_configuration,
7  optimization_backend_configuration,
8  jrp_ising,
9  optimizer_configuration)
10
11  # compile
12  qaoa.compile(jrp_ising)
13
14  # get the initial ising cost, using the initial variational parameters
15  initial_ising_cost = qaoa.evaluate_circuit(
16  qaoa.variate_params.asdict()
17  )['cost']
18
19  # do the QAOA optimization and get result
20  qaoa.optimize()
21  qaoa_result = qaoa.result
22  . . .
23
24
25  def create_and_configure_qaoa(self, device, circuit_configuration,
26  backend_configuration, ising, optimizer_configuration = None):
27  qaoa = QAOA()
28  qaoa.set_device(device)
29  qaoa.set_circuit_properties(**circuit_configuration)
30  qaoa.set_backend_properties(**backend_configuration)
31  if optimizer_configuration is not None:
32  qaoa.set_classical_optimizer(**optimizer_configuration)
33  qaoa.compile(ising)
34
35  return qaoa

```

Código B.1: Implementación del Solucionador QAOA. El método `create_and_configure_qaoa` se encarga de crear y configurar el algoritmo para su uso, mientras que `__run_qaoa_workflow` utiliza la instancia del algoritmo creado para optimizarlo y obtener sus resultados.

```

1  # all the JRP initialization configurations for the JRPRandomGenerator
2  jrp_init_configurations = [
3  {'num_agents':5 , 'num_vacnJobs':3 , 'penalty1':2.5, 'penalty2':3,
4   'control_restrictions':False},
5  {'num_agents':3 , 'num_vacnJobs':5 , 'penalty1':2.5, 'penalty2':3,
6   'control_restrictions':False},
7  {'num_agents':6 , 'num_vacnJobs':3 , 'penalty1':2.5, 'penalty2':3,
8   'control_restrictions':False},
9  {'num_agents':3, 'num_vacnJobs':6, 'penalty1':2.5, 'penalty2':3,
10   'control_restrictions':False},
11  {'num_agents':5, 'num_vacnJobs':4, 'penalty1':2.5, 'penalty2':3,
12   'control_restrictions':False},
13  {'num_agents':4, 'num_vacnJobs':5, 'penalty1':2.5, 'penalty2':3,
14   'control_restrictions':False},
15  {'num_agents':4, 'num_vacnJobs':4, 'penalty1':2.5, 'penalty2':3,
16   'control_restrictions':False}
17  ]

```

Código B.2: Configuraciones para inicializar instancia JRP implementadas en Python.

```

1  circuit_configurations =[
2  {
3      'p': 3,
4      'param_type': 'standard',
5      'init_type': 'ramp',
6      'mixer_hamiltonian': 'x'
7  },
8  {
9      'p': 4,
10     'param_type': 'standard',
11     'init_type': 'ramp',
12     'mixer_hamiltonian': 'x'
13 },
14 {
15     'p': 5,
16     'param_type': 'standard',
17     'init_type': 'ramp',
18     'mixer_hamiltonian': 'x'
19 }
20 ]
21
22 optimization_backend_configuration ={
23     'n_shots': 10000
24 }
25
26 evaluation_backend_configuration ={
27     'n_shots': 20
28 }
29
30 optimizer_configuration={
31     'method' : 'Powell',
32     'maxfev': 10000,
33     'tol': 0.01,
34     'optimization_progress': True,
35     'cost_progress': True,
36     'parameter_log': True
37 }

```

Código B.3: Hiperparámetros para QAOA implementados en Python.

```

1  n_instances_per_jrpInitConf = 5
2  solver = TestQAQASolver()
3
4  for itr,jrp_conf in enumerate(jrp_init_configurations):
5      solver.set_fixedInstances(jrp_conf,n_instances_per_jrpInitConf)
6      for itr2,circuit_conf in enumerate(circuit_configurations):
7          print('JRP CONF ',itr,' - CIRCUIT CONF ',itr2)
8
9      name = '_jrpInitConf'+str(itr)+'_circuitConf'+str(itr2)
10     solver.sample_workflows_with_fixedInstances(
11         str(name),
12         circuit_conf,
13         optimizer_configuration,
14         optimization_backend_configuration,
15         evaluation_backend_configuration
16     )
17     file_name = 'conf'+name+'.json'
18     !cp {file_name} /content/drive/MyDrive/results_experiment_colab
19
20     clear_output(wait=True)

```

Código B.4: Estructura iterativa para ejecutar el flujo de trabajo principal con cada configuración de entrada.

```

1  def sample_workflows_with_fixedInstances(self, configuration_name, circuit_configuration,
2  optimizer_configuration, optimization_backend_configuration,
3  evaluation_backend_configuration, device=None):
4
5  # evaluate if the fixed instances has been previously created
6  if self.fixedInstances is None:
7  raise ValueError('There fixed instances were not created.
8  Create them using set_fixedInstances() method.')
9
10 # if necessary, create the device
11 if device is None:
12 device = create_device(location='local', name='qiskit.shot_simulator')
13
14 # starts the sampling
15 samples = {
16     'jrp_configuration': self.jrp_init_configuration,
17     'circuit_configuration': circuit_configuration,
18     'optimizer_configuration': optimizer_configuration,
19     'optimization_backend_configuration': optimization_backend_configuration,
20     'evaluation_backend_configuration': evaluation_backend_configuration
21 }
22 for sample_index, jrp_instance_dict in enumerate(self.fixedInstances):
23 #create the object for the instance dictionary
24 jrp = JRPCClassic(jrp_instance_dict)
25
26 # get the important results
27 print('running sample ', sample_index)
28 approximation_ratio, standard_gain_difference, ising_cost_difference,
29 opt_standard_solution, final_standard_solution, qaoa_result = self.__run_workflow(
30 jrp, circuit_configuration,
31 optimizer_configuration,
32 optimization_backend_configuration,
33 evaluation_backend_configuration,
34 device
35 )
36
37 # creates the sample dats structure and saves it
38 postprocessed_qaoa_result = self.postprocess_qaoaresult(qaoa_result.asdict())
39 sample = {
40     'instance': jrp_instance_dict,
41     'approximation_ratio': approximation_ratio,
42     'standard_gain_difference': standard_gain_difference,
43     'ising_cost_difference': ising_cost_difference,
44     'opt_standard_solution': opt_standard_solution,
45     'final_standard_solution': final_standard_solution,
46     'result': postprocessed_qaoa_result
47 }
48 samples[sample_index] = sample
49
50 # save the json and compress it
51 with open('./conf%s.json'%(str(configuration_name)), 'w', encoding='utf-8') as file:
52 json.dump(samples, file, ensure_ascii=False, indent=4)

```

Código B.5: Implementación del método `sample_workflows_with_fixedInstances`.

```

1  def __run_workflow(self,jrp,circuit_configuration, optimizer_configuration,
2  optimization_backend_configuration,
3  evaluation_backend_configuration,device):
4
5  # initial configuration
6  initial_standard_solution = [-1 for i in range(jrp.instance_dict['num_agents'])]
7  initial_standard_gain = jrp.calculate_standard_gain(initial_standard_solution)
8
9  #optimal configuration
10 opt_standard_solution,opt_standard_gain = jrp.solve_standard_with_bruteforce(debug_every=0)
11
12 # final configuration - after QAQA optimization
13 final_standard_solution,final_standard_gain,initial_ising_cost,final_ising_cost,qaoa_result
    = self.__run_qaoa_workflow(
14     jrp,
15     circuit_configuration,
16     optimizer_configuration,
17     optimization_backend_configuration,
18     evaluation_backend_configuration,device
19 )
20
21 if opt_standard_gain == 0:
22     approximation_ratio = None
23 else:
24     approximation_ratio = final_standard_gain / opt_standard_gain
25     standard_gain_difference = final_standard_gain - initial_standard_gain
26     ising_cost_difference = final_ising_cost - initial_ising_cost
27
28 return approximation_ratio,standard_gain_difference,ising_cost_difference,
    opt_standard_solution,final_standard_solution,qaoa_result

```

Código B.6: Implementación del método `__run_workflow`.

```

1  solver = EvaluationQAQASolver()
2
3  solver.sample_workflows(files, 'conf_jrpInitConf0_circuitConf0', 0)
4  solver.sample_workflows(files, 'conf_jrpInitConf1_circuitConf0', 0)
5  solver.sample_workflows(files, 'conf_jrpInitConf2_circuitConf0', 0)
6  solver.sample_workflows(files, 'conf_jrpInitConf3_circuitConf0', 0)
7  solver.sample_workflows(files, 'conf_jrpInitConf4_circuitConf0', 0)
8  solver.sample_workflows(files, 'conf_jrpInitConf5_circuitConf0', 0)
9  solver.sample_workflows(files, 'conf_jrpInitConf6_circuitConf0', 0)
10
11
12 solver.sample_workflows(files, 'conf_jrpInitConf0_circuitConf1', 0)
13 solver.sample_workflows(files, 'conf_jrpInitConf1_circuitConf1', 0)
14 . . .
15
16 solver.sample_workflows(files, 'conf_jrpInitConf0_circuitConf2', 0)
17 solver.sample_workflows(files, 'conf_jrpInitConf1_circuitConf2', 0)
18 . . .

```

Código B.7: Instanciación de `EvaluationQAQASolver` para el postprocesamiento.

```

1  class EvaluationQAQASolver(QAQASolver):
2  def __init__(self):
3  pass
4
5  def sample_workflows(self, files, filename, instance):
6  optimized_params = files[filename][str(instance)]['result']['optimized']['angles']
7  keys = files.keys()
8  p = files[filename]['circuit_configuration']['p']
9  samples = {
10     'reused_instance':{
11         'filename': filename,
12         'instance': instance
13     }
14 }
15 for key in keys:
16     circuit_configuration = files[key]["circuit_configuration"]
17     aux_p = circuit_configuration['p']
18     if p != aux_p:
19         continue
20     samples[key] = {}
21
22     evaluation_backend_configuration = files[key]["evaluation_backend_configuration"]
23     for ins in range(5):
24         if filename in key and str(instance) in str(ins):
25             continue
26         instance_dict = files[key][str(ins)]["instance"]
27
28         jrp = JRPCClassic(instance_dict)
29         opt_standard_solution = files[key][str(ins)]["opt_standard_solution"]
30         opt_standard_gain = jrp.calculate_standard_gain(files[key][str(ins)]["opt_standard_solution"]
31         ])
32         jrp_ising = super().get_jrp_ising(jrp)
33
34         device = create_device(location='local', name='qiskit.shot_simulator')
35         qaoa = super().create_and_configure_qaoa(device, circuit_configuration,
36         evaluation_backend_configuration,
37         jrp_ising)
38         qaoa.compile(jrp_ising)
39
40         preliminary_qubo_solutions = qaoa.evaluate_circuit(optimized_params)['measurement_results']
41         final_standard_solution, final_standard_gain = super().filter_qubo_solutions(jrp,
42         preliminary_qubo_solutions)
43
44         if opt_standard_gain == 0:
45             approximation_ratio = None
46         else:
47             approximation_ratio = final_standard_gain / opt_standard_gain
48
49         sample = {'approximation_ratio':approximation_ratio,
50         'opt_standard_solution':opt_standard_solution,
51         'final_standard_solution':final_standard_solution,}
52
53         samples[key][str(ins)] = sample
54         # save the json and compress it
55         with open('./EvaluationQAQASolver_%s_instance%s.json'%(str(filename),str(instance)), 'w',
56         encoding='utf-8') as file:
57             json.dump(samples, file, ensure_ascii=False, indent=4)

```

Código B.8: Implementación de la clase `EvaluationQAQASolver`, encargada de realizar la aplicación de Aprendizaje por Transferencia.